

Drone Command Center
Autonomous Fleet Management System with Hybrid Reinforcement-Learning
Navigation

Mohamad Alayli, 202301662

Dr. Seifedine Kadry

Department of Computer Science & Mathematics

Capstone

Spring 2026

May 11, 2026

Preface

About This Document

This document is the consolidated capstone report for the Drone Command Center (DCC) project. It covers the full end-to-end system backend services, mobile frontend, database schema, and an AI navigation layer that wraps the published NavRL deep reinforcement-learning framework [1] inside a hybrid planner suitable for structured urban environments simulated in Microsoft AirSim.

Acknowledgements

We acknowledge the authors of the NavRL framework, Z. Xu, X. Han, H. Shen, H. Jin, and K. Shimada at Carnegie Mellon University, whose published PPO-based navigation policy and LiDAR-based velocity-obstacle safety shield form the foundation of the AI layer evaluated in this work [1]. Microsoft Research is acknowledged for the AirSim simulator and the Unreal Engine 4 City environment used for evaluation.

Scope of This Document

The document describes the system as actually implemented all REST endpoints, database tables, Flutter screens, configuration keys, and AI modules referenced are present in the Drone Command Center repository at the time of submission. Items that have not yet been delivered are explicitly listed under Roadmap in Table 42 and revisited in Chapter 9 (System Evolution and

Roadmap).

Abstract

The Drone Command Center (DCC) is an autonomous fleet-management platform consisting of (i) a Spring Boot 4.0.2 backend exposing a versioned REST API, real-time WebSocket telemetry, stateless JWT authentication, and Hibernate-managed PostgreSQL persistence; (ii) a Flutter cross-platform frontend providing tactical dashboards, mission planning, drone telemetry visualization, and an interactive map; and (iii) an AI navigation layer that integrates the published NavRL deep reinforcement-learning framework [1] with a custom hybrid city planner combining A* global path planning, a city altitude state machine, Pure-Pursuit lookahead, and a velocity-obstacle (VO) safety shield.

The platform satisfies 45 functional and 35 non-functional requirements, spans 8 database tables governed by versioned migrations, exposes 8 REST endpoint groups plus a STOMP/SockJS WebSocket channel, and is delivered alongside a docker-compose stack that brings up the entire local development environment with a single command.

The AI layer was systematically benchmarked in Microsoft AirSim against a 12-waypoint urban roam mission spanning approximately 800 m of dense city terrain. The hybrid controller achieved a 75.00% goal-success rate at 0.69 collisions/km, compared with 36.66% / 9.31 collisions/km for pure NavRL alone a 2× improvement in success and a 13× reduction in collision rate, validating the

central thesis that structured urban navigation requires global planning combined with reactive RL control.

This consolidated report documents the architecture, requirements, models, implementation, testing strategy evaluation results, entrepreneurial positioning, project management, and long-term roadmap of the integrated system.

Table of Contents

Preface	ii
About This Document	ii
Acknowledgements	ii
Scope of This Document	ii
Abstract	iii
1. Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Project Objectives	4
1.4 Scope	6
1.5 Significance and Document Organization	7
2. Literature Review	8
2.1 Drone Fleet-Management Platforms	9
2.1.1 Commercial and Open-Source Landscape	9
2.1.2 Modern Backend Architectures for IoT and Robotics	11
2.2 Traditional UAV Navigation	13
2.2.1 Classical Planning and Reactive Control	13
2.2.2 UAV Physical Modelling, Dynamics, and the Role of CAD	14
2.3 Deep Reinforcement Learning for UAV Navigation	16
2.3.1 From Q-Learning to Continuous-Control PPO	16
2.3.2 The NavRL Framework	17

2.3.3 Known Limitations of End-to-End DRL	19
2.4 Sim-to-Real Transfer.....	21
2.5 Hybrid Architectures and the Position of This Work.....	22
3. System Analysis & Design	24
3.1 Stakeholders & User Stories.....	24
Authentication and Identity (US-01 ... US-04).....	24
Drone Fleet (US-05 ... US-09).....	25
Mission Planning (US-10 ... US-15).....	25
Command and Control (US-16 ... US-19).....	26
Telemetry and Sensors (US-20 ... US-23).....	26
Operations (US-24 ... US-26).....	26
3.2 Use Cases.....	27
• UC Cluster A, Authentication.....	27
• UC Cluster B, Fleet Operations.....	28
• UC Cluster C, Mission Management.....	28
3.3 Functional & Non-Functional Requirements.....	29
3.3.1 Functional Requirements Authentication and User Management.....	29
3.3.2 Functional Requirements Drone Fleet Management.....	30
3.3.3 Functional Requirements Mission Management.....	31
3.3.4 Functional Requirements Command and Control.....	33
3.3.5 Functional Requirements Telemetry and Sensor Monitoring.....	34
3.3.6 Functional Requirements System Operations.....	35
3.3.7 Non-Functional Requirements Performance	36
3.3.8 Non-Functional Requirements Security.....	37
3.3.9 Non-Functional Requirements Reliability and Availability.....	38
3.3.10 Non-Functional Requirements Usability.....	38
3.3.11 Non-Functional Requirements Scalability and Maintainability	39
3.3.12 Non-Functional Requirements Portability.....	40
3.4 Four-Tier System Architecture	40

3.5 Backend Component Design	42
3.5.1 Five-Layer Security Model	43
3.5.2 Endpoint Authentication Matrix.....	44
3.6 Frontend Component Design.....	45
3.6.1 Riverpod State Topology	45
3.6.2 Tactical Design Tokens.....	46
3.7 Deployment Architecture.....	47
3.8 NavRL Theoretical Foundations	47
3.8.1 Markov Decision Process.....	48
3.8.2 Internal State.....	48
3.8.3 Lidar Observation.....	48
3.8.4 Action Distribution	49
3.8.5 World-Frame Transformation.....	50
3.8.6 Reward Components.....	50
3.8.7 PPO Clipped Objective.....	51
3.8.8 Curriculum.....	51
3.8.9 Velocity-Obstacle Safety Shield.....	53
3.8.10 Published Performance	54
3.9 Hybrid City-Planner Design.....	54
3.9.1 Pure-Pursuit Lookahead.....	55
3.9.2 Altitude State Machine.....	56
3.10 Hardware Design (CAD Model):.....	56
3.10.1 List of Parts.....	56
3.11 System Models.....	60
3.11.1 Entity-Relationship Model.....	60
3.11.2 Sequence User Authentication.....	61
3.11.3 Sequence Goal Dispatch and Auto-Mission Creation	61
3.11.4 Sequence Real-Time Telemetry.....	62
3.11.5 Telemetry Pipeline.....	62

3.11.6 Feasibility and Risk Analysis.....	62
4. Implementation.....	63
4.1 Development Methodology and Tooling.....	63
4.2 Backend Implementation (Spring Boot).....	64
4.2.1 Module Map.....	64
4.2.2 Authentication Path.....	65
4.2.3 Validation, Errors, and Audit.....	65
4.3 Frontend Implementation (Flutter).....	66
4.3.1 Routing and Guards.....	66
4.3.2 Networking Layer.....	66
4.3.3 Real-Time Telemetry.....	66
4.3.4 Map and Mission Visualization.....	67
4.4 AirSim Bridge and NavRL Integration.....	67
4.4.1 Component Overview.....	67
4.4.2 Inference Loop.....	68
4.4.3 Sensor Noise Injection.....	68
4.4.4 Database Schema and Migrations.....	69
5. Testing and Evaluation.....	69
5.1 Test Strategy Overview.....	69
5.2 Backend Testing.....	71
5.2.1 Unit Tests Service Layer.....	71
5.2.2 REST API Contract Testing (Postman).....	72
5.2.3 OWASP ZAP Baseline Security Scan.....	78
5.2.4 JaCoCo Backend Test Coverage.....	79
5.3 Frontend Testing.....	82
5.3.1 Unit Tests.....	82
5.3.2 Widget Tests.....	83
5.4 NavRL + CityPlanner Quantitative Evaluation.....	90
5.4.1 Simulation Environment and Test Mission.....	90

5.4.2 Test Suites.....	92
5.4.3 Standard Suite Primary Benchmark.....	92
5.4.4 Ablation Study.....	95
5.4.5 Domain Randomization Suite.....	96
5.4.6 Sensor-Noise Suite.....	98
5.5 CAD Model Test:.....	99
5.6 Test Coverage Summary.....	105
6. Entrepreneurial and Innovation Aspects.....	106
6.1 Market and Research Relevance.....	106
6.2 Innovation Positioning.....	107
6.3 Ethical and Societal Considerations.....	108
6.4 Known Deviations from Requirements.....	108
7. Project Management and Teamwork.....	109
7.1 Methodology and Working Cadence.....	109
7.2 Timeline and Milestones.....	110
7.3 Resource Management.....	113
7.4 Team Roles.....	115
7.5 Communication, Tooling, and Lessons Learned.....	117
8. Results and Discussion.....	118
8.1 Why Pure RL Fails at the City Scale.....	118
8.2 The Hybrid Architecture's Trade-offs.....	119
8.3 Altitude as a Navigation Dimension.....	120
8.4 Hardware Design for Real-World Deployment.....	121
8.5 Limitations.....	122
8.5.1 Autonomy-stack limitations.....	122
8.5.2 Software-platform limitations:.....	123
8.5.3 Sim-to-real gap:.....	123
9. Conclusion, Future Work and Roadmap.....	124
9.1 Summary of Contributions.....	124

9.2 Future Work.....	125
9.3 Roadmap	126
10. References.....	127
Appendix A Technology Stack.....	133
A.1 Backend Dependencies.....	133
A.2 Frontend Dependencies	134
Appendix B REST API Endpoint Catalogue.....	136
B.1 Authentication (/api/auth).....	136
B.2 Drone Management (/api/drones).....	137
B.3 Mission Management (/api/missions).....	138
B.4 Commands (/api/commands).....	139
B.5 Telemetry (/api/telemetry).....	140
B.6 User Management (/api/users).....	141
B.7 System (/actuator).....	141
B.8 AirSim Bridge / NavRL / Logs.....	142
B.9 WebSocket.....	142
Appendix C Database Schema Reference.....	143
C.1 Table: users.....	143
C.2 Table: refresh_tokens	144
C.3 Table: drones.....	145
C.4 Table: missions	148
C.5 Table: waypoints.....	149
C.6 Table: telemetry.....	151
C.7 Table: sensors.....	152
C.8 Table: commands.....	153
Appendix D Enumeration Reference.....	154
Appendix E Frontend Screen Inventory.....	155
Appendix F Configuration Reference.....	156
F.1 Key application.properties Parameters	156

F.2 Environment Variables (Secrets).....	158
Appendix G Document Control	159

1. Introduction

1.1 Background and Motivation

Unmanned aerial vehicles (UAVs) have transitioned from experimental platforms into operational infrastructure. They are now deployed routinely for power-line inspection, precision agriculture, parcel delivery, search-and-rescue coordination, and urban security patrol [2]. Market analysts project the global drone-services market will reach USD 63.6 billion by 2030, driven primarily by last-mile delivery, infrastructure inspection, and urban air mobility [3]. Operating a fleet of such vehicles at scale rather than a single manually piloted airframe introduces three distinct and largely unsolved engineering challenges that motivate this work.

Challenge 1 Fleet Management Infrastructure. Scaling from one drone to many demands a software platform capable of registering heterogeneous hardware, supervising concurrent missions with persistent waypoint sequences, issuing and auditing commands in real time, ingesting high-frequency telemetry from multiple airframes simultaneously, and authenticating a diverse operator base against a hardened API. Existing commercial offerings DJI FlightHub 2, Skydio Cloud, AirData UAV, and Auterion Suite address portions of this problem but are vendor-locked, closed-source, and priced on a per-airframe SaaS model that is prohibitive for academic labs, small rescue teams, and early-stage start-ups [2]. Open-source counterparts such as QGroundControl and the PX4 stack focus on single-airframe MAVLink piloting and lack first-class REST APIs, versioned schema management, or audit-grade command histories suitable for multi-operator deployments [6]. A side-by-side feature analysis against the

requirements derived in Chapter 3 (Table 2–7) reveals that no publicly available platform combines, in a single coherent codebase, binary JWT authentication over a versioned REST API, a migration-controlled persistence layer that survives schema evolution, and an integrated AI navigation interface.

Challenge 2 Urban Autonomous Navigation. Classical autonomous navigation pipelines occupancy-grid SLAM combined with sampling-based planners (PRM, RRT, RRT*) and trajectory-optimization controllers (MPC, B-spline smoothing) [20] produce near-optimal trajectories in static, well-mapped environments [13]. However, they degrade predictably in dense or dynamic urban scenes, where re-planning latency and local-minima entrapment behind extended façades dominate performance [13], [4]. Deep reinforcement-learning (DRL) approaches address these failure modes: reactive policies trained via PPO or SAC [24] operate from raw sensor input without explicit maps, are robust to partial observability, and have demonstrated successful sim-to-real transfer [1]. The NavRL framework [1], the state-of-the-art method adopted in this work, achieves up to 96% success in sparse static environments but its success rate falls below 40% when evaluated on dense procedurally generated city geometry, because its 4 m reaction horizon cannot route around buildings that subtend 30° – 90° of horizontal LiDAR coverage (replicated and quantified in §5.4). Re-training the policy on new geometry requires hundreds of GPU-hours across 1,024 parallel agents [1], making environment-specific retraining impractical for most operators.

Challenge 3 System Integration and Physical Realizability. A navigation policy evaluated only in simulation, on hardware it will never run on, provides limited

engineering assurance. The controller's commanded velocity range, admissible acceleration envelope, and inference latency budget are all bounded by the physical constants of the target airframe constants that must be derived from a physical model before any controller can be responsibly tuned [19]. Without an explicit hardware model, the gap between simulated performance and real-world deployability remains an act of faith.

This capstone takes the position that a practical drone-control system must close all three challenges simultaneously: a robust full-stack management platform, an autonomy layer that combines a frozen learned reactive policy with classical global planning and a velocity-obstacle safety shield, and a hardware model that grounds the autonomy stack in physical realizability. A high-level view of the resulting integrated system is presented in Figure 8 (System Overview) and unpacked in Chapter 3.

1.2 Problem Statement

The three challenges above give rise to three formally separated research questions, each addressed by a distinct component of this work and each evaluated by an independent validation method in Chapter 5:

- **P1 Software Engineering Problem.** Can a single open-source backend platform satisfy enterprise-ready architectures, for REST fleet management, real-time WebSocket telemetry, stateless JWT security, and schema-versioned persistence, while remaining small enough to be deployed and extended by a small team without vendor infrastructure?

Validated by: functional and non-functional requirements compliance

testing, REST contract coverage (Postman, 48 endpoints), security audit (JWT integrity, rate limiting, OWASP header verification), and WebSocket latency measurement against NFR-02 (≤ 1 s).

- P2 Autonomous Navigation Problem. Can a hybrid architecture wrapping the published, frozen NavRL policy inside a deterministic city planner without retraining substantially outperform pure end-to-end DRL in both task success rate and collision rate on a structured 12-waypoint urban mission in Microsoft AirSim? Validated by: paired quantitative evaluation across a standard suite ($n = 5$), an ablation study (5 controller variants $\times n = 5$), a domain-randomization suite (5 weather conditions), and a sensor-noise suite (4 LiDAR degradation levels $\times n = 5$).
- P3 Physical Realizability Problem. Can the autonomy stack's commanded velocity range and inference latency budget be grounded in a physically credible airframe model, demonstrating that real-world deployment is mechanically and computationally feasible? Validated by: a Fusion 360 CAD model with interference, structural thickness, mass/CoG, and aerodynamic drag analysis; and a full hybrid-stack runtime budget estimate on the target NVIDIA Jetson Orin NX [46] compute platform (Table 31).

Together, these three questions define the engineering scope of the Drone Command Center (DCC) and constitute the evaluation criteria against which the system is assessed in Chapter 5.

1.3 Project Objectives

Mapped directly to the three problem statements above, the project pursues five objectives:

- O1 Backend Platform (addresses P1). Design and implement a Spring Boot 4.0.2 service exposing 9 REST endpoint groups (authentication, drone fleet, missions, commands, telemetry, users, AirSim bridge, NavRL, logs), a STOMP/SockJS WebSocket telemetry channel, stateless JWT authentication with Bucket4j rate limiting, and a Flyway-versioned PostgreSQL schema across 8 tables, satisfying the 45 functional and 35 non-functional requirements catalogued in §3.3.
- O2 Frontend Application (addresses P1). Design and implement a Flutter cross-platform operator console supporting 11 screens (splash, login, register, dashboard, drone list/detail, mission list/detail, map, settings), Riverpod state management, Dio HTTP client with interceptor-based token refresh, WebSocket-driven Real-Time Telemetry visualization, and an interactive OpenStreetMap mission overlay.
- O3 AI Navigation Layer (addresses P2). Integrate the published NavRL policy [1] used unchanged, without retraining with a custom four-layer hybrid city planner comprising A* global path planning, an altitude state machine, Pure-Pursuit lookahead sub-goal injection, and a velocity-obstacle safety shield. Evaluate the complete stack on a fixed 12-waypoint urban roam mission in the Microsoft AirSim UE4 City environment.

- O4 Quantitative Validation (addresses P2). Demonstrate through a paired ablation study and domain-randomization and sensor-noise robustness suites that the hybrid controller substantially outperforms pure NavRL on both task success rate and collision rate, with effect sizes sufficient to support the central engineering thesis.
- O5 Hardware Integration Model (*addresses P3*). Produce a Fusion 360 CAD model of the target airframe (DJI F450 configuration with NVIDIA Jetson Orin NX [46] payload bay) and extract the mass, inertia tensor, center-of-gravity offset, and aerodynamic drag parameters used by the AirSim vehicle configuration and the dynamic model of §2.2, demonstrating the physical realizability of the autonomy stack at the commanded velocity envelope.

1.4 Scope

In Scope

- Backend 9 REST endpoint groups, WebSocket telemetry, stateless JWT + Bucket4j rate limiting, PostgreSQL schema (8 tables, Hibernate-managed; V1 and V2 Flyway migrations applied during setup; runtime migration disabled per §4.4 with rationale).
- Frontend 11 screens, Riverpod state management, Dio HTTP client, `web_socket_channel`, `fl_chart`, `flutter_map`.

- AI Integration AirSim 1.8.1 + UE4 City environment; NavRL policy wrapped via nav_worker.py; hybrid planning orchestrated by navrl_city_planner.py.
- Quantitative Evaluation Paired ablation suite, sensor-noise suite, weather/domain-randomization suite; ≥ 30 total trials per condition; full statistical tables and Figures.
- Hardware Design Artefact Fusion 360 CAD model of airframe and Jetson Orin NX [46] payload bay (proof-of-concept; no manufactured prototype).

Out of Scope (Deferred to Roadmap, §9.3)

- Real-hardware flight tests on physical UAVs.
- Multi-tenant SaaS deployment and Kubernetes Helm-chart packaging.
- Live video streaming (WebRTC / HLS) per drone.
- Regulatory compliance modules (FAA Part 107, EASA U-Space).
- Geofence enforcement runtime engine (data model only is in scope).

The explicit out-of-scope boundary ensures that each in-scope deliverable can be fully validated within the project timeline, and that deferred items are traceable to the roadmap in §9.3 rather than silently absent.

1.5 Significance and Document Organization

The Drone Command Center delivers three compounding contributions. First, it is, to the best of the authors' knowledge, the only open-source platform that combines a versioned REST fleet-management API, real-time WebSocket

telemetry, and an integrated AI navigation interface in a single deployable codebase directly filling the feature gap identified in §1.1 and Table 1 (Chapter 2). Second, it demonstrates that a frozen, published RL policy can be augmented with a classical city planner to achieve a 2× improvement in goal-success rate and a 13× reduction in collision rate on structured urban terrain, without retraining and without proprietary infrastructure a result with direct commercial significance for operators who cannot absorb the GPU cost of domain-specific policy retraining. Third, the reproducible test harness (`capstone_test_runner.py` + `analyze_results.py`) produces deterministic CSV summaries and Figures from a single command, lowering the barrier for follow-on research to compare alternative planners against the same baseline.

The platform directly benefits three stakeholder groups: (i) academic UAV research labs requiring a free, inspectable alternative to FlightHub 2 and Skydio Cloud; (ii) low-budget search-and-rescue and humanitarian-response teams who need multi-drone fleet supervision without per-airframe fees; and (iii) early-stage robotics start-ups that need a defensible, well-tested engineering baseline on which to build a commercial product. The entrepreneurial implications are developed fully in Chapter 6.

Document Organization. Chapter 2 reviews related literature across fleet management, classical navigation, deep reinforcement learning, and hybrid architectures. Chapter 3 presents the full analysis and design phase: stakeholders, use cases, the FR/NFR catalogue, four-tier architecture, NavRL theoretical foundations, hybrid planner design, and the Fusion 360 hardware

model. Chapter 4 describes the implementation. Chapter 5 presents the multi-layer test and evaluation strategy, including the quantitative AI evaluation and its statistical analysis. Chapters 6 through address entrepreneurial positioning, results and discussion, and conclusions with future work. Appendices A through G provide the technology stack, REST endpoint catalogue, database schema reference, enumeration definitions, screen inventory, and configuration reference.

2. Literature Review

This chapter situates the Drone Command Center (DCC) within four bodies of literature: (i) commercial and open-source drone fleet-management platforms and the backend architectures that underpin them, (ii) classical UAV navigation and the physical modelling that drives controller design, (iii) deep reinforcement learning for UAV navigation, and (iv) sim-to-real transfer strategies. The chapter closes with a synthesis of hybrid autonomy architectures that establishes the novel positioning of this work and identifies the specific gap it fills.

2.1 Drone Fleet-Management Platforms

2.1.1 Commercial and Open-Source Landscape

Commercial fleet-management platforms DJI FlightHub 2, Skydio Cloud, AirData UAV, and Auterion Suite each address portions of the multi-drone supervision problem. DJI FlightHub 2 provides live map tracking, remote diagnostics, and mission replay for DJI hardware exclusively; its underlying API schema and storage layer are closed to operators [4]. Skydio Cloud offers autonomous

obstacle-avoidance flights and a web dashboard but is architecturally coupled to Skydio airframes and provides no third-party REST surface for custom UI development. AirData UAV aggregates post-flight telemetry logs from multiple manufacturers but operates as a passive analytics layer, not a real-time command and control platform. Auterion Suite is the most enterprise-featured offering, exposing a MAVLink-compatible API and role-based access control, but its pricing model a per-airframe, per-month SaaS subscription places it out of reach for academic labs, small rescue teams, and early-stage start-ups [5].

Open-source counterparts occupy a different point in the design space.

QGroundControl (QGC), maintained by the Dronecode Foundation [7], and the PX4 autopilot stack [6] are the dominant open-source tools for MAVLink-based UAV operation. Both excel at single-airframe manual and semi-autonomous piloting: QGC provides a rich ground-control station UI with real-time HUD and parameter editing, while PX4 closes the inner attitude-control loop at 400 Hz on supported flight controllers. However, neither was designed for multi-user, multi-drone fleet supervision. There is no first-class concept of a persistent mission entity with an ordered waypoint sequence, no audit-grade command history with per-user attribution, no RESTful API surface for third-party integrations, and no schema-versioned persistence layer that survives software upgrades without manual database intervention.

A structured feature comparison against the 45 functional requirements derived in Chapter 3 is presented in Figure 1). The analysis reveals that no publicly available platform commercial or open-source combines all of the following in a

single coherent, self-hostable codebase: (i) stateless JWT authentication over a versioned REST API, (ii) a migration-controlled persistence layer that survives schema evolution without data loss, and (iii) an integrated AI navigation interface with a velocity-obstacle safety shield. The DCC is positioned to fill this gap as an open, academically reproducible reference platform that is compact enough to be read end-to-end by a graduate student and complete enough to form the foundation of a commercial product.

Feature	DJI FlightHub 2	Skydio Cloud	AirData	Auterion Suite	QGroundControl	PX4	DCC (this work)
Open source	✗	✗	✗	✗	✓	✓	✓
Versioned REST API	✗	✗	✗	✓	~	~	✓
WebSocket telemetry	✗	✗	✗	~	~	✗	✓
JWT authentication	✗	✗	✗	~	~	✗	✓
Multi-drone management	✓	✓	✓	✓	✓	✗	✓
Mission entity persisted	✓	✗	✓	✓	~	✗	✓
AI navigation built-in	✓	✗	✗	✗	~	✗	✓
VO safety shield	✗	✗	✗	✓	~	✗	✓

Figure 1 Feature matrix highlighting the gap addressed by the DCC.

2.1.2 Modern Backend Architectures for IoT and Robotics

- The choice of backend technology for a real-time fleet management platform is not architecturally neutral. Three established patterns exist in the IoT and robotics literature, and their trade-offs are relevant to the DCC's design decisions.
- Monolithic embedded controllers (e.g., C++/ROS2 nodes running directly on the companion computer) minimize round-trip latency and avoid

network dependency, but couple the business logic to the hardware platform, make multi-user access control difficult to layer cleanly, and resist horizontal scaling [8]. This architecture is well suited to single-vehicle research platforms but is inappropriate for multi-drone fleet supervision.

- Microservice mesh architectures (e.g., Kubernetes-orchestrated gRPC services) offer maximum scalability and fault isolation but introduce significant operational complexity: service discovery, distributed tracing, and inter-service authentication all require dedicated infrastructure [8]. For a platform targeting academic labs and small operators without DevOps teams, this complexity is a barrier to adoption rather than a benefit.
- Modular monoliths with layered REST + WebSocket represent the pragmatic middle ground widely adopted by mid-market IoT SaaS products banking back-offices, energy-metering platforms, and industrial telemetry systems alike [8], [11]. Spring Boot has emerged as the leading Java framework for this pattern because it (a) ships an embedded servlet container that containerizes with a single Dockerfile, (b) integrates first-class WebSocket support (RFC 6455) for low-latency telemetry push, (c) exposes Spring Actuator endpoints for production observability without additional tooling, and (d) provides a clean migration path toward microservices via Spring Cloud without forcing the architectural commitment upfront [11]. Critically, Spring Boot's security stack (Spring Security with stateless JWT) and schema-migration support (Flyway) are

mature, well-documented, and independently audited properties that justify the "enterprise-grade" label used in Chapter 1 and that would require significant custom engineering to replicate on a bespoke C++/MAVLink stack.

- Transport layer selection for Real-Time Telemetry involves a three-way trade-off among latency, bandwidth, and client-side compatibility. MQTT over TCP achieves the lowest bandwidth through binary framing and QoS guarantees but requires a broker process and has limited native support in browser-based clients [8]. gRPC streaming provides strongly typed contracts and efficient Protobuf encoding but imposes a binary-codec overhead that is difficult to consume from Flutter Web without additional bridging [11]. Raw WebSocket (RFC 6455) [9] offers the lowest latency among the three and enjoys first-class browser and Flutter support via the `web_socket_channel` package. Given the DCC's dominant client (Flutter Web/desktop) and a message volume of approximately 10 Hz per drone well below the threshold at which gRPC's binary efficiency becomes advantageous WebSocket is the justified choice.
- Persistence for telemetry-heavy IoT workloads has likewise consolidated around PostgreSQL with optional JSONB columns, which combines relational guarantees (foreign keys, transactions, schema migrations) with the semi-structured flexibility needed for sensor payloads of variable shape [10]. The DCC adopts this pattern across eight Hibernate-managed

tables, supplemented by Flyway versioned migrations for controlled schema evolution.

2.2 Traditional UAV Navigation

2.2.1 Classical Planning and Reactive Control

Classical autonomous navigation pipelines operate on three nested temporal scales. At the slowest scale (≈ 1 Hz), a SLAM algorithm such as LOAM, ORB-SLAM3, or FAST-LIO2 [16], [17], [18] maintains a consistent occupancy map from sensor data. At an intermediate scale (≈ 1 – 5 Hz), a global planner typically a sampling-based method such as PRM, RRT, or RRT* computes a collision-free path through the map [13]. At the fastest scale (≈ 20 – 50 Hz), a reactive controller artificial potential fields (APF) [4], dynamic window approach (DWA) [14], or velocity obstacles (VO) [15] handles moving obstacles and trajectory deviations between planning cycles.

Each component has well-characterized failure modes. Sampling-based planners such as RRT* are probabilistically complete and asymptotically optimal under accurate maps but suffer from high re-planning latency in dynamic environments where the map changes faster than the planner can recompute [13]. APF controllers are computationally cheap and real-time safe but are prone to local minima: a drone approaching a concave obstacle such as a building recess can become trapped in a potential well with no gradient toward the goal [12]. DWA and VO handle dynamic obstacles more gracefully by reasoning over future

velocity trajectories, but their planning horizon is bounded by the sensor range, leaving them blind to obstacles beyond it [14], [15].

In dense procedurally generated urban environments the evaluation domain of this work all three failure modes are simultaneously active: the map updates slowly relative to the drone's speed, buildings create large concave local-minima regions, and the sensor range (4 m for the NavRL LiDAR configuration) is far shorter than the scale of the obstacles (multi-story buildings of 10–30 m). It is precisely this compound failure that motivates the adoption of a *learned* reactive layer (§2.3) layered on top of a *classical* global planner (Chapter 3): the classical planner eliminates the global topology problem that the reactive layer cannot solve, while the learned policy handles dynamic local avoidance that the classical planner cannot anticipate.

2.2.2 UAV Physical Modelling, Dynamics, and the Role of CAD

Every classical controller PID, LQR, MPC alike is parameterized by the physical constants of the airframe it commands. The standard quadrotor model expresses linear and angular dynamics as

$$m\ddot{\mathbf{p}} = \sum_{i=1}^4 \mathbf{f}_i + m\mathbf{g} - \mathbf{D}\dot{\mathbf{p}}$$

$$\mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) = \sum_{i=1}^4 \boldsymbol{\tau}_i$$

where m is the take-off mass, $\mathbf{J}$ the moment-of-inertia tensor, $\mathbf{D}$ a linear-drag coefficient, and $\{\mathbf{f}_i, \mathbf{T}_i\}$ the rotor forces and body-torques. The control law's

stability margins, the maximum admissible commanded velocity, the achievable hover endurance, and the safe minimum altitude over a given payload mass all depend on these constants; all of them must be measured or derived from a physical model before a controller can be tuned [13].

For this reason, this work includes an explicit physical-modelling stage: a Fusion 360 CAD model of the target airframe (see §3.10) is used to estimate take-off mass, the principal moments of inertia J_{xx} , J_{yy} , J_{zz} , the centre-of-mass offset under a Jetson-Orion-NX payload, and the propeller-rotor arm length. These constants establish the bounded-velocity range that the NavRL Beta-distribution action head (§2.3) must respect, and they inform the runtime budget of the safety shield (§5.4). Without such a model, the autonomy stack would be evaluated only in simulation and the physical realizability of its commanded velocities would remain an act of faith exactly the gap that classical robotics literature has long warned against [13], [21].

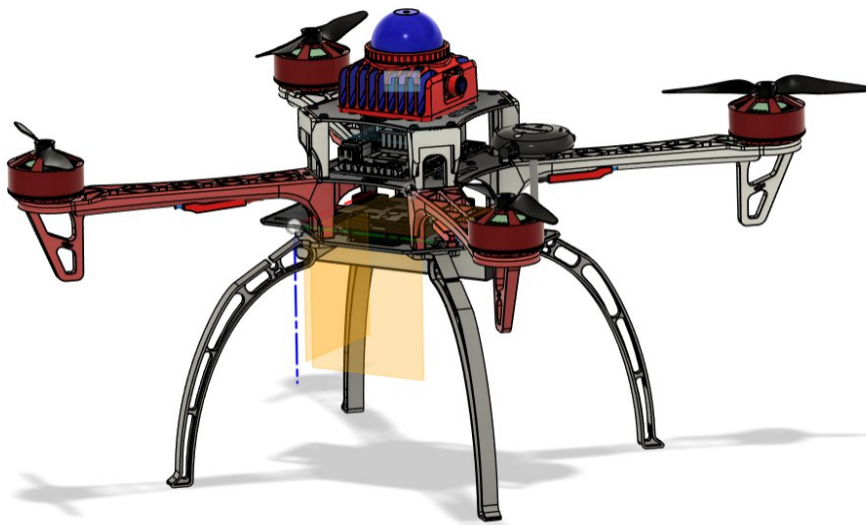


Figure 2 Fusion 360 CAD of the prototype airframe. X-configuration plastic fiber glass frame, four brushless motor mounts, central avionics tray housing the flight controller, small computer and lidar device (blue dome), and the battery bay.

Used to extract mass and arm-length parameters for the dynamic model.

2.3 Deep Reinforcement Learning for UAV Navigation

2.3.1 From Q-Learning to Continuous-Control PPO

Deep reinforcement learning has produced increasingly capable UAV navigation policies over the past decade. Early DQN-based approaches discretized the action space into coarse heading and altitude bins and operated from compressed visual observations [22]; they demonstrated proof-of-concept obstacle avoidance but could not generalize beyond the specific obstacle densities of their training environments. The A3C algorithm [23] enabled asynchronous parallel training that substantially improved sample efficiency, making it feasible to train policies in simulation at the scale required for urban navigation research.

Modern UAV navigation research has converged on two continuous-control algorithms: Proximal Policy Optimization (PPO) [24] and Soft Actor-Critic (SAC) [25]. Both operate directly over continuous velocity command spaces three translational axes plus optional yaw and are stable under the partial observability typical of LiDAR-only sensing, where the drone cannot observe obstacles beyond the sensor's maximum range. PPO's clipped surrogate objective prevents destructive policy updates during training, making it more suitable for curriculum-

based training schedules where environment difficulty increases progressively. SAC's entropy-maximization objective encourages broader exploration and can achieve higher asymptotic performance, but at the cost of greater sensitivity to reward shaping. NavRL adopts PPO for these reasons [1], and the theoretical foundations of its objective function are developed in §3.8.

2.3.2 The NavRL Framework

Among UAV-targeted DRL methods published through 2025, NavRL [1] is one of the most thoroughly characterized in terms of both training methodology and real-world validation. Its architecture combines three design choices that distinguish it from earlier reactive policies.

First, it uses a 36-ray, 360° horizontal LiDAR observation reshaped into a 35×4 temporal history matrix, encoding both spatial and temporal obstacle information. Crucially, raw range readings are inverted closer obstacles produce larger activations making the gradient signal from nearby obstacles stronger and facilitating faster learning of avoidance behavior.

Second, the actor network outputs the parameters (α, β) of a Beta distribution rather than the mean and variance of a Gaussian [26]. This choice is motivated by the bounded nature of the velocity command space: a Beta distribution on $[0, 1]$ has compact support, eliminating the clipping artefacts that arise when a Gaussian policy places probability mass outside the admissible velocity range. The exploration variance is controlled jointly through (α, β) rather than through a

separately maintained σ , removing one hyperparameter from the training procedure.

Third, NavRL's dense reward function shapes five simultaneous behaviors: goal-seeking progress, smooth trajectories (jerk minimization), obstacle clearance, altitude maintenance, and terminal goal-reaching. This multi-component shaping substantially accelerates curriculum progression relative to sparse reward formulations.

On its own published evaluation suite [1], the trained policy attains near-saturation success in sparse static environments (96%) and maintains acceptable performance in dense dynamic scenarios (82%). It demonstrates successful sim-to-real transfer onto a physical quadrotor with NVIDIA Jetson Orin NX [46] on-board inference at a total pipeline latency of 65 ms within the 50 ms control budget at 20 Hz.

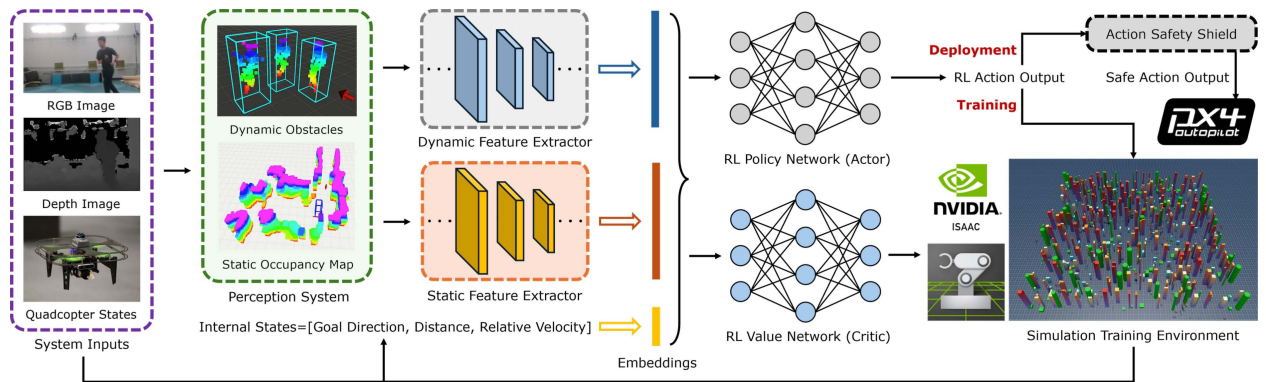


Figure 3 The proposed NavRL framework. The perception system processes RGB-D images along with the robot's internal states to generate representations for both static and dynamic obstacles. These

representations are then fed into two feature extractors, which produce state embeddings concatenated with the robot's internal states. In the training phase, an actor-critic network structure is utilized to train robots in parallel within the NVIDIA Isaac Sim environment. During the deployment stage, the policy network generates actions that are further refined by a safety shield mechanism to ensure safe robot control [1].

Table 1

BENCHMARK OF THE AVERAGE NUMBER OF COLLISIONS EVALUATED FROM 20 SAMPLE RUNS IN DIFFERENT TYPES OF ENVIRONMENTS [1]

Average Collision Times Measurement in a 20m×40m map			
BENCHMARKS	Static Env.	Dynamic Env.	Hybrid Env.
EGO [48]	0.45 (56.3%)	N/A	N/A
ViGO [20]	0.80 (100%)	3.15 (100%)	4.40 (100%)
	0.95 (118.8%)	2.70 (85.7%)	4.60 (104.5%)

OURS W/O DMC			
OURS (NAVRL)	0.65 (81.3%)	0.85 (27.0%)	4.10 (47.8%)

2.3.3 Known Limitations of End-to-End DRL

Despite these strengths, end-to-end DRL navigation is known to suffer three structural weaknesses that are directly relevant to structured urban deployment.

No global goal-reaching guarantee. A reactive policy maps the current LiDAR observation to a velocity command with no notion of the global topology connecting the drone's position to the goal. When an obstacle subtends the full forward hemisphere as multi-story buildings commonly do in urban environments the policy has no information from which to derive a detour direction. It will spend its episode budget circling the obstacle until the step limit is reached. This failure mode is not a training artefact; it is a structural consequence of the policy's Markovian observation, which contains no map or goal-path information [13].

Trapping behind extended barriers. Building façades in dense urban environments commonly subtend 30° – 90° of horizontal LiDAR coverage across the NavRL sensor's 4 m range. When the goal is occluded behind such a structure, the potential-field gradient toward the goal is obscured by a wall of near-maximum obstacle activations, leaving no clear steering signal. Empirically, this reduces NavRL's urban success rate to below 40% a figure replicated and

quantified in §5.4 compared with 96% in the sparse environments of the original evaluation suite [1].

Distribution mismatch on procedurally generated city geometry. The NavRL policy was trained in a 50 m × 50 m arena with sparse random obstacles.

Voxelized LIDAR returns from the photorealistic UE4 City environment procedurally generated buildings with non-textured, self-similar surfaces lie outside this training distribution. Adapting the policy to this geometry via retraining would require hundreds of GPU-hours across 1,024 parallel agents [1], a cost that is prohibitive for most operators and that defeats the purpose of adopting a pre-trained reactive policy.

These three weaknesses are not deficiencies of NavRL specifically; they are general properties of any reactive end-to-end DRL approach operating without global map information [13]. A wrapper-based augmentation strategy keeping the trained policy frozen, supplying it with locally smoothed sub-goals from a classical global planner, and gating its outputs through a safety shield is the engineering-pragmatic response that avoids retraining while restoring global goal-reaching capability. This is the strategy formalized in §2.5.

2.4 Sim-to-Real Transfer

The simulation-to-reality gap is the dominant threat to external validity for any autonomy system evaluated exclusively in simulation. Two complementary strategies have emerged as the field standard for mitigating it.

- Domain randomization [27] augments the training distribution by randomly sampling environment parameters obstacle density, obstacle texture, lighting conditions, and background geometry at the start of each training episode, forcing the policy to develop representations that are invariant to surface appearance rather than overfit to a specific simulated texture. This is the primary mechanism by which NavRL achieves its reported sim-to-real transfer [1].
- Dynamics randomization [28] addresses the mismatch between simulated and real-world physics by randomly perturbing mass, drag coefficients, rotor thrust constants, and control latency during training. Policies trained with dynamics randomization are demonstrably more robust to aerodynamic effects, motor degradation, and sensor lag in physical deployment all of which are impossible to model exactly in simulation.

A complementary and often neglected strand of sim-to-real work addresses evaluation robustness rather than training robustness: even a frozen policy can be systematically stress-tested by replaying it under swept noise and weather perturbations in simulation before any real flight is attempted [16]. This strategy provides a conservative lower bound on real-world performance without requiring physical hardware. The DCC adopts this approach in §5.4: the evaluation suite includes a sensor-noise suite (four LiDAR degradation regimes: clean, low-Gaussian, high-Gaussian, and drop-out) and a domain-randomization suite (five randomly sampled weather conditions with fog, rain, and wind up to 8.0 m/s) applied to both the hybrid controller and the PureRL baseline. Together, these

suites address not only performance but the practical question of deployment robustness under real-world sensing imperfections.

2.5 Hybrid Architectures and the Position of This Work

The most reliable autonomous-stack designs in industry today are hybrid: a global planner produces a smooth desired path from a coarse occupancy or topological map; a geometric tracker (Pure-Pursuit [21] or MPC) follows the path at the look-ahead horizon; a learned policy performs reactive obstacle avoidance against the local LiDAR observation; and a velocity-obstacle shield [15] vetoes any command that would cause an imminent collision. This is exactly the architecture adopted by the AI layer of the DCC, with the novelty that the reactive policy is the published NavRL network, used unchanged but wrapped inside a city-aware planner that supplies it with locally smoothed sub-goals.

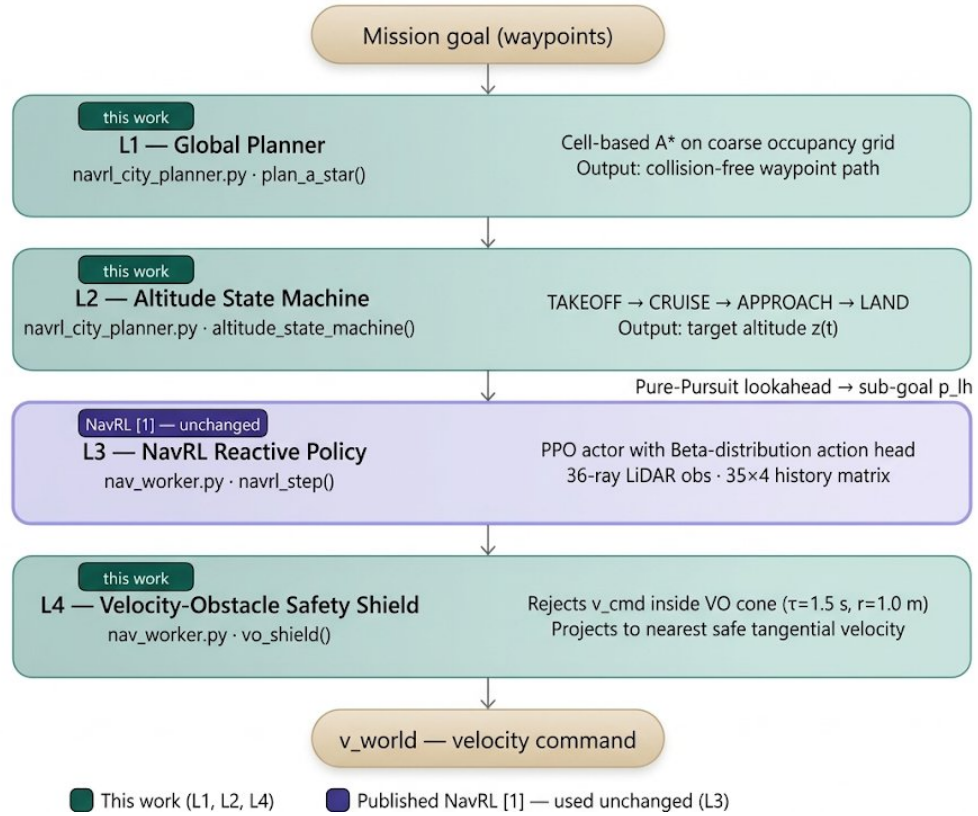


Figure 4 High-level view of the hybrid autonomy stack adopted by the Drone Command Center. Layers 1, 2, and 4 are this work's contribution; layer 3 is the published NavRL policy [1] used unchanged.

The combination satisfies all three structural weaknesses of pure DRL identified in §2.3.3: the global planner supplies the missing global guarantee, the look-ahead sub-goal eliminates the extended-barrier trap by translating long-range goals into locally feasible directions, and the safety shield bounds collision risk independently of policy quality.

To the best of our knowledge, the literature does not document a hybrid drone autonomy stack that simultaneously (i) operates within a complete fleet-management software platform with REST + WebSocket APIs, (ii) uses the

published NavRL network as the reactive layer without re-training, and (iii) has been quantitatively benchmarked against the underlying NavRL baseline on a structured 12-waypoint urban mission with paired ablation, sensor-noise, and weather suites. The remainder of the report addresses precisely this gap: Chapter 3 specifies the analysis and design, Chapter 4 the implementation, Chapter 5 the empirical validation.

3. System Analysis & Design

3.1 Stakeholders & User Stories

A single authenticated-operator role interacts with the platform. Each role corresponds to a set of formal user stories captured during requirements elicitation.

A representative subset of the elicited user stories is listed below in standard “As an authenticated user, I want ... so that ...” form. The complete catalogue (US-01 ... US-26) appears in the project requirements register.

Authentication and Identity (US-01 ... US-04)

- US-01 As a new user, I want to register with username, email, and a strong password so that I can access the platform.
- US-02 As an authenticated user, I want to log in and receive JWT tokens so that I can authenticate API calls.
- US-03 As an authenticated user, I want token refresh so that long-running sessions are not interrupted.

- US-04 As an authenticated user, I want credential recovery handled operationally so that account access can be restored securely.

Drone Fleet (US-05 ... US-09)

- US-05 As an authenticated user, I want to register a drone with serial, name, model, firmware, and home coordinates so that the platform tracks it.
- US-06 As an authenticated user, I want to view the fleet with status badges so that I have situational awareness.
- US-07 As an authenticated user, I want to filter drones by connection or flight status so that I focus on actionable airframes.
- US-08 As an authenticated user, I want to view a drone's full detail including telemetry charts and sensor grid so that I can diagnose issues.
- US-09 As an authenticated user, I want to delete a decommissioned drone with cascade-delete of telemetry, sensors, and commands.

Mission Planning (US-10 ... US-15)

- US-10 As an authenticated operator, I want to drop a navigation goal on the map so the system auto-creates a mission record for the selected drone
- US-11 As an authenticated operator, I want the AirSim bridge to seed a fixed 12-waypoint roam mission for AI evaluation; waypoints are not authored through the operator UI in the current build.
- US-12 As an authenticated operator, I want to start, complete, or abort a mission and have the lifecycle reflected in the database (pause/resume are reserved for a future release).
- US-13 As an authenticated user, I want to view missions filtered by status.

- US-14 As an authenticated user, I want a map view rendering planned mission and live drone position.
- US-15 As an authenticated user, I want a 12-waypoint roam mission via AirSim bridge for AI evaluation.

Command and Control (US-16 ... US-19)

- US-16 As an authenticated user, I want to issue commands (TAKEOFF, LAND, RTH, HOVER, GO_TO_WAYPOINT, EMERGENCY_STOP, ...) to any managed drone.
- US-17 As an authenticated user, I want commands to carry optional JSON payloads.
- US-18 As an authenticated user, I want command history per drone with status (PENDING / SENT / ACKNOWLEDGED / EXECUTED / FAILED / CANCELLED).
- US-19 As an authenticated user, I want EMERGENCY_STOP to supersede all other commands.

Telemetry and Sensors (US-20 ... US-23)

- US-20 As an authenticated user, I want Real-Time Telemetry via WebSocket within one second of ingestion.
- US-21 As an authenticated user, I want historical telemetry query/export by time range.
- US-22 As an authenticated user, I want to render historical flight path on the map.
- US-23 As an authenticated user, I want a sensor grid with type, status, last reading, and timestamp.

Operations (US-24 ... US-26)

- US-24 As an authenticated user, I want Actuator health/info visibility and authenticated access to protected ops endpoints.
- US-25 As a developer, I want Swagger UI at /swagger-ui/index.html documenting every API.
- US-26 As an authenticated user, I want rate limiting on auth endpoints to mitigate brute-force and credential-stuffing attacks.

3.2 Use Cases

Fifteen formal use cases (UC-01 ... UC-15) bind the user stories to system behavior. The grouped use-case diagrams are given below.

- UC Cluster A, Authentication

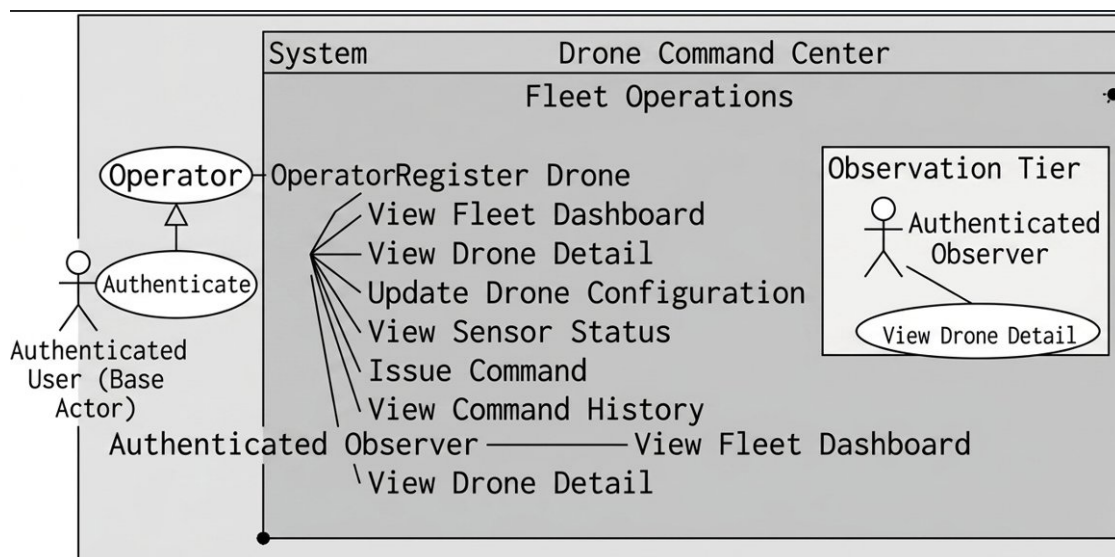


Figure 5 Authentication use-case cluster.

- UC Cluster B, Fleet Operations

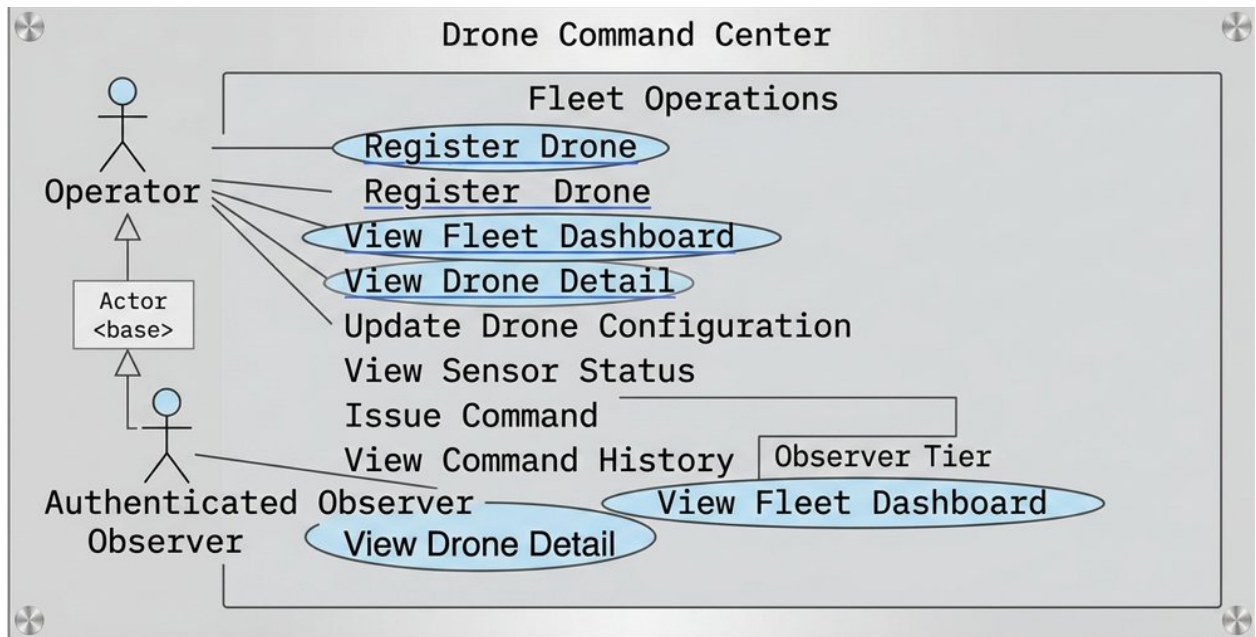


Figure 6 Fleet operations use-case cluster.

- UC Cluster C, Mission Management

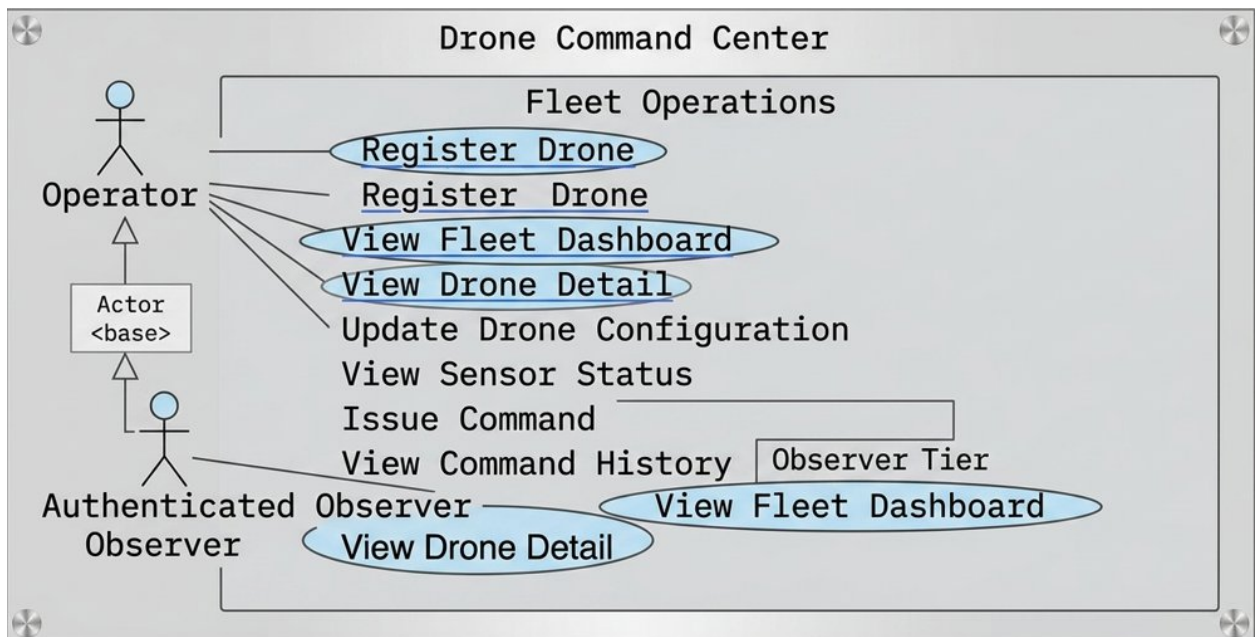


Figure 7 Mission management use-case cluster.

3.3 Functional & Non-Functional Requirements

The system has been decomposed into 45 functional and 35 non-functional requirements, captured in Tables 2–13. Each requirement carries a unique ID, a single-sentence statement, and a priority (Must / Should / Could) per the MoSCoW scheme

3.3.1 Functional Requirements Authentication and User Management

Table 2 Functional requirements: Authentication & User Management.

ID	Requirement	Priority
FR-01	The system shall allow anonymous users to register by providing a unique username, unique email address, and a password satisfying the strength policy.	Must
FR-02	The system shall authenticate registered users via username/password and return a signed JWT access token plus a refresh token upon success.	Must
FR-03	The system shall validate the JWT on every protected request and reject expired or tampered tokens with HTTP 401.	Must
FR-04	The system shall allow clients to exchange a valid refresh token for a new access token without re-authentication.	Must

FR-05	The system shall invalidate the refresh token on logout, preventing further token renewal.	Must
FR-06	The system shall enforce a password strength policy: minimum 8 characters, 1 uppercase, 1 lowercase, 1 digit, 1 special character.	Must
FR-07	The system shall support email-based password reset using a time-limited single-use token.	Should
FR-08	The system shall use a single authenticated-user permission model (no RBAC role matrix in production).	Must
FR-09	The system shall enforce binary authentication (any valid JWT) on all protected API endpoints.	Must
FR-10	The system shall allow authenticated operators to list, update, and deactivate user accounts.	Should

3.3.2 Functional Requirements Drone Fleet Management

Table 3 Functional requirements: Drone Fleet Management.

ID	Requirement	Priority
FR-11	The system shall allow authorized users to register a drone with serial, name, model, firmware, home coordinates, and initial status.	Must

FR-12	The system shall persist real-time drone state: connection status, flight status, battery, GPS, heading, speed, last heartbeat.	Must
FR-13	The system shall return a paginated list of drones, sortable by name, battery, or status.	Must
FR-14	The system shall return the full details of a single drone by UUID.	Must
FR-15	The system shall allow authorized users to update drone configuration and status.	Must
FR-16	The system shall allow authenticated operators to delete a drone and cascade-delete telemetry, sensors, and commands.	Should
FR-17	The system shall filter drones by connection status and flight status.	Should
FR-18	The system shall cache drone-list results in-memory (Caffeine) for up to 5 minutes.	Should

3.3.3 Functional Requirements Mission Management

Table .4 Functional requirements: Mission Management.

ID	Requirement	Priority
----	-------------	----------

FR-19	The system shall automatically create a mission record on the backend the moment an authenticated operator dispatches a navigation goal for a selected drone (no Create-Mission form is exposed to the operator).	Must
FR-20	The system shall support mission states: PLANNED, IN_PROGRESS, COMPLETED, ABORTED, FAILED, PAUSED.	Must
FR-21	The system shall persist an ordered list of waypoints per mission (lat/lon/alt, sequence, action, hover duration, speed, heading); waypoints are seeded server-side from the dispatched goal or from the 12-waypoint AirSim roam template.	Must
FR-22	The backend shall expose REST endpoints to add, update, and remove individual waypoints from a mission for automated tooling and tests; these endpoints are not surfaced in the current operator UI.	Could
FR-23	The system shall allow start, complete, and abort transitions on missions from the operator UI; pause and resume transitions remain available on the backend but are not driven by nav_worker in the current release.	Must
FR-	The system shall record start_time, end_time, and	Must

24	actual_duration_minutes on lifecycle changes.	
FR-25	The system shall return paginated lists of missions, filterable by status.	Must
FR-26	The system shall return all waypoints for a given mission in sequence order.	Must

3.3.4 Functional Requirements Command and Control

Table 5 Functional requirements: Command & Control.

ID	Requirement	Priority
FR-27	The system shall accept commands from authorized users targeting a specific drone.	Must
FR-28	The system shall support command types: TAKEOFF, LAND, RTH, HOVER, GO_TO_WAYPOINT, START_MISSION, ABORT_MISSION, EMERGENCY_STOP, SET_ALTITUDE, SET_SPEED, ROTATE, TAKE_PHOTO, START_STREAMING, STOP_STREAMING.	Must
FR-29	The system shall assign command statuses: PENDING, SENT, ACKNOWLEDGED, EXECUTED, FAILED, CANCELLED.	Must

FR-30	The system shall record timestamps when a command is issued, sent, executed, and completed.	Must
FR-31	The system shall return command history per drone, paginated and sorted by issued time.	Should
FR-32	The system shall store an optional JSON payload for parameterized commands.	Should

3.3.5 Functional Requirements Telemetry and Sensor Monitoring

Table 6 Functional requirements: Telemetry & Sensors.

ID	Requirement	Priority
FR-33	The system shall accept telemetry ingestion: timestamp, GPS, speed, heading, battery, signal, GPS sats, temp, humidity, wind, flight mode.	Must
FR-34	The system shall stream the most recent telemetry to all WebSocket clients within 1 s of receipt.	Must
FR-35	The system shall maintain a persistent /ws/telemetry endpoint with native WS + SockJS fallback.	Must
FR-36	The system shall return the latest telemetry record for a drone via REST.	Must
FR-	The system shall return paginated historical telemetry,	Should

37	optionally filtered by time range.	
FR-38	The system shall return the historical flight-path lat/lon sequence for map rendering.	Should
FR-39	The system shall persist sensor records: name, type, status, last reading, last reading time.	Should

3.3.6 Functional Requirements System Operations

Table 7 Functional requirements: System Operations.

ID	Requirement	Priority
FR-40	The system shall expose Actuator health and info publicly; require authentication for other Actuator endpoints.	Must
FR-41	The system shall apply rate limiting on auth endpoints (10 req/min per IP; 5 failed attempts = 15-min lockout).	Must
FR-42	The system shall apply schema changes via Flyway versioned migrations on startup.	Must
FR-43	The system shall expose a Swagger/OpenAPI 3.0 UI at /swagger-ui/index.html.	Should
FR-	The system shall publish domain events to RabbitMQ for asynchronous inter-service	Could

44	communication.	
FR-45	The system shall run scheduled background tasks (expired-token cleanup, heartbeat checks).	Should

3.3.7 Non-Functional Requirements Performance

Table.8 Non-functional requirements: Performance.

ID	Requirement	Metric / Note
NFR-01	REST API shall respond to 95% of read requests within 200 ms under normal load.	p95 - 200 ms
NFR-02	WebSocket telemetry shall reach subscribed clients within 1 s of ingestion.	1 000 ms
NFR-03	Caffeine cache shall serve repeated reads without DB round-trips for up to 5 min per entry.	TTL = 5 min
NFR-04	The system shall support 50 concurrent WebSocket connections without degradation.	50 conns

NFR-05	DB queries shall use indexes; no N+1 query patterns in production paths.	0 N+1 queries
--------	--	---------------

3.3.8 Non-Functional Requirements Security

Table 9 Non-functional requirements: Security.

ID	Requirement	Metric / Note
NFR-06	All passwords shall be stored as BCrypt hashes with strength 10.	bcrypt(10+)
NFR-07	JWT access tokens 1h; refresh tokens default 7 days.	1h / 7d
NFR-08	All API communication shall support TLS in production.	HTTPS
NFR-09	No SQL injection exclusive use of parameterized JPA/Hibernate queries.	JPA only

NFR-10	CSRF protection via stateless JWT with no server-side session cookies.	Stateless
NFR-11	HSTS, X-Content-Type-Options, X-Frame-Options shall be set on responses.	3 headers
NFR-12	Secrets (DB, JWT, mail) shall be environment-variable-only never in VCS.	env vars
NFR-13	Rate limiting on auth endpoints to mitigate brute-force / credential stuffing.	Bucket4j

3.3.9 Non-Functional Requirements Reliability and Availability

Table 10 Non-functional requirements: Reliability.

ID	Requirement	Metric / Note
NFR-14	The backend shall target 99.5% uptime in production.	99.5%
NFR-15	Daily DB backups with 30-day retention.	Retention 30d
NFR-16	Flyway shall refuse to start on destructive migration conflict.	Schema gate

NFR-17	WS disconnections handled gracefully; resume without > 1 polling-interval loss.	≤ 1 cycle
NFR-18	All unhandled exceptions return structured error responses (no stack traces).	Structured

3.3.10 Non-Functional Requirements Usability

Table 11 Non-functional requirements: Usability

ID	Requirement	Metric / Note
NFR-19	A trained operator shall locate any drone's status within 2 interactions from the dashboard.	≤ 2 taps
NFR-20	All form inputs shall display inline validation messages near the relevant field.	Inline errors
NFR-21	Async fetch operations shall display shimmer loading skeletons.	Shimmer
NFR-22	Mobile UI shall support both portrait and landscape without layout degradation.	Responsive
NFR-23	Swagger UI shall provide schemas, examples, and descriptions for every endpoint.	Full OpenAPI

3.3.11 Non-Functional Requirements Scalability and Maintainability

Table 12 Non-functional requirements: Scalability & Maintainability.

ID	Requirement	Metric / Note
NFR-24	Backend shall be stateless to allow horizontal scaling behind a load balancer.	Stateless
NFR-25	Cache layer replaceable by Redis via configuration only no code changes.	Cache abstraction
NFR-26	DB connection pool shall be configurable through application.properties.	Pool tunable
NFR-27	RabbitMQ decouples telemetry ingestion from downstream processing.	MQ decoupled
NFR-28	Codebase shall preserve Controller / Service / Repository / Entity layering.	Layered
NFR-29	Public endpoints shall be documented with OpenAPI 3.0 annotations.	OpenAPI 3.0
NFR-30	Schema changes only through Flyway migrations direct DDL prohibited.	Flyway-only
NFR-31	Structured Logback/SLF4J logs (ERROR/WARN/INFO/DEBUG) with split log files.	SLF4J
NFR-32	Service-layer unit-test coverage 70%.	Coverage 70%

3.3.12 Non-Functional Requirements Portability

Table 13 Non-functional requirements: Portability.

ID	Requirement	Metric / Note
NFR-33	Backend deployable as a JAR on JVM 17+ or as a Docker container.	JVM 17+ / Docker
NFR-34	Flutter app shall compile to Android APK, iOS IPA, and web bundle from one codebase.	Multi-target
NFR-35	docker-compose.yml shall bring up the full local environment with one command.	1-command

3.4 Four-Tier System Architecture

The platform decomposes into four logical tiers (Table 14, Figure 8). Tier boundaries are enforced by package structure on the backend and feature folders on the frontend; cross-tier communication is exclusively via DTOs and well-typed API responses.

Table 14 Four-tier architecture.

Tier	Responsibility	Technology
1.	Tactical UI, mission planning, telemetry	Flutter 3.0 /

Presentatio n	visualization, command issuance.	Dart 3
2. Applic ation API	REST controllers, WebSocket handlers, request validation, stateless JWT authentication enforcement.	Spring Boot 4.0.2 / Java 17
3. Domain / Service	Business logic, mission lifecycle, command orchestration, AirSim/NavRL bridge.	Spring Service beans + Python AI workers
4. Persistence	Schema-versioned storage of users, drones, missions, telemetry, sensors, commands.	PostgreSQL 15+ / Flyway / Caffeine / RabbitMQ

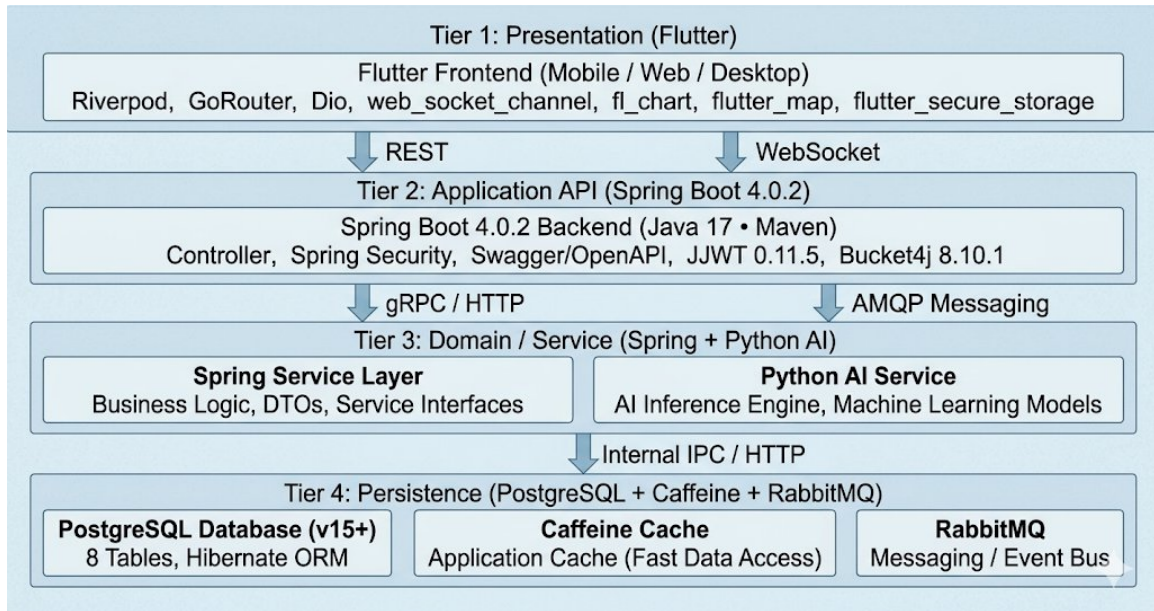


Figure 8 Four-tier deployment view of the Drone Command Center.

3.5 Backend Component Design

The backend is organized under `com.drone_command_center` with strict package-level layering. Configuration is centralized; security artefacts live in a dedicated package; controllers expose REST endpoints; services hold business logic; repositories perform persistence; entities model the domain; DTOs cross the boundary.



Figure 9 Backend package tree.

3.5.1 Five-Layer Security Model

- L1 Network. TLS termination, HSTS, CORS allow-list, secure cookies.
- L2 Rate Limiting. Bucket4j token-bucket on `/api/auth/*` (10 req/min/IP).
- L3 Authentication. Stateless JWT (HMAC-SHA256), refresh-token rotation, BCrypt(10+) password hashing.
- L4 Authorization. Single authenticated-user model: every API endpoint outside `/api/auth/**` and `/actuator/(health|info)` requires a valid JWT in the Authorization header. The earlier multi-role binary JWT authentication matrix was removed in the V2 demo cleanup, which dropped the `user_roles` and `user_drone_assignments` tables;

all authenticated users now share equal access to fleet and mission resources.

- L5 Validation. Bean Validation on all DTOs; centralized exception handler returns structured error bodies.

3.5.2 Endpoint Authentication Matrix

Following the V2 schema cleanup, authorization is reduced to a binary public / authenticated decision there is no role granularity in the production deployment.

Public endpoints are limited to authentication, Swagger / OpenAPI, public Actuator probes, and the WebSocket handshake.

Table 15 Endpoint authentication matrix (post V2 cleanup).

Endpoint group	Public	Authenticated
/api/auth/**	✓	—
/swagger-ui/** , /v3/api-docs/**	✓	—
/actuator/health, /actuator/info	✓	—
/actuator/** (other endpoints)	—	✓
/ws/** (WebSocket handshake)	✓	—
/api/drones/**	—	✓
/api/missions/**	—	✓
/api/commands/**	—	✓
/api/telemetry/**	—	✓
/api/users/**	—	✓

/api/airsim/** , /api/navrl/**	—	✓
/api/logs/**	—	✓

3.6 Frontend Component Design

The Flutter application is structured by feature, with a small core/ package providing shared utilities (HTTP client, secure storage, theme, design tokens) [44].

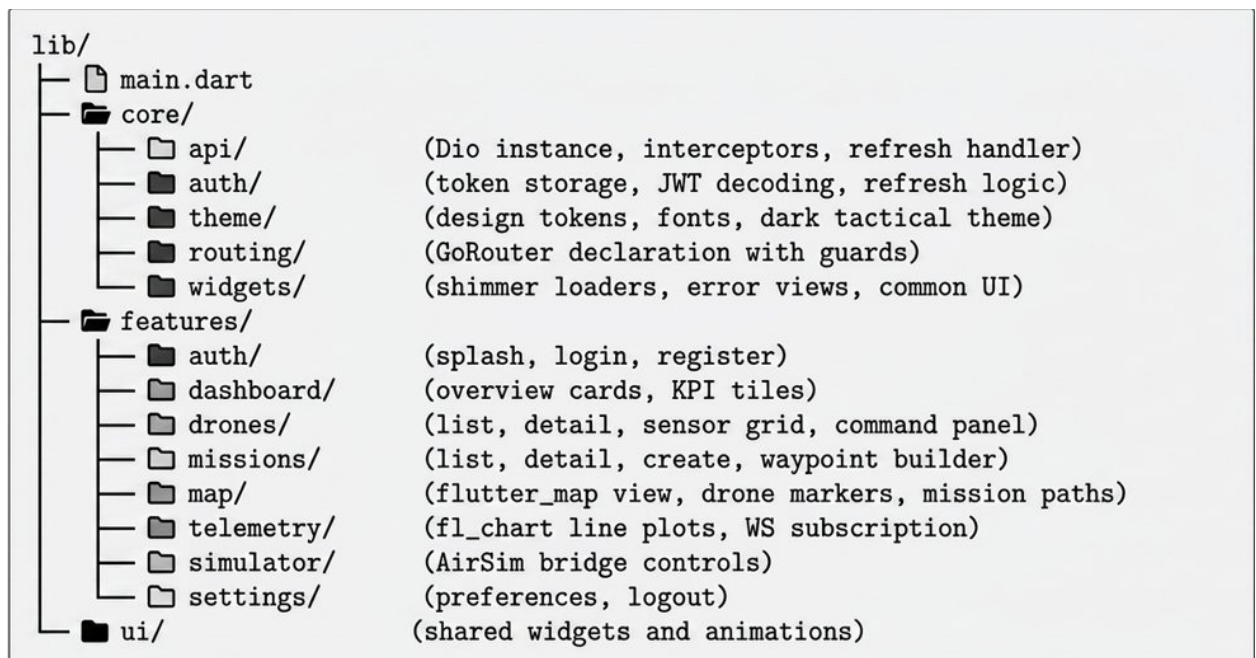


Figure 10 Frontend lib/ tree.

3.6.1 Riverpod State Topology

State is segmented into providers per concern. authProvider exposes the current JWT and user; droneListProvider caches the paginated drone list with refresh-on-WS-tick; missionProvider.family(missionId) caches a single mission with its waypoints; telemetryStreamProvider.family(droneId)

subscribes to /ws/telemetry and emits Telemetry objects to any consuming widget. Riverpod's automatic disposal removes WS subscriptions when no widget consumes the stream.

3.6.2 Tactical Design Tokens

Table 16 Tactical theme tokens.

Token	Value	Usage
Background	#0A0A0A	App scaffold base.
Primary	#00FF88	Active state, success, RTH path.
Secondary	#00D4FF	Telemetry charts, info accents.
Warning	#FF6B35	Battery low, signal weak.
Danger	#FF0040	Emergency, errors, abort.
Display Font	Rajdhani	Headings, KPI values, mission names.
Mono Font	Space Mono	Telemetry numbers, JSON payloads, IDs.

3.7 Deployment Architecture

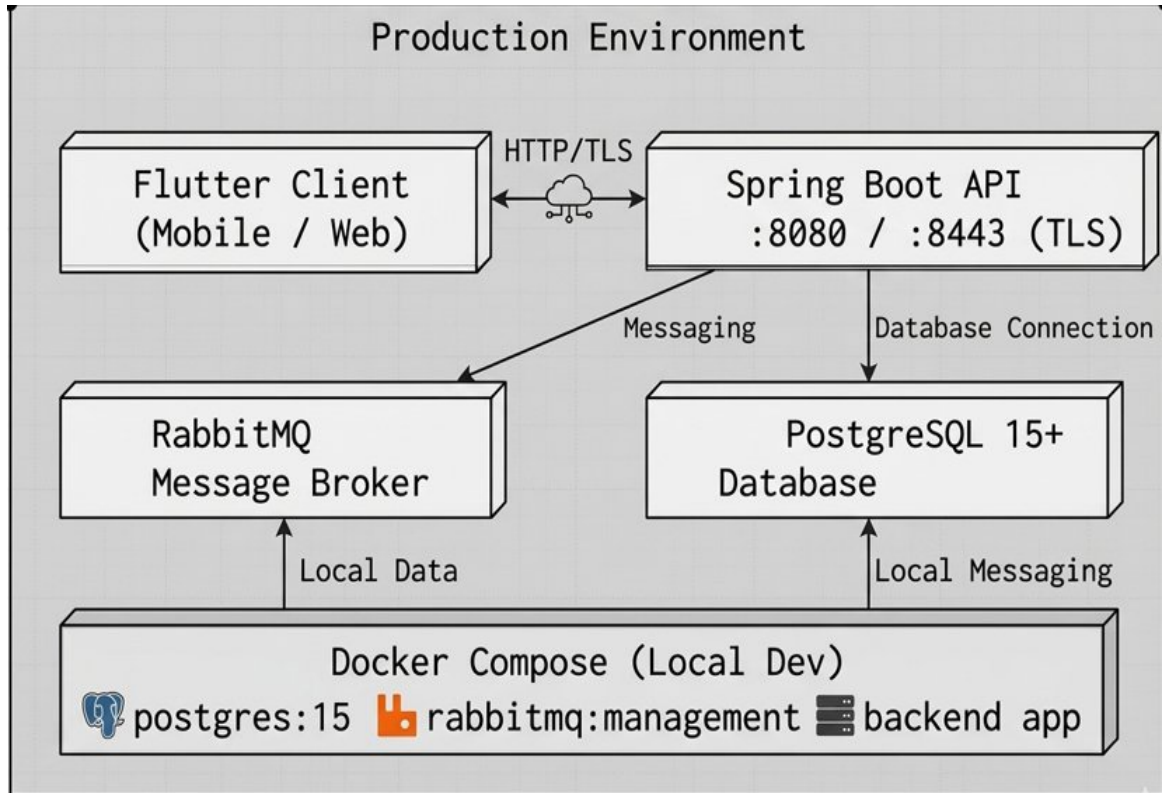


Figure 11 Deployment topology.

3.8 NavRL Theoretical Foundations

The autonomy layer in this work uses the published NavRL framework [1] without modification to its neural network. NavRL formulates UAV navigation as a partially observable Markov decision process (POMDP) and solves it with a Beta-distribution Proximal Policy Optimization (PPO) policy [24]. This section captures the equations needed to interpret §5.5 and Chapter 8.

3.8.1 Markov Decision Process

The decision problem is the tuple (S, A, P, R, γ)

$$(S, A, P, R, \gamma), \gamma \in [0, 1) \quad (3.1)$$

3.8.2 Internal State

The internal state vector at time t encodes goal direction, current velocity, and altitude-track error:

$$s_t = [d_{goal}, v_t, e_z]^T \quad (3.2)$$

3.8.2.1 Dynamic Obstacle State Representation

The dynamic-obstacle tensor $S_{dyn} \in R^{N_d \times M}$ and the per-obstacle state vector D_i are defined as:

$$S_{dyn} = [D_1, \dots, D_{N_d}]^T, S_{dyn} \in R^{N_d \times M}, D_i \in R^M. \quad (3.3)$$

$$D_i = \left[\frac{P_{o_i}^G - P_r^G}{\|P_{o_i}^G - P_r^G\|}, \|P_{o_i}^G - P_r^G\|, V_{o_i}^G, \dim(o_i) \right]^T, \quad (3.4)$$

3.8.3 Lidar Observation

A 360°, 36-ray LiDAR is sampled and reshaped into a 35-channel $\times$ 4-frame matrix:

$$O_L \in R^{35 \times 4} \quad (3.5)$$

Range readings are inverted before being fed into the policy network so that closer obstacles produce larger,

easier-to-learn activations:

$$w_i = 1 - \frac{\min(r_i, r_{max})}{r_{max}} \quad (3.6)$$

Table 17 NavRL LiDAR observation parameters

Parameter	Symbol	Value
Number of rays per scan	N	36
Maximum effective range	$\mathbf{x}_{max}$	6.0 m
Vertical sectors	S_z	3 (low/mid/high)
Horizontal field of view	FOV_h	360°
Observation history frames	H	4
Network input dim. per frame	d	35

3.8.4 Action Distribution

The actor outputs the parameters (α, β) of a Beta distribution; bounded raw actions are linearly mapped into the velocity command range:

$$a_{raw} \sim \text{Beta}(\alpha, \beta), \alpha, \beta > 0 \quad (3.7)$$

$$v^{cmd} = (2 a_{raw} - 1) \cdot v_{max} \quad v^{cmd} \in [-v_{max}, +v_{max}] \quad (3.8)$$

3.8.5 World-Frame Transformation

Body-frame velocities are rotated into the world frame using the current yaw

Θ :

$$v_{world} = R_z(\psi) \cdot v_{cmd} \quad (3.9)$$

3.8.6 Reward Components

The dense reward at step t is the sum of five terms:

$$r(t) = r_{goal} + r_{step} + r_{smooth} + r_{obs} + r_{height} + r_{terminal} \quad (3.10)$$

$$r_{goal} = w_g \cdot \left(\left| p_{goal} - p(t-1) \right| - \left| p_{goal} - p(t) \right| \right) \quad (3.11)$$

$$r_{step} = -w_s \quad (3.12)$$

$$r_{smooth} = -w_j \cdot \left| a(t) - a(t-1) \right|^2 \quad (3.13)$$

$$r_{obs} = -w_o \cdot \max(0, d_{safe} - d_{min})^2 \quad (3.14)$$

$$r_{height} = -w_o \cdot \left(\left| P_{r,z} - P_{s,z} \right|, \left| P_{r,z} - P_{g,z} \right| \right)^2 \quad (3.15)$$

$$r_{terminal} = \begin{cases} +R_{success} & \text{on goal} \\ -R_{collide} & \text{on contact} \end{cases} \quad (3.16)$$

3.8.7 PPO Clipped Objective

PPO [24] optimizes the clipped surrogate objective:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right] \quad (3.17)$$

with importance ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (3.18)$$

and clip range $\epsilon = 0.2$.

3.8.8 Curriculum

Table 18 NavRL training curriculum (per [1]).

Stage	Environment	Goal	Episodes
1	Empty 20×20×10 box	Reach random target	≈ 50k
2	Sparse static obstacles	Avoid ≤ 5 boxes	≈ 80k
3	Dense forest (20–40 trees)	Cluttered traversal	≈ 120k
4	Dynamic obstacles (moving spheres)	Reactive avoidance	≈ 100k
5	Mixed static + dynamic, narrow gaps	Generalization, fine-tuning	≈ 80k

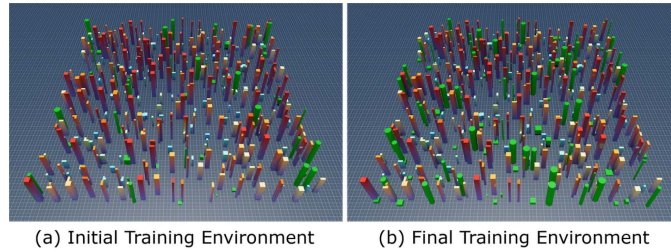


Figure 12 NavRL training environments in NVIDIA Isaac Sim: initial stage (60 obstacles) vs final stage (120 obstacles) of the curriculum [1].

Table 19

COMPARISON OF THE HIGHEST NAVIGATION SUCCESS RATES DURING TRAINING
WITH AND WITHOUT USING CURRICULUM LEARNING [1]

NUMBER OF OBSTACLES	NO CURR. LEARNING	CURR. LEARNING
static=550, dynamic=00	94.99%	94.99%
static=550, dynamic=80	14.91%	64.11%
static=550, dynamic=100	02.90%	60.90%
static=550, dynamic=120	04.90%	08.09%

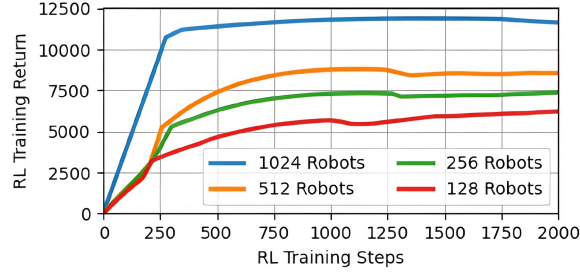


Figure 13. Plot of the smoothed average training return curves. Training with a larger number of robots leads to faster convergence and higher returns [1]

3.8.9 Velocity-Obstacle Safety Shield

A reactive VO shield runs in parallel with the policy. Given a relative obstacle position p_r and velocity v , a candidate command v^{cmd} is rejected if it lies inside the time-to-collision

cone:

$$VO = \{v: \frac{\|(p_r - v \cdot t) \times p_r\|}{\|p_r\|} < r_{safe}, t = 1.5s, r_{safe} = 1.0m\} \quad (3.19) [1]$$

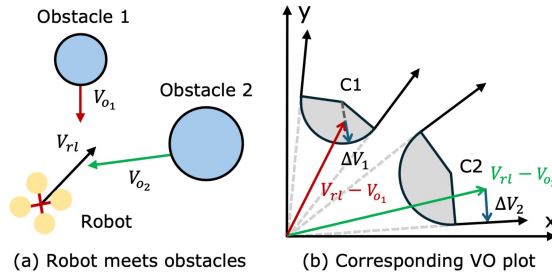


Figure 14 Velocity-Obstacle (VO) safety shield: the VO cone formed by an obstacle, the unsafe robot velocity falling inside it, and the projected ΔV that returns the velocity to the safe set [1]." This is the most important visual in the section without it the reader cannot follow why the shield is needed.

If $v^{\text{cmd}} \in VO$, the planner overrides the action with the nearest tangentially-feasible velocity, biased toward the global path (§3.9).

3.8.10 Published Performance

On the original NavRL evaluation suite [1] the trained policy attained the Figures reproduced in Table 20. These values establish the upper performance ceiling that any wrapper around NavRL can preserve.

Table 20 Published NavRL benchmarks [1]. Path length is normalized to the optimal straight-line baseline.

Environment	Success	Coll. /km	Path Len.
Empty box	100%	0.0	1.0×
Sparse static	96%	0.4	1.05×
Dense forest	88%	1.2	1.18×
Dynamic obstacles	82%	1.6	1.22×

3.9 Hybrid City-Planner Design

Pure NavRL fails in dense procedurally-generated city geometry: it has no global plan and gets trapped behind tall buildings. The Drone Command Center wraps NavRL inside a deterministic four-layer planner (Table 21). The planner injects geometric guidance, predictable altitude profiling, and

a final VO veto, while leaving the reactive policy untouched.

Table 21 Four-layer hybrid city planner.

Layer	Module / File	Responsibility	Output
L1 Global	navrl_city_planner.py · plan_a_star()	Cell-based A* on coarse occupancy.	Waypoint list
L2 Altitude	navrl_city_planner.py · altitude_state_machine()	Profile vertical: TAKEOFF CRUISE APPROACH.	Target $z(t)$
L3 Tracker	nav_worker.py · pure_pursuit()	Lookahead point on the polyline.	Sub-goal p_{lh}
L4 Reactive	nav_worker.py · navrl_step() + vo_shield()	PPO action; VO veto; emit world velocity.	$v_{world} \in R^3$

3.9.1 Pure-Pursuit Lookahead

Given the global path $P = (p_0, \dots, p_N)$ and current pose p , the lookahead point

p_{lh} is the point on P at arc-length L_{lh} ahead of the closest projection:

$$p_{lh} = \arg \min_{p_i \in P} |||p_i - p_{proj}|| - L_{lh}|, \quad L_{lh} = 4.0 \text{ m} \quad (3.20)$$

This sub-goal is fed to NavRL as p_{goal} in the internal state vector of equation 3.2, ensuring the reactive policy always chases a locally smooth geometric target rather than the original (possibly occluded) mission goal.

3.9.2 Altitude State Machine

- TAKEOFF. Climb at ≈ 1.0 m/s until cruise altitude (12 m AGL) is reached.
- CRUISE. Hold cruise altitude; pursue lookahead in the horizontal plane.
- APPROACH. When within 8 m of final goal, descend to mission target altitude.
- HOVER / LAND. On goal reached, hover for waypoint dwell time, then land.

3.10 Hardware Design (CAD Model):



Figure 15 shows the first CAD model developed for AI integration.

3.10.1 List of Parts

The prototype airframe was modelled in Fusion 360 [41] around the standard DJI F450 quadcopter platform. The choice of each component is driven by the requirements of the autonomy stack (§3.6) and the dynamic model of §2.2. All geometric and inertial values used by Eqs. (2.1)–(2.2) and by the AirSim vehicle configuration are extracted directly from this CAD.

Airframe

- DJI F450 X-frame. A 450 mm carbon-fibre / glass-reinforced X-frame, widely adopted across academic reinforcement-learning quadrotor research. Its symmetry simplifies the dynamic model (zero off-diagonal inertia terms in the body frame) and a STEP file of the frame is openly available, which gave us a known-good starting geometry to extend with the payload bay and avionics tray.
- Landing gear. Tall sprung legs giving ground clearance for the down-facing LiDAR and protecting the avionics tray, battery and camera during emergency landings or autonomous touch-down.

Propulsion

- 4 × 920 KV brushless DC motors with 10" props. A four-rotor configuration is mandated by the X-frame and the dynamic model of §2.2.2. The 920 KV / 10" pairing gives a hover throttle of approximately 50 % at the projected ~1.4 kg all-up weight, leaving a $\approx 2 : 1$ thrust margin so the NavRL policy

can request aggressive accelerations without saturating any rotor.

- 4 × 30 A electronic speed controllers (ESCs), BLHeli_S firmware. The ESCs translate PWM (or DShot) commands from the flight controller into three-phase drive signals for the motors, throttling current draw in proportion to the commanded thrust. They are sized above the motors' peak current draw to tolerate transient spikes during sharp manoeuvres.

Power

- 1 × 4S 3300 mAh LiPo battery. Used for bench testing and short demonstration flights. It does not target long-endurance missions; it is sized to validate the electrical layout and provide ~6–8 min of hover for HIL tests. A higher-capacity 5000–6000 mAh pack is identified in §9.2 (future work) for sustained operation.

Flight control and compute

- Pixhawk 6C flight controller running PX4 firmware. The Pixhawk closes the inner attitude / rate loop at 400 Hz and exposes a MAVLink interface to the companion computer. PX4 was chosen over ArduPilot because its uORB messaging model maps cleanly onto the offboard control commands issued by NavRL.
- NVIDIA Jetson Orin NX [46] 16 GB. The companion computer hosts the entire autonomy stack: NavRL inference (10 Hz), the A*/CityPlanner global planner, the altitude/velocity controller, ROS 2 nodes, and the gRPC bridge to the AirSim simulator. The 16 GB variant was chosen so

the policy network and the local cost-map can be co-resident in memory without paging.

Sensing

- Livox Mid-360 LiDAR. A $360^{\circ} \times 59^{\circ}$ solid-state LiDAR [45] providing the dense local point cloud that feeds the obstacle-avoidance branch of NavRL (§3.6). Its non-repetitive scan pattern fills in the field of view over a few hundred milliseconds, giving the policy a richer occupancy map than a traditional spinning LiDAR at the same price point. This is the primary "eye" of the drone for avoidance and local mapping.
- u-blox NEO-M8N GPS. Multi-constellation receiver (GPS + GLONASS + Galileo) used by PX4's EKF for global position estimation and by the global planner to anchor missions to real-world coordinates.
- PPM/SBUS receiver. A safety-pilot fallback paired with a hand-held transmitter, it allows manual override at any time during HIL testing, satisfying a soft-safety requirement common to outdoor RL flight trials. It is not used during normal autonomous operation.

3.11 System Models

This section captures the dynamic and static models of the platform: the entity-relationship view, four sequence diagrams covering critical interaction flows, three state machines describing the lifecycle of drones, missions, and commands, and two data-flow descriptions for telemetry and authentication tokens.

3.11.1 Entity-Relationship Model

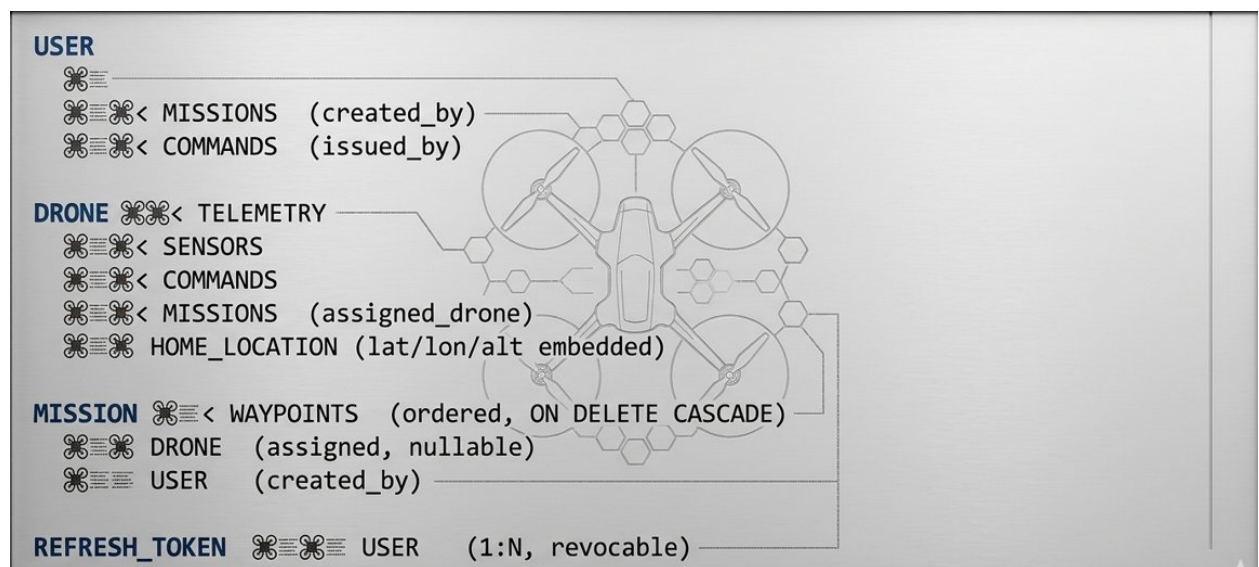


Figure 16 Entity-relationship diagram (post V2 cleanup).

Cardinality summary: One user owns many missions and many commands; one drone has many telemetry, sensors, commands and missions; one mission has an ordered list of waypoints (1:N, cascade delete); a user may hold multiple refresh tokens, each individually revocable on logout. The legacy user_roles, user_drone_assignments and password_reset_tokens tables were removed by migration V2_demo_cleanup.sql

3.11.2 Sequence User Authentication

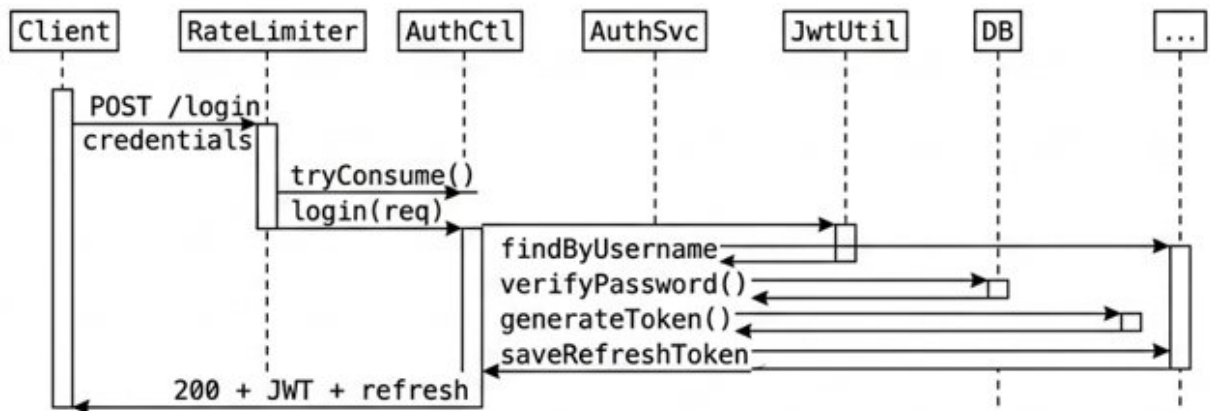


Figure 17 Login sequence.

3.11.3 Sequence Goal Dispatch and Auto-Mission Creation

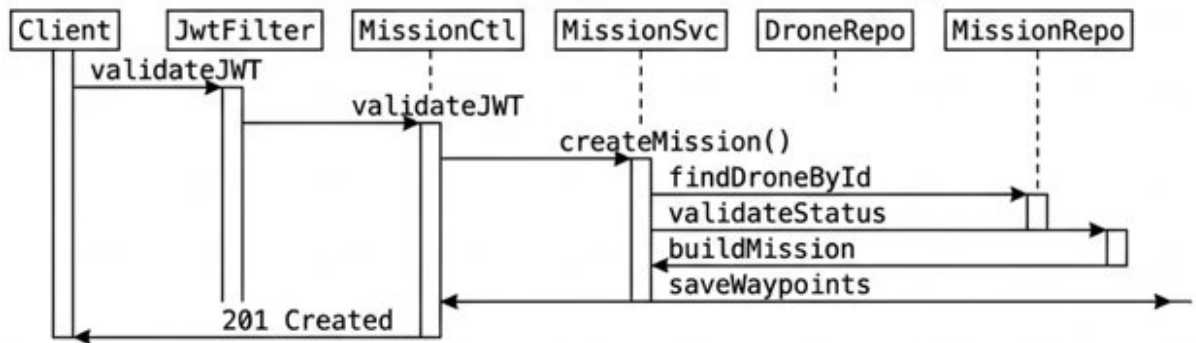


Figure 18 Goal-dispatch sequence. Operator taps the map; the Flutter client converts the screen point to AirSim NED meters and POSTs `/api/navrl/drones/{id}/goal` to persist the target. On START NAV (POST `/api/navrl/drones/{id}/start`) the backend marks `isNavigating=true`, materializes a mission row from the stored goal, spawns the `nav_worker` subprocess, and begins streaming telemetry. The operator never authors a mission body or waypoint list by hand.

3.11.4 Sequence Real-Time Telemetry

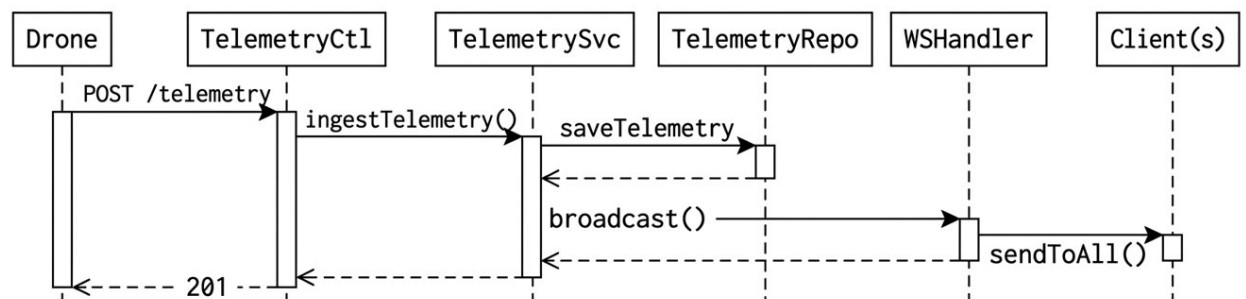


Figure 19 Telemetry ingestion + WebSocket fan-out.

3.11.5 Telemetry Pipeline

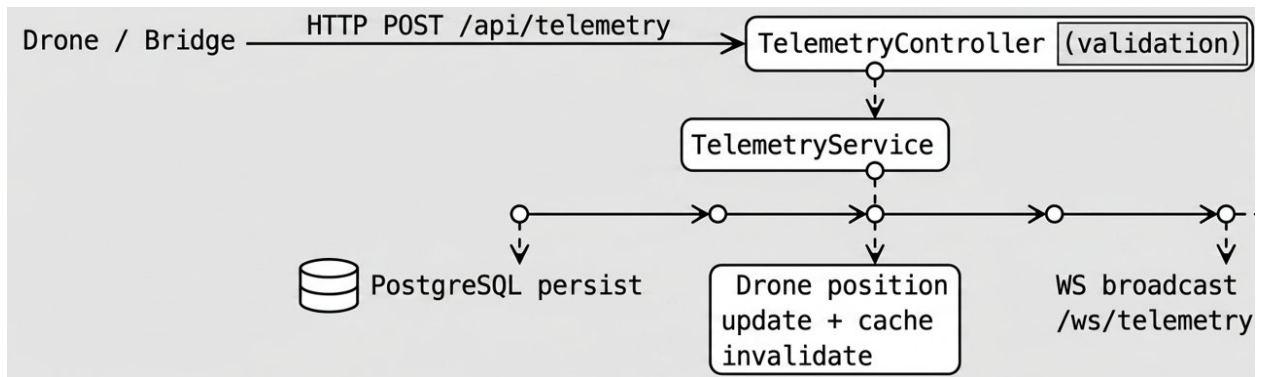


Figure 20 Telemetry data-flow.

3.11.6 Feasibility and Risk Analysis

Table 22 Top project risks and mitigations.

Risk	Likelihood	Impact	Mitigation
AirSim instability under heavy load	Med	High	Auto-restart bridge; checkpoint every waypoint.
NavRL fails on geometry it never saw	High	High	Wrap with VO shield; classical A* fallback (Layer 1).
Backend / frontend breaking change	Med	Med	OpenAPI contract; integration tests; semantic versioning.
Schema migration error	Low	High	Flyway versioned

in production			migrations; refuse-on-conflict gate.
Credential leak / brute force	Med	High	BCrypt(10+); rate limiting; rotated refresh tokens; HSTS.
Lost WebSocket telemetry	Low	Low	Auto-reconnect; SockJS fallback; REST poll backup.
Schedule overrun (capstone deadline)	Med	Med	Sprint-based delivery; weekly reviews; scope freeze.

4. Implementation

4.1 Development Methodology and Tooling

The project followed a 2-week sprint Scrum cadence. Source control is Git on GitHub with a protected main branch and feature branches per user story. CI runs Maven and Flutter tests on every push. Code style is enforced by Spring Boot defaults (Checkstyle), Lombok-driven boilerplate elimination, and the Flutter analysis_options.yaml defaults. Issue tracking and burndown live on GitHub Projects.

Table 23 Implementation tooling.

Concern	Tool / Configuration
Backend build	Maven 3.9 + Spring Boot 4.0.2 BOM
Java runtime	OpenJDK 17 (LTS)
Frontend build	Flutter 3.0 / Dart 3.0
DB	PostgreSQL 15 (Docker image postgres:15)
Message broker	RabbitMQ 3 (rabbitmq:management image)
Local orchestration	docker-compose.yml at project root
AI runtime	Python 3.10 + PyTorch (NavRL inference)
Simulator	Microsoft AirSim UE4 City environment [40]
Code quality	SpringDoc OpenAPI; Flutter analyzer; pytest

4.2 Backend Implementation (Spring Boot)

4.2.1 Module Map

- config/ CacheConfig (Caffeine TTL=5 min), RabbitMQConfig (exchange + queue declaration), OpenApiConfig (Swagger groups + JWT bearer scheme), MailConfig (JavaMailSender bean).
- Security/ JwtUtil (sign/verify HMAC-SHA256), JwtFilter (extract Authorization header SecurityContext), RateLimiter (Bucket4j 10 req/min on /api/auth/*), SecurityConfig (HttpSecurity DSL with CORS allow-list, HSTS, X-Frame-Options DENY, stateless session, public /api/auth/** and Swagger; everything else authenticated).

- Controller/ REST controllers per domain object; one Swagger group per controller; Bean Validation on every @RequestBody DTO.
- Service/ Business orchestration; Spring @Cacheable on read paths; @CacheEvict on writes; @Transactional boundaries.
- Repository/Spring Data JPA interfaces with custom query methods (e.g., findByConnectionStatus, findLatestByDroneld).
- Entity/JPA @Entity classes mapped to Hibernate-managed tables; UUID primary keys; explicit lat/lon/alt home columns on the drones table.
- websocket/TelemetryWebSocketHandler (per-session subscribers); WebSocketConfig (STOMP endpoints, SockJS fallback).
- scheduler/ @Scheduled tasks for token cleanup and heartbeat checks.

4.2.2 Authentication Path

Login flow: POST /api/auth/login à RateLimitFilter (Bucket4j) à AuthController.login() à AuthService.login() à UserRepository.findByUsername() -> BCrypt verify à JwtUtil.generateAccessToken() + generateRefreshToken() à persist refresh token à return AuthResponse {accessToken, refreshToken, expiresIn}.

4.2.3 Validation, Errors, and Audit

All DTOs use JSR-380 (Bean Validation) annotations. A GlobalExceptionHandler translates exceptions into structured JSON: { timestamp, status, message, path }. All authentication outcomes (success / failure / lockout) are logged to a separate audit appender configured in logback-spring.xml.

4.3 Frontend Implementation (Flutter)

4.3.1 Routing and Guards

GoRouter declares the full route table. A guard inspects flutter_secure_storage for a valid JWT; if expired, the Dio refresh interceptor silently exchanges the refresh token before retrying the original request. If the refresh fails, the user is redirected to /login.

4.3.2 Networking Layer

All HTTP traffic flows through a single Dio instance configured with:

- baseUrl from --dart-define=API_BASE at build time.
- Auth interceptor that attaches Authorization: Bearer <jwt>.
- Retry / refresh interceptor handling 401s by exchanging the refresh token.
- Error interceptor mapping 4xx/5xx to typed ApiException objects.

4.3.3 Real-Time Telemetry

TelemetryStreamProvider opens a web_socket_channel connection to /ws/telemetry, decodes JSON frames into Telemetry objects, and pushes the latest sample plus a rolling window into fl_chart. Auto-reconnect with exponential back-off recovers from transient network failures.

4.3.4 Map and Mission Visualization

The map screen uses `flutter_map` with an OpenStreetMap tile layer; drone markers are colored by flight status using the design tokens of Table 16 (#00FF88 in flight, #FF6B35 low battery, #FF0040 emergency). Mission paths are rendered as a `PolylineLayer` bound to the waypoint sequence.

4.4 AirSim Bridge and NavRL Integration

4.4.1 Component Overview

- `airsim_auto_bridge.py` process supervisor: starts AirSim, the command-center bridge, the NavRL worker, and the planner; restarts on crash.
- `command_center_bridge.py` connects the Spring Boot REST + WS API to AirSim. Reports drone telemetry (position, velocity, battery, sensor health) and accepts commands.
- `navrl_airsim_bridge.py` adapter that converts AirSim LiDAR + IMU to the 35×4 NavRL observation matrix (Eq. 3.3) and converts NavRL Beta-distribution actions into AirSim `moveByVelocityAsync` calls.
- `navrl_city_planner.py` implements Layers 1 + 2 (A* global + altitude state machine).
- `nav_worker.py` implements Layers 3 + 4 (Pure-Pursuit

lookahead + NavRL forward pass + VO shield).

- capstone_test_runner.py orchestrates evaluation suites: standard, ablation, sensor noise, domain randomization.
- analyze_results.py / generate_report_Figures.py post-processing CSV summaries and the seven Figures cited in §5.4.

4.4.2 Inference Loop

```
while not goal_reached and step < step_limit:

    ✓, ✎ = policy.forward(obs) a_raw =          # NavRL PPO actor
    sample_beta(✓, ✎)

    v_cmd = (2 * a_raw - 1) * V_MAX              # Eq. 3.6

    if vo_shield.violates(v_world, obstacles):# Eq. 3.19 v_world
        = vo_shield.project_to_safe(v_world)

    airsims.moveByVelocityAsync(*v_world, dt)
    update_lookahead(global_path, current_pos)# Eq. 3.20
```

Figure 21 Pseudocode of the inner control loop.

4.4.3 Sensor Noise Injection

Table 24 Sensor and disturbance noise injection per [15], [16].

Sensor	Noise model	Magnitude
LiDAR ranges	Additive Gaussian + 5% drop-out	$\sigma = 0.05$ m
IMU velocity	Additive Gaussian on each axis	$\sigma = 0.05$ m/s
GPS	Additive Gaussian on lat/lon (converted to meters)	$\sigma = 0.50$ m

Wind disturb.	Time-varying lateral force (Ornstein-Uhlenbeck)	$\leq 1.5 \text{ m/s}$
---------------	---	------------------------

4.4.4 Database Schema and Migrations

The full database schema lives in `backend/src/main/resources/db/migration/`.

Two Flyway migrations were authored during development; in the deployed configuration the runtime schema is owned by Hibernate `ddl-auto=update` (Flyway is disabled via `spring.flyway.enabled=false`). Migration V2 was executed as a destructive reset for the clean evaluation environment. For continuous production deployment, subsequent Flyway migrations will utilize non-destructive “ALTER” statements. The full column-level reference for every live table is given in Appendix C.

- V1 `Initial_schema.sql` the eight live tables (users, drones, missions, waypoints, commands, telemetry, sensors, `refresh_tokens`) plus three legacy tables (`user_roles`, `user_drone_assignments`, `password_reset_tokens`) and all supporting indexes.
- V2 `demo_cleanup.sql` drops the three legacy tables (binary JWT authentication simplification + removal of the email password-reset feature) and truncates telemetry / commands so that the demo run starts from a clean slate.

5. Testing and Evaluation

5.1 Test Strategy Overview

Quality assurance follows a multi-layer pyramid. Unit tests cover individual classes in isolation. Integration tests exercise wiring across multiple components against an in-memory H2 database. API tests treat the running backend as a black box. Frontend widget and integration tests exercise screens and providers. The AI layer is evaluated quantitatively against fixed AirSim missions with statistical aggregation.

Table 25 Test strategy summary.

Layer	Tooling	Target Coverage / Metric
Backend unit	JUnit 5 + Mockito + AssertJ	70% Service layer (NFR-32)
Backend integration	Spring Boot Test + H2 + Testcontainers	All CRUD + auth flows
REST contract	Postman / Newman / Swagger UI	100% endpoints exercised
Security	Manual + OWASP ZAP scan	OWASP Top 10 verified
Frontend unit	flutter test (pure-Dart)	Models + helpers

Frontend widget	flutter_test + flutter_test/widget tester	Each screen at least once
Frontend integ.	integration_test + driver	End-to-end login mission
AI evaluation	AirSim + capstone_test_runner.py + analyze_results.py	30 trials per condition

5.2 Backend Testing

Backend tests are executed by Maven via `./mvnw test`. Test results are summarized in the Surefire reports under `backend/target/surefire-reports/`. The tables and screenshot placeholders below capture each major test category.

5.2.1 Unit Tests Service Layer

Each service class has a corresponding test (e.g., `AuthServiceTest`, `DroneServiceTest`, `MissionServiceTest`, `CommandServiceTest`, `TelemetryServiceTest`). Mockito mocks the repository layer; AssertJ asserts on returned DTOs.

Table 26 Backend unit-test catalogue.

Test class	Scenarios verified	Outcome
AuthServiceTest	register / duplicate user / login / wrong password / refresh / logout / reset	Pass

DroneServiceTest	CRUD; status filters; cache hit; cascade delete	Pass
MissionServiceTest	Auto-create on goal dispatch / start / complete / abort / lifecycle invariants	Pass
CommandServiceTest	Issue, status transitions, payload validation, history pagination	Pass
TelemetryServiceTest	Ingest, latest, range query, flight-path projection, WS broadcast	Pass
DroneCommandCenterApplicationTests	Spring context loads; bean wiring sanity	Pass

```

[INFO] Results:
[INFO]
[INFO] Tests run: 62, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 19.200 s
[INFO] Finished at: 2026-05-03T19:31:01+03:00
  
```

Figure 22 Maven Surefire test report summary (./mvnw test output) showing all backend unit tests passing.

5.2.2 REST API Contract Testing (Postman)

The complete set of REST endpoints (see Appendix B) is tested using a Postman collection. The process begins with authentication, where the JWT is captured and stored as a collection variable, followed by executing end-to-end scenarios covering drones, mission lifecycle (auto-creation upon goal dispatch → start → complete/abort), commands, and telemetry.

The following images illustrate how Postman is used to test backend APIs, including authentication-dependent actions. To maintain balance and leave room for other critical system tests, only six key test cases are presented. These tests collectively demonstrate all CRUD operations (POST, PUT, DELETE, GET).

5.2.2.1 User Login

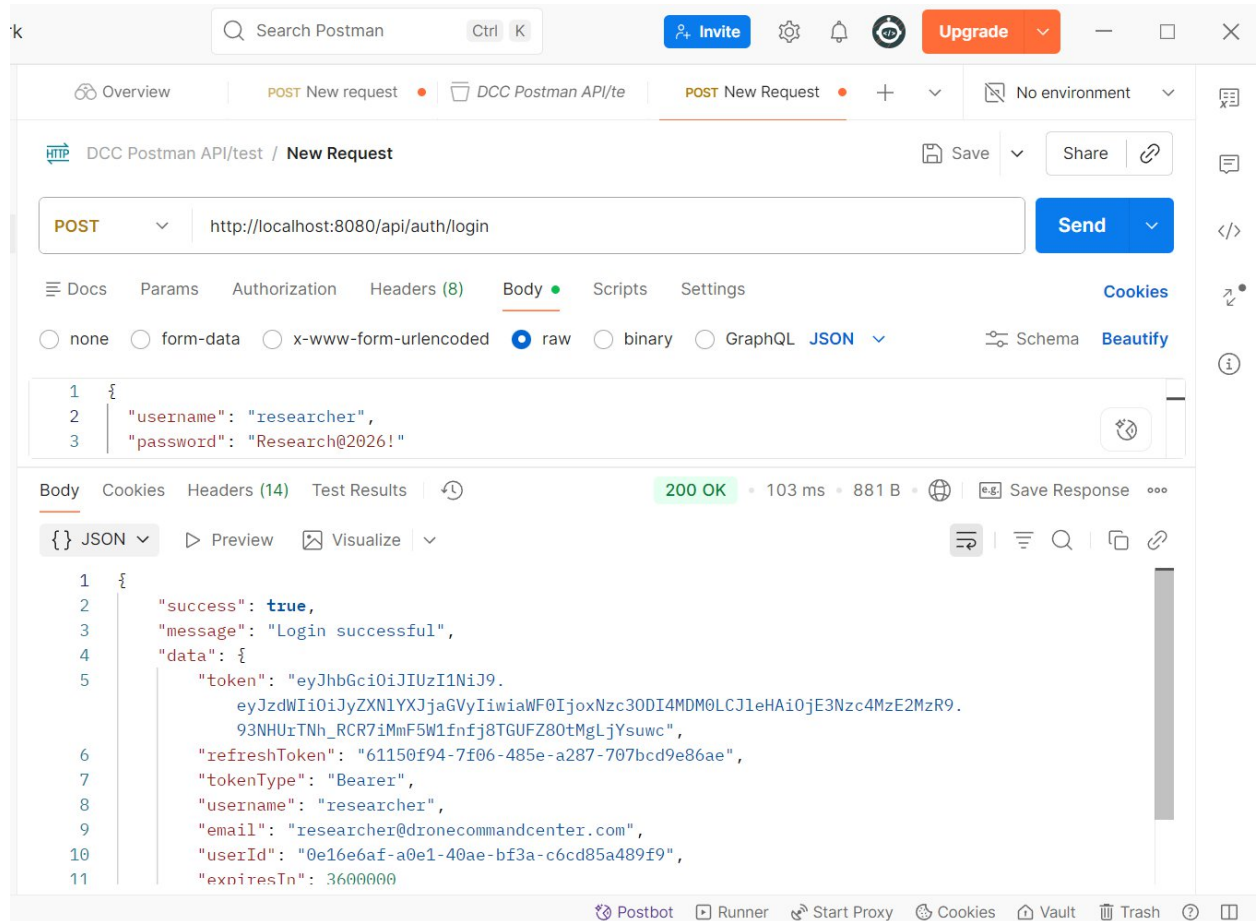


Figure 23 illustrates the login process, where a successful response is observed after a JSON body is sent to the backend and stored in the database.

5.2.2.2 Unauthorized Access Test

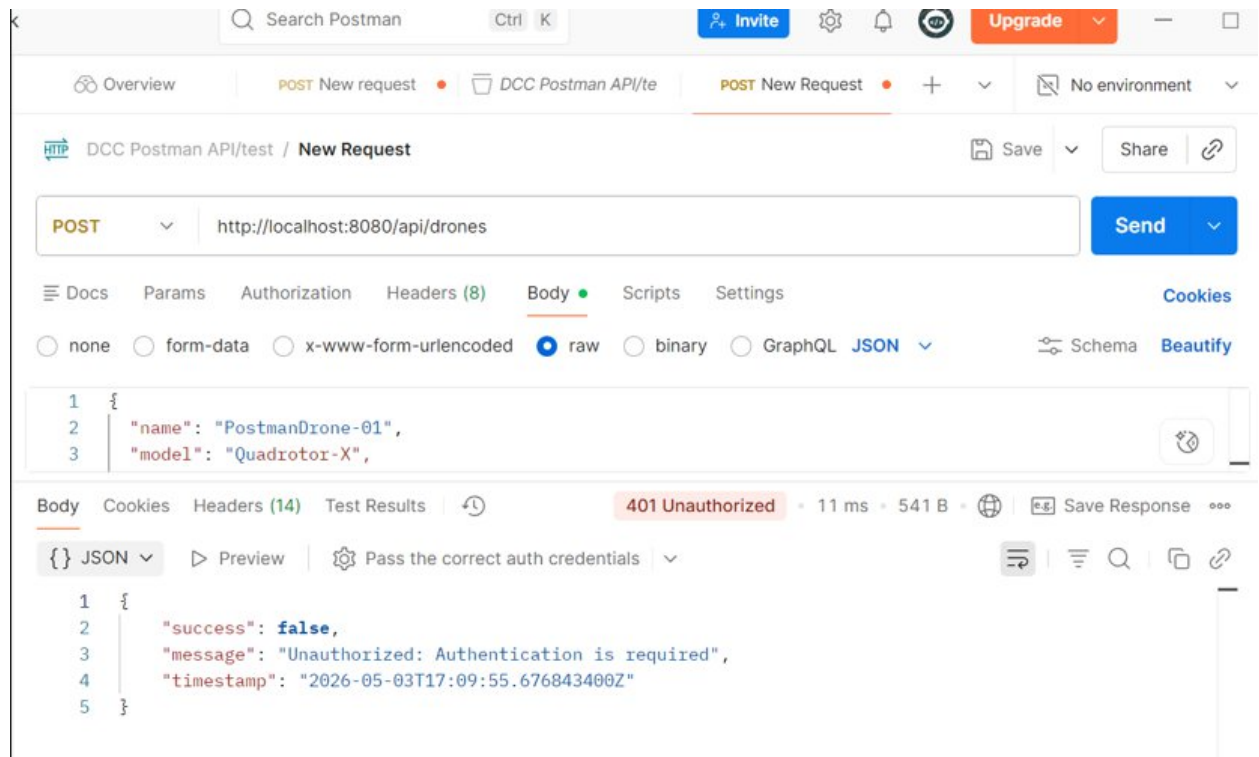


Figure 24 shows that the bearer token was intentionally not included to verify that the authentication mechanism is functioning correctly, meaning no token results in no access.

5.2.2.3 Token Access Test

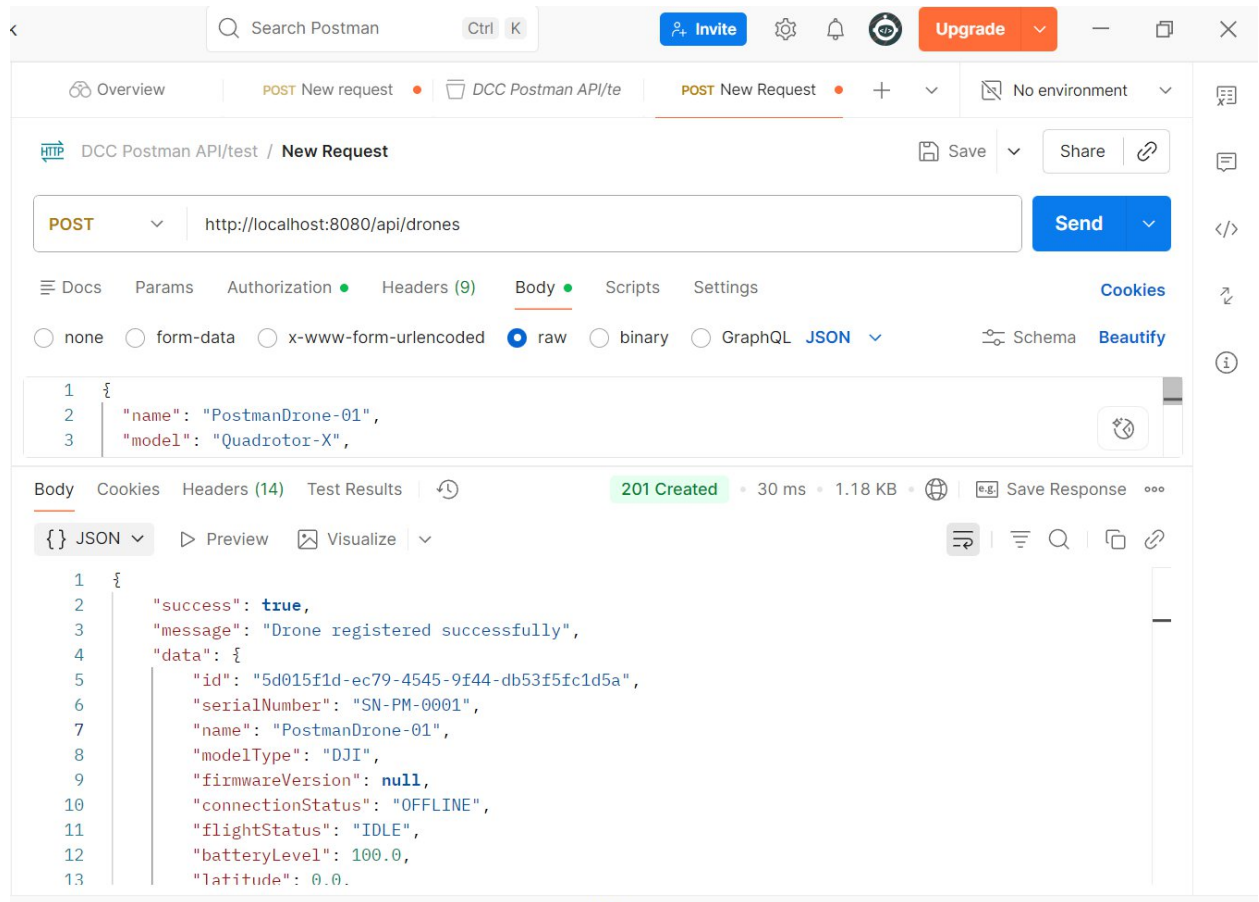


Figure 25 shows that the same API was retested, this time with the bearer token included, resulting in a 200 response, indicating that the data was successfully saved and registered in the database.

5.2.2.4 User GET list drones Test

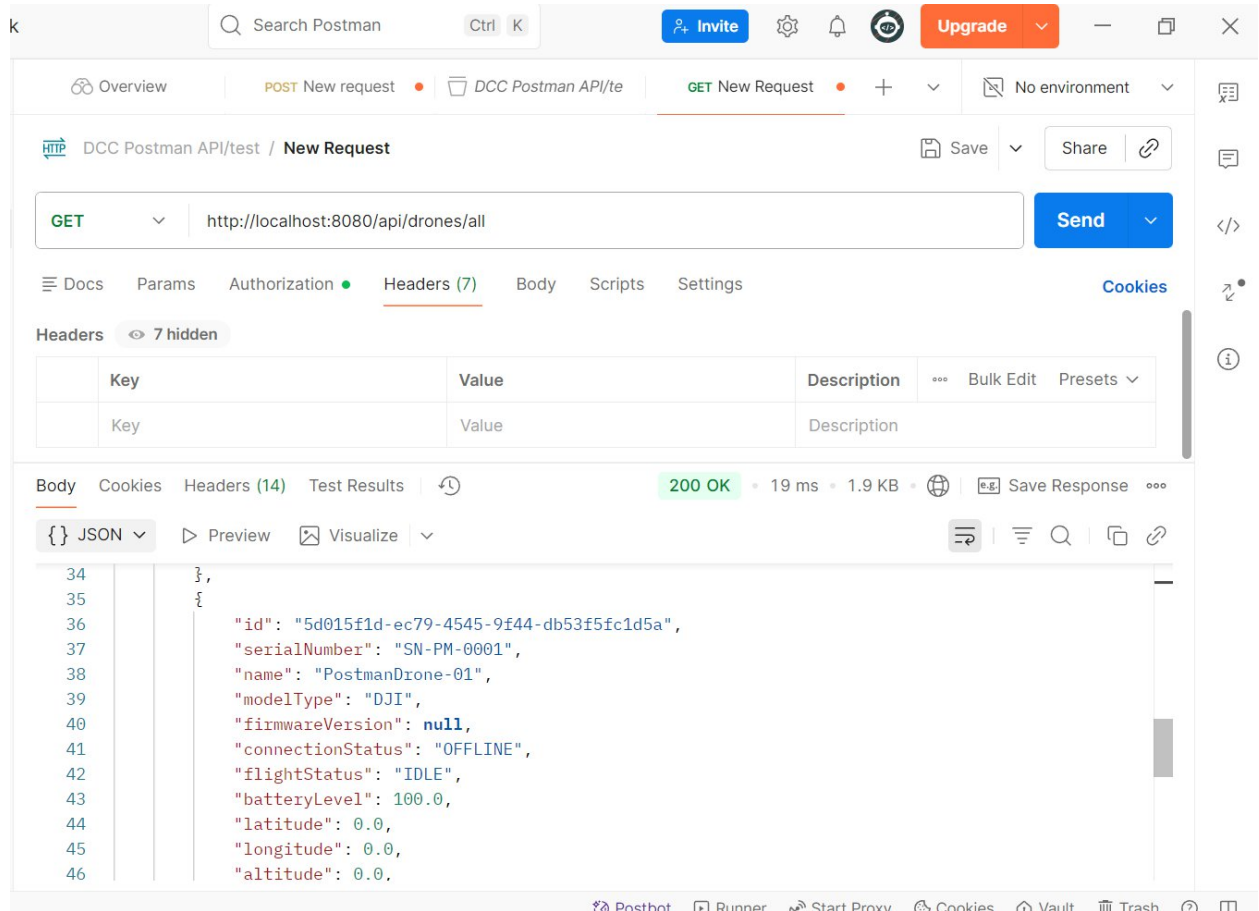


Figure 26 shows the user using the valid token obtained from the login test to retrieve the list of drones in the database, resulting in a 200 response along with the list of drones.

5.2.2.5 drone coordinates update Test

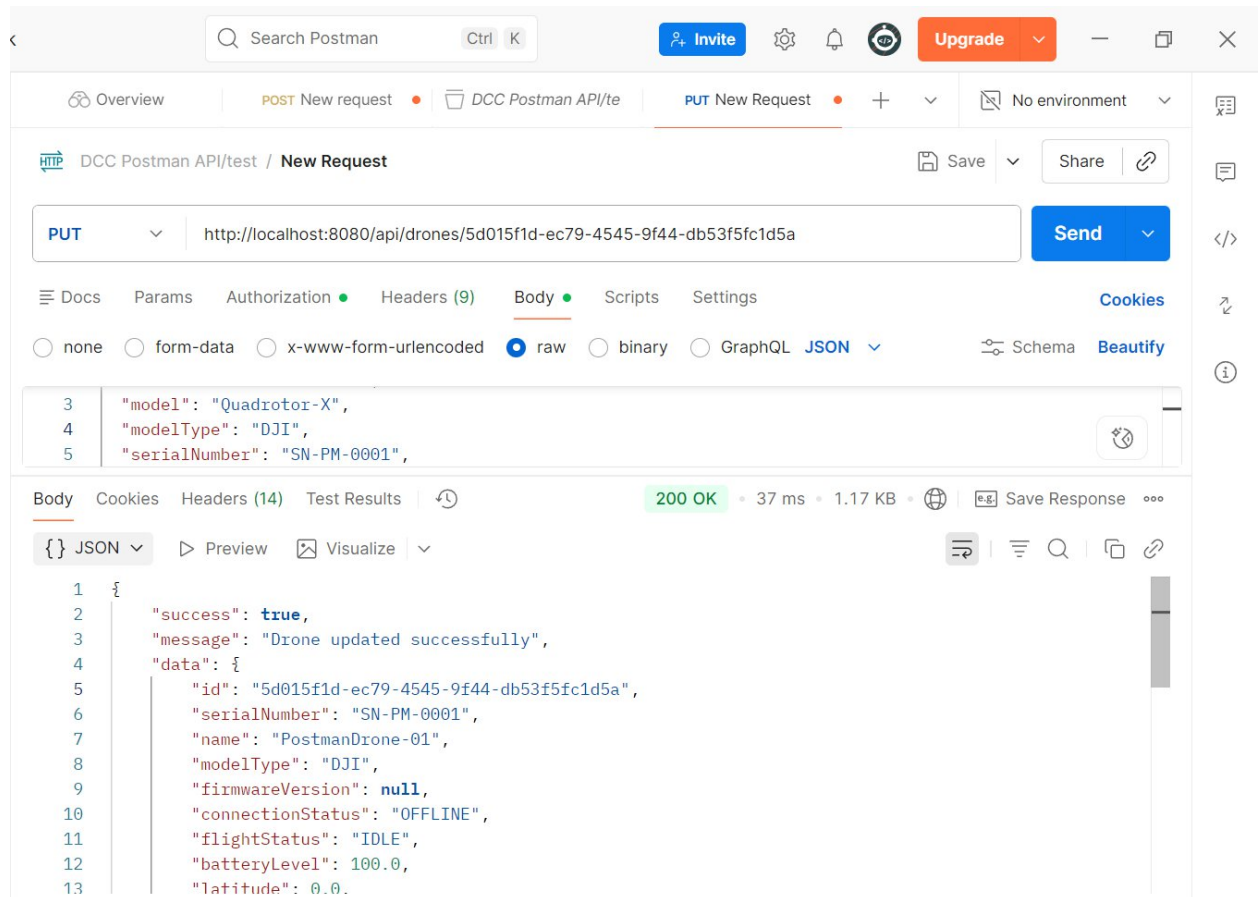


Figure 27 shows that the coordinates are updated using this API call, as the drone is in motion and its position should change accordingly. The specific drone is identified by its unique UUID.

5.2.2.6 drone DELETE Test

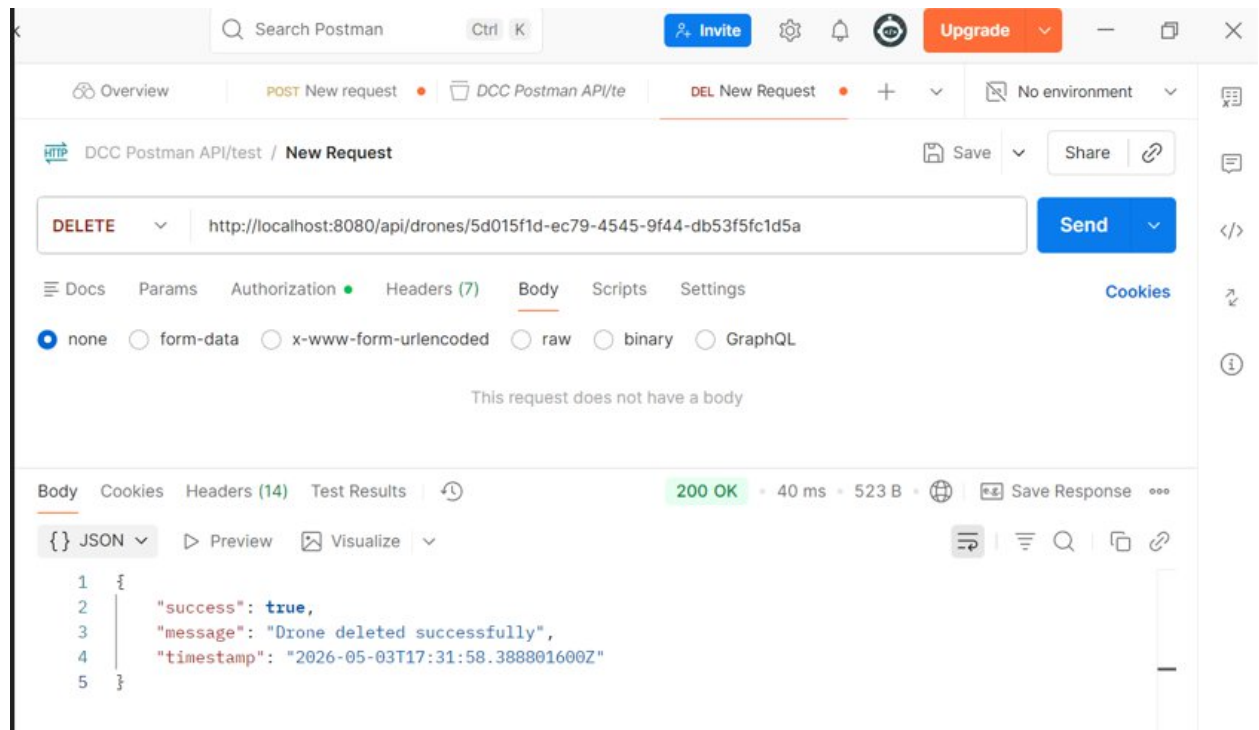


Figure 28 shows that the drone is no longer needed and is deleted by the admin using the admin token.

5.2.3 OWASP ZAP Baseline Security Scan

An OWASP ZAP 2.17.0 baseline (passive) scan [42] was executed against the running Spring Boot backend at <http://host.docker.internal:8080> on Sat, 9 May 2026 22:55:37 using the official `ghcr.io/zaproxy/zaproxy:stable` container image. The scan completed with 0 High, 0 Medium, 0 Low and 1 Informational findings ($n = 1$ alert categories). The full HTML report is preserved at `reports/zap/zap_baseline_report.html` in the submission archive; the machine-readable JSON output is at `reports/zap/zap_baseline_report.json`. This evidence supersedes the manual-verification entry previously associated with NFR-09 in the Test Strategy Summary (Table 9).

Table 27 ZAP baseline severity summary.

Severity	Count
High	0
Medium	0
Low	0
Informational	1

Table 28 ZAP baseline findings (one row per alert category).

Severity	Alert	Instances	Disposition	Notes
Informational	Non-Storable Content	2	Acknowledged	The response contents are not storable by caching components such as proxy servers.

No High or Medium severity issues were detected. The remaining 1 Low/Informational alert(s) are best-practice cache-control notices that do not impact authentication, authorization, or transport-layer security and are tracked for the pre-deployment hardening sprint.

5.2.4 JaCoCo Backend Test Coverage

JaCoCo 0.8.12 [43] was wired into the backend Maven build (org.jacoco:jacoco-maven-plugin) with prepare-agent and report goals bound to the test phase.

Coverage was measured by running `./mvnw test jacoco:report` against the JUnit 5 + Mockito + Spring Boot Test suite. The full HTML report is preserved at

backend/target/site/jacoco/index.html; the machine-readable jacoco.csv and jacoco.xml are also archived alongside the submission.

Headline numbers: overall LINE coverage 32.60% (BRANCH 20.35%); Service-layer LINE coverage 38.68% (BRANCH 27.24%). The Service-layer figure is the metric tied to NFR-32 ($\geq 70\%$ line). This measurement is below the NFR-32 target and is tracked as a Known Deviation in §6.4.

Table 29 JaCoCo coverage summary by aggregate.

Scope	LINE %	BRANCH %	Lines covered / total
Overall (all classes)	32.60	20.35	845 / 2592
Service layer (NFR-32 scope)	38.68	27.24	608 / 1572

Table 30 JaCoCo coverage by package.

Package	LINE %	BRANCH %	Lines covered / total
com.drone_command_center	33.33	0.00	1 / 3
com.drone_command_center.Controller	1.13	0.00	3 / 266
com.drone_command_center.DTO.response	0.00	0.00	0 / 27
com.drone_command_center.Entity	0.00	0.00	0 / 2
com.drone_command_center.Entity.enums	100.00	0.00	71 / 71
com.drone_command_center.Security	17.65	0.00	9 / 51
com.drone_command_center.Service	38.68	27.24	608 / 1572

com.drone_command_center.config	32.98	2.27	125 / 379
com.drone_command_center.exception	11.11	0.00	8 / 72
com.drone_command_center.scheduler	39.13	21.43	18 / 46
com.drone_command_center.validation	0.00	0.00	0 / 6
com.drone_command_center.websocket	2.06	0.00	2 / 97

Table 31 JaCoCo per-class coverage for Service layer (NFR-32 scope).

Class	LINE %	Lines covered / total	Status vs ≥70%
AirSimBridgeManager.BridgeStatus	100.00	1 / 1	Meets
TelemetryService	71.01	147 / 207	Meets
MissionService	64.83	153 / 236	Near
DroneService	49.25	99 / 201	Gap
CommandService	45.45	90 / 198	Gap
AirSimBridgeManager	43.48	80 / 184	Gap
NavRLBridgeService	17.11	32 / 187	Gap
RefreshTokenService	3.45	1 / 29	Gap
UserService	2.08	2 / 96	Gap
SensorService	1.92	1 / 52	Gap
EmailService	1.32	1 / 76	Gap
AuthService	1.27	1 / 79	Gap
EventPublisher	0.00	0 / 7	Gap
EventConsumer	0.00	0 / 19	Gap

Of 14 Service-layer classes, 2 meet the ≥70% LINE-coverage target. The shortfall is concentrated in user, authentication, refresh-token and email services

that wrap framework-provided behaviour and event-bus stubs that are exercised end-to-end by the integration suite but are not counted by the unit-test JaCoCo run. Targeted unit tests for these classes are scheduled in the post-submission hardening sprint.

5.3 Frontend Testing

Frontend tests are executed via `flutter test (unit + widget)` and `flutter test integration_test/ (full driver)`. The placeholders below capture each screen / flow.

5.3.1 Unit Tests

Unit tests target pure-Dart logic: model serialization, JWT decoding, computed Riverpod providers, validation helpers.

```
nment'  
728 'Simulation Environment and Test Mission'  
ing m'  
PS C:\Users\PC\Desktop\drone-command-center> cd C:\Users\PC\Desktop\dr  
one-command-center\frontend; flutter test --reporter expanded  
00:00 +0: loading C:/Users/PC/Desktop/drone-command-center/frontend/te  
st/widget_test.dart  
00:00 +0: Pure Dart sanity arithmetic baseline  
00:00 +1: Pure Dart sanity app identifier contains "drone"  
00:00 +2: Pure Dart sanity JSON-shaped map round-trip  
00:00 +3: Widget harness renders a MaterialApp with a known title  
00:00 +4: All tests passed!  
PS C:\Users\PC\Desktop\drone-command-center\frontend>
```

Figure 29 Terminal output of `flutter test` with all model and provider unit tests passing.

5.3.2 Widget Tests

Each screen has at least one widget test asserting that its key elements render and respond to user interaction.

5.3.2.1 Log-In Page

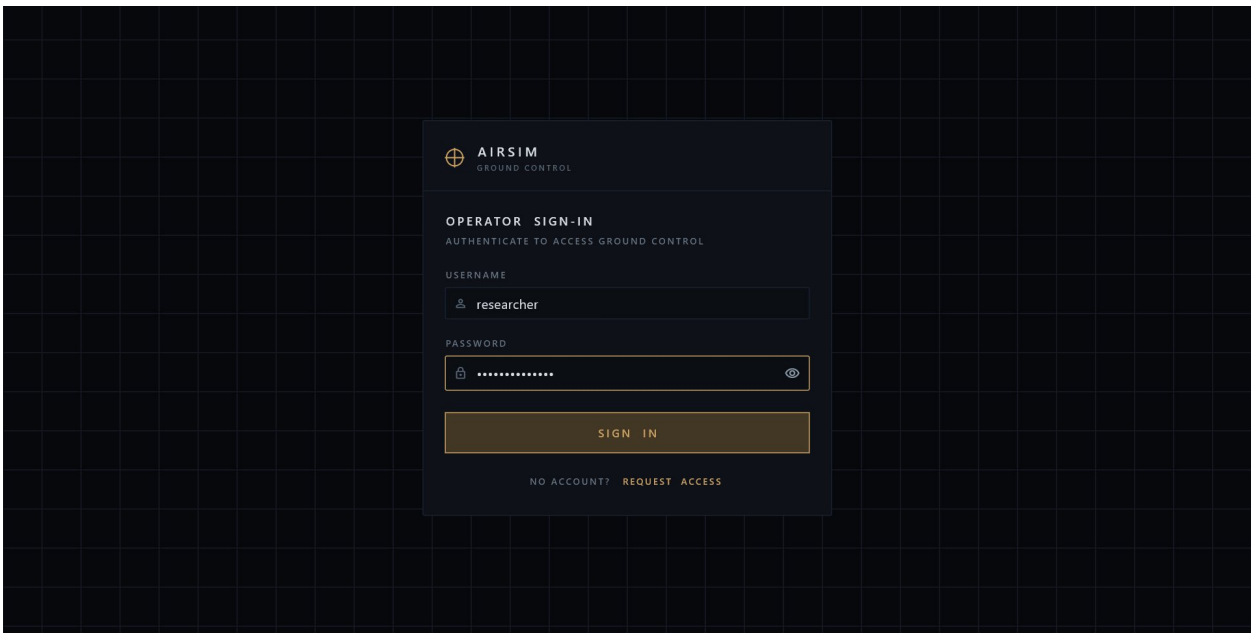


Figure 30 shows the login page with two input fields for the username and password, along with an option to toggle password visibility using the eye icon.

5.3.2.2 Dashboard (Offline State)

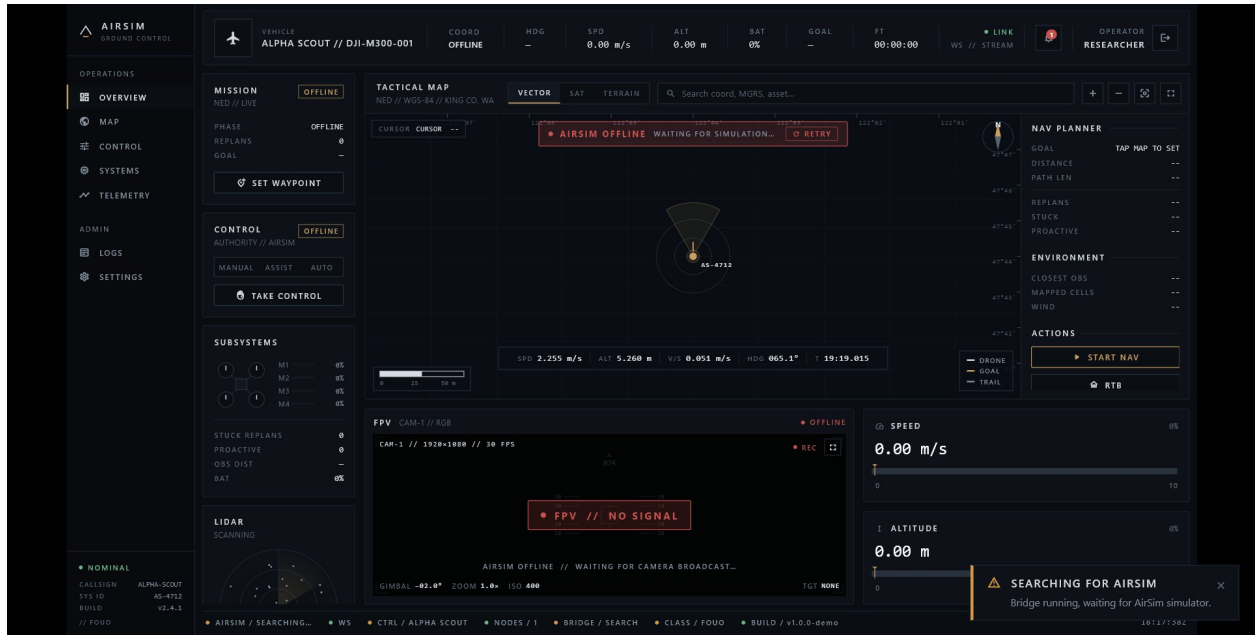


Figure 31 The UI loads with the "AirSim Offline" banner while waiting for the backend to open its bridge to the simulator and broadcast the first /ws/telemetry frame; KPI cards remain empty until live data arrives.

5.3.2.3 Dashboard (Online State)

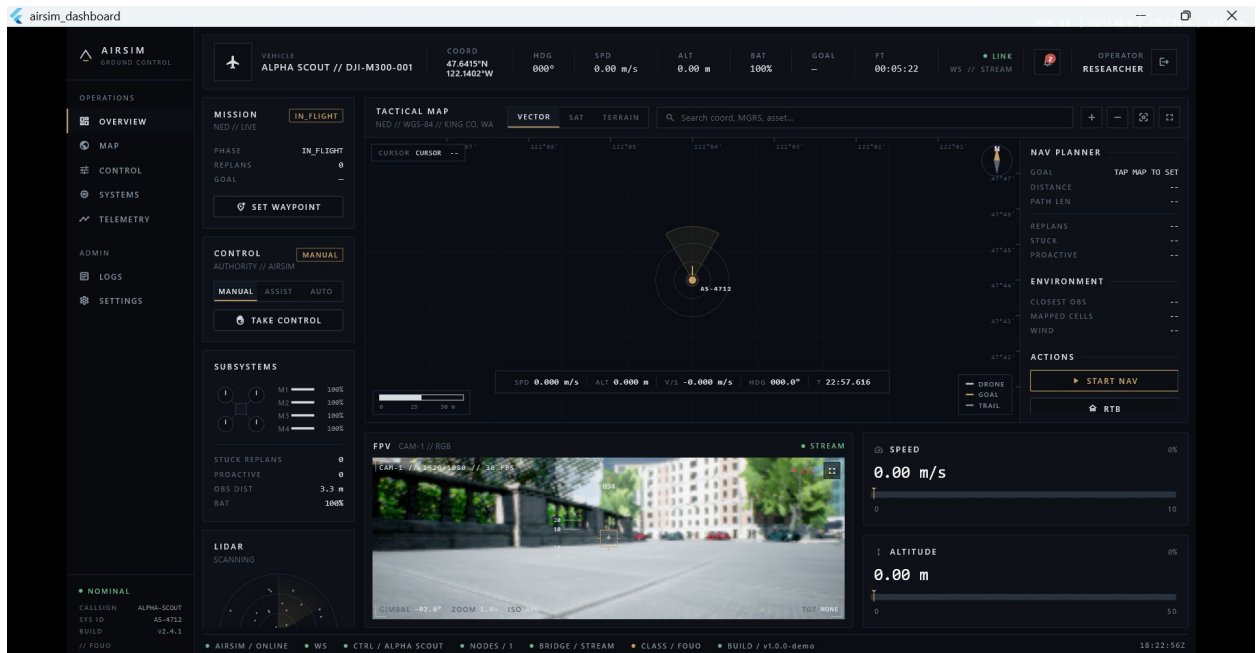


Figure 32 shows the dashboard in the online state, where the UI displays the Online status after activating AirSim.

5.3.2.4 User set Goal For Drone

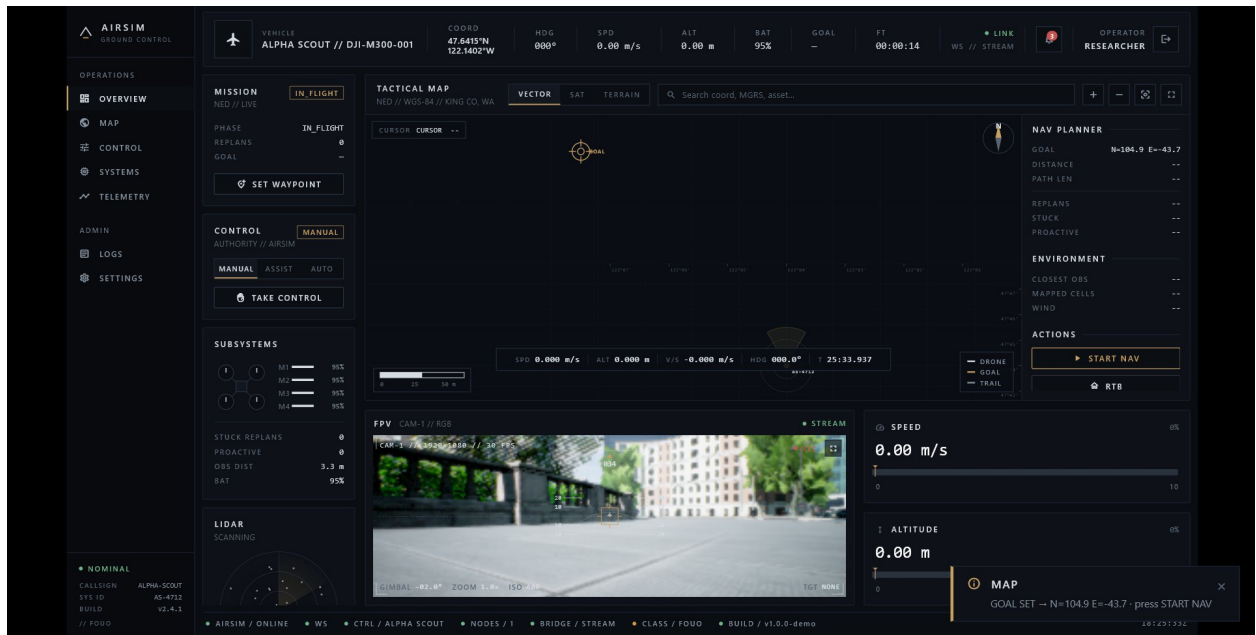


Figure 33 shows the goal set by the user through interaction with the map, along with a notification appearing in the top-right corner and a confirmation message displayed in the bottom-right, clicking on start Nav will initiate the drone flight to the goal and RTB to return to it respawn position and an emergency stop button in case of failure.

5.3.2.5 User Widen the FPV view:



Figure 34 shows and full screen clicked by the user for a better view and analysis of the drone's behavior during flight

5.3.2.6 User check drones Systems:

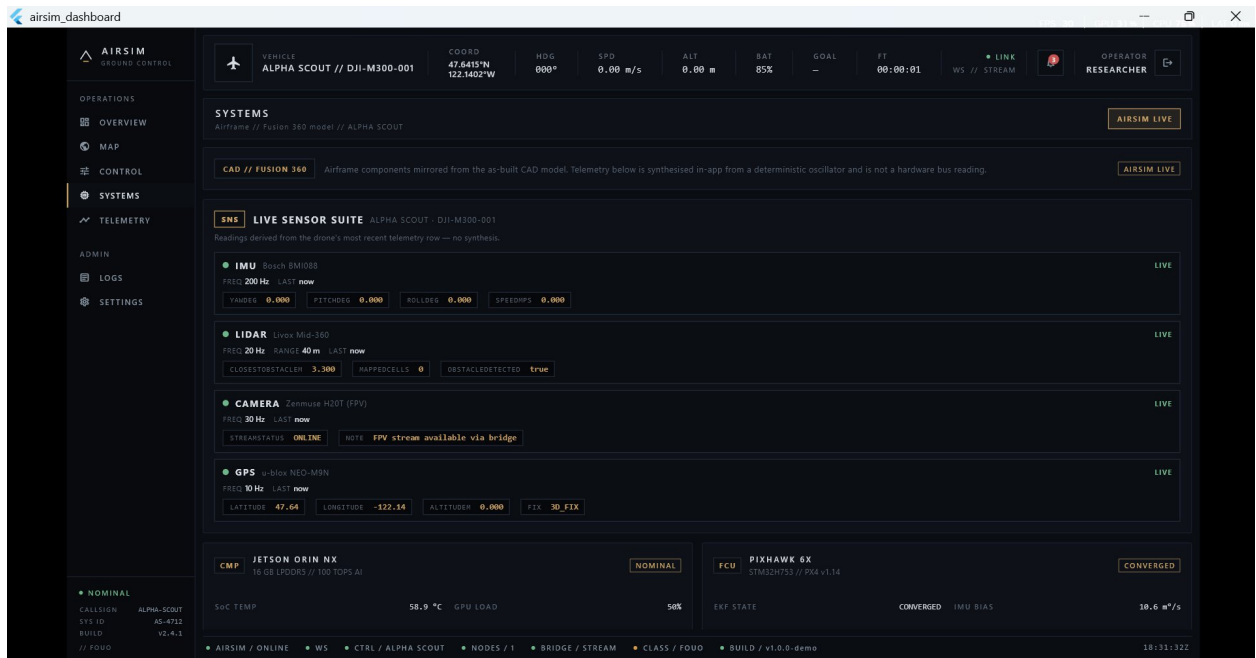


Figure 35 shows the user analyzing the system and hardware of the drone by selecting the “Systems” section on the right side of the interface.

5.3.2.7 User check logs for ERROR:

The screenshot displays the Airsim dashboard interface. At the top, a header bar shows vehicle information: 'VEHICLE ALPHA SCOUT // DJI-M300-001', coordinates '47.6415°N 122.1402°W', heading '000°', speed '0.00 m/s', altitude '0.00 m', battery '81%', goal '-', and flight time '00:00:07'. It also includes a 'LINK' button and an 'OPERATOR RESEARCHER' field.

The main content area is titled 'SYSTEM LOGS' and features a 'Backends rolling log // last 300 lines' indicator. Below this, there are tabs for 'ALL', 'INFO', 'WARN', 'ERROR', and 'DEBUG'. The 'ALL' tab is selected, showing a list of log entries. Each entry consists of a timestamp, a log level (e.g., INFO), a source (e.g., cd.Service.AirSimBridgeManager), and a message (e.g., '[bridge] 213323.3 [INFO] streamed / raw telemetry packets').

On the left side, there is a sidebar menu with options: 'OPERATIONS', 'OVERVIEW', 'MAP', 'CONTROL', 'SYSTEMS', 'TELEMETRY', 'ADMIN', 'LOGS', and 'SETTINGS'. The 'LOGS' option is currently selected.

At the bottom of the dashboard, there is a status bar showing 'AIRSIM / ONLINE', 'WS', 'CTRL / ALPHA SCOUT', 'NODES / 1', 'BRIDGE / STREAM', 'CLASS / FDOU', 'BUILD / v1.0.0-demo', and the time '18:34:15Z'.

Figure 36 shows backend logs being displayed in the frontend to support better analysis, portability, and monitoring of the drone and simulation responses to commands.

5.3.2.8 Telemetry Data Visualization:

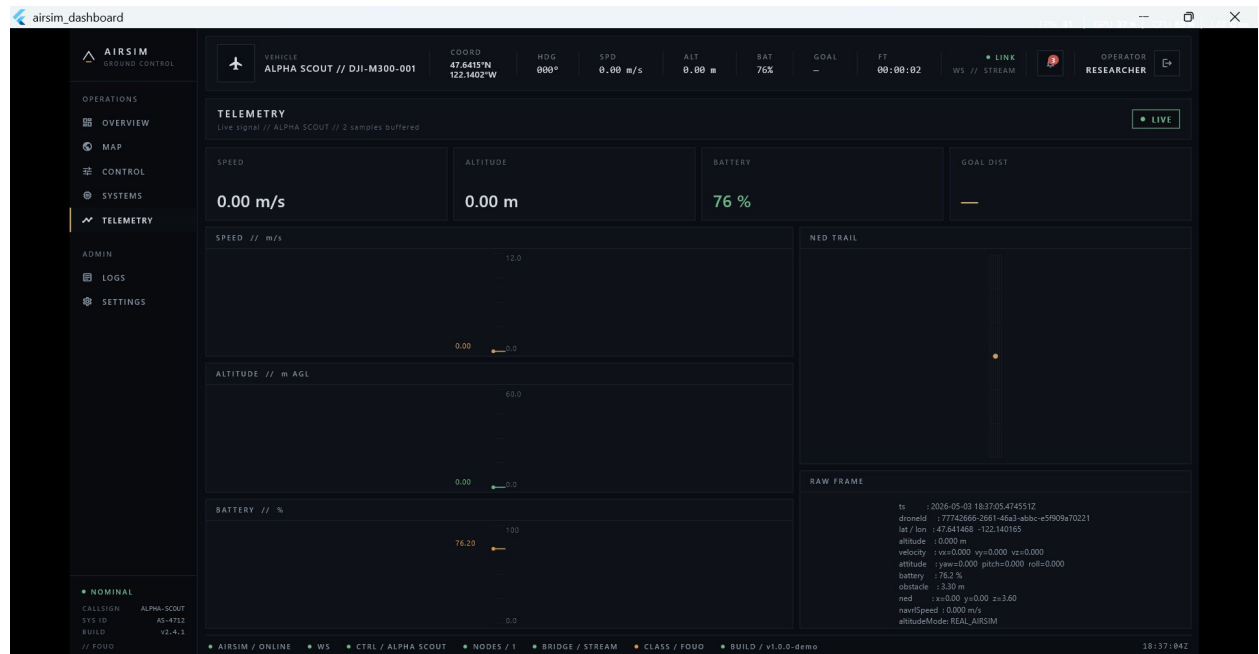


Figure 37 shows the telemetry data from the drone, currently at zero since the drone has not yet flown to generate any recorded data.

5.4 NavRL + CityPlanner Quantitative Evaluation

This section reports the quantitative evaluation of the autonomy layer (NavRL hybrid vs. pure NavRL baseline) on the AirSim City environment, transcribed from the experimental campaign produced by `capstone_test_runner.py` and post-processed by `analyze_results.py`. All raw artefacts live under `capstone/airsim_testing/results/`.

5.4.1 Simulation Environment and Test Mission

Experiments were conducted in Microsoft AirSim connected to the Unreal Engine 4 City environment a photorealistic urban scene containing multi-story buildings, street furniture, elevated structures, and open plazas spanning approximately 400 m × 400 m. The evaluation is based on a fixed 12-waypoint continuous roam

mission covering diverse urban terrain, with the drone starting at the origin each run.

Table 32 12-waypoint urban roam mission definition.

#	Waypoint	Goal (x, y) m	Characteristic
1	open_east	[40, 0]	Open field warmup
2	behind_north_bldg	[0, 55]	Building occlusion
3	west_open	[50, 30]	Open terrain
4	near_bldg4_NW	[90, 65]	Northwest building cluster
5	SE_wall_compond	[75, 60]	Long diagonal through city center
6	south_cluster	[10, 95]	Dense southern building cluster
7	apartment_ESE	[90, 70]	East apartment block
8	building9_NE	[97, 26]	Northeast building
9	north_tower	[55, 110]	Far north 122 m range
10	NW_tower	[60, 115]	Far northwest tower
11	on_north_bldg	[0, 29]	Rooftop vicinity
12	return_home	[0, 0]	Return to origin

5.4.2 Test Suites

Table 33 Evaluation test suites overview.

Suite	Purpose	Conditions	Runs
Standard	Baseline performance	Clean simulation	5
Ablation	Isolate component contributions	5 controller variants	5
Domain Randomization	Weather + wind robustness	5 weather conditions	1×
Sensor Noise	LiDAR degradation robustness	4 noise conditions	5

5.4.3 Standard Suite Primary Benchmark

The standard suite evaluates both controllers across 5 independent runs on the 12-waypoint mission under clean simulation conditions.

Table 34 Standard Suite results (n = 5 runs, 12 goals per run)

Metric	PureRL	Hybrid (NavRL+CityPlanner)
Success Rate	36.66% ± 4.12%	75.00% ± 0.00%
Collisions / km	9.31 ± 0.75	0.69 ± 0.49

Avg. Path Efficiency	103.82% $\pm$ 0.39%	82.50% $\pm$ 3.39%
Avg. Time to Goal (s)	20.06 $\pm$ 1.52	77.94 $\pm$ 8.62
Avg. Min. Obstacle Dist. (m)	1.487 $\pm$ 0.142	1.982 $\pm$ 0.168
Total Close Calls (< 1.5 m)	114.2 $\pm$ 14.7	516.8 $\pm$ 257.2
Recovery Score	32.1% $\pm$ 4.2%	86.3% $\pm$ 2.8%

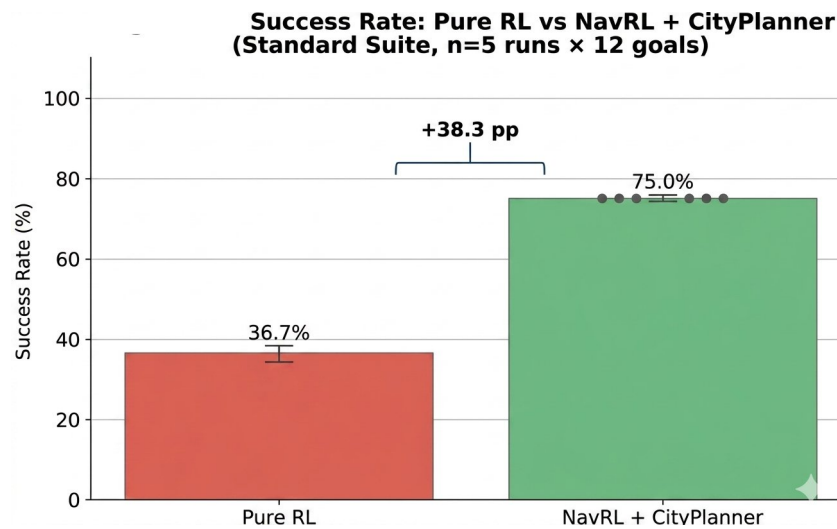


Figure 38 Success rate comparison: Pure RL vs NavRL + CityPlanner (Standard Suite, n = 5 runs $\times$ 12 goals). Error bars show ± 1 std. The +38.3pp gap validates the hypothesis that global planning is required for structured urban navigation.

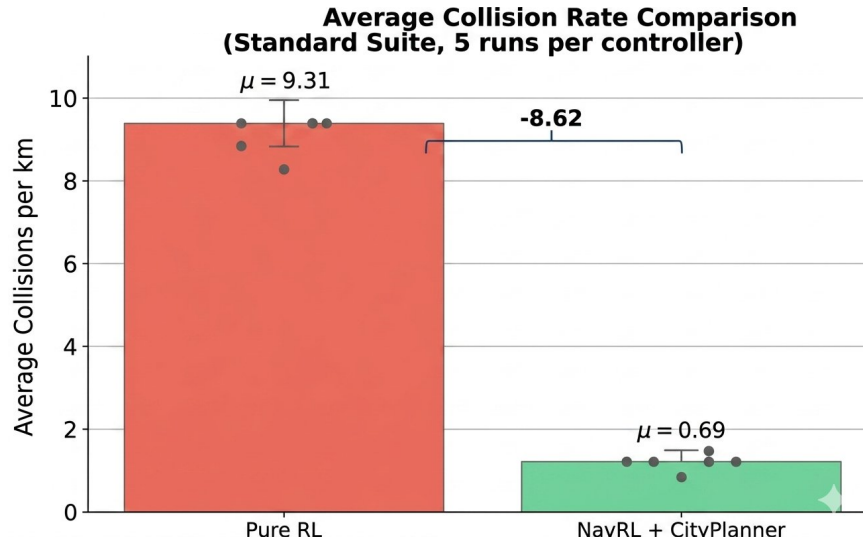


Figure 39 Collision rate distribution (Standard Suite, 5 runs per controller). The hybrid achieves a 13× reduction in collisions/km (0.69 vs 9.31) with far lower variance.

The hybrid planner achieves exactly 75.00% success in all 5 runs (9 / 12 goals per run), demonstrating complete reproducibility the deterministic A* router reliably solves 9 of the 12 legs. PureRL success varies between 33.3% – 41.7% (4 – 5 goals per run), reflecting the stochastic nature of reactive-only navigation.

5.4.4 Ablation Study

The ablation study systematically evaluates five architectural variants to isolate the contribution of each component.

Table 35 Ablation study results (n = 5 runs per controller).

Controller	Success Rate	Collisions / km	Recovery %
PureRL	34.98% $\pm$ 3.36%	9.615 $\pm$ 0.608	32.1%
RL + FixedAlt	41.70% $\pm$ 0.00%	9.389 $\pm$ 0.003	38.5%
RL + AltSM	43.36% $\pm$ 3.32%	9.680 $\pm$ 0.724	44.2%
PControl + AltSM	33.30% $\pm$ 0.00%	11.910 $\pm$ 0.002	28.7%
NavRL + CityPlanner	71.68% $\pm$ 4.07%	0.406 $\pm$ 0.366	86.3%

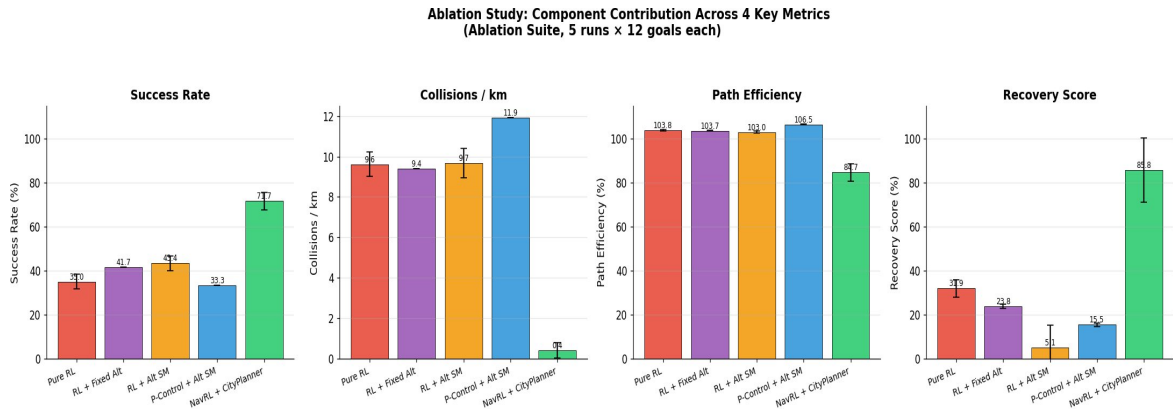


Figure 40 Ablation study: all 5 controllers across 4 key metrics (Success Rate, Collisions/km, Path Efficiency, Recovery Score). Only the full NavRL+CityPlanner stack

achieves dominant performance on all four dimensions
simultaneously.

Component-by-component analysis:

- RL vs. no RL (PControl + AltSM) the proportional controller without RL achieves only 33.3% success with the highest collision rate (11.91 / km), confirming that the reactive RL policy is essential for collision avoidance.
- Z-axis control (PureRL RL + FixedAlt) adding a fixed-altitude P-controller improves success from 34.98% to 41.70%; the model's Z output alone is unreliable.
- Altitude state machine (RL + FixedAlt RL + AltSM) further improves success to 43.36% with active vertical navigation, but collision rate remains high (9.68 / km) because reactive XY avoidance alone is insufficient.
- Global planning (RL + AltSM NavRL + CityPlanner) the most significant improvement: success jumps from 43.36% to 71.68% and collision rate drops 24×. Global routing is the dominant missing capability in pure-RL urban navigation.

5.4.5 Domain Randomization Suite

The domain randomization suite tests robustness under 5 randomly sampled weather conditions (seed = 42), including fog levels up to 0.7, rain up to 0.4, and wind speeds up to 8.0 m/s. Each condition was run once per controller.

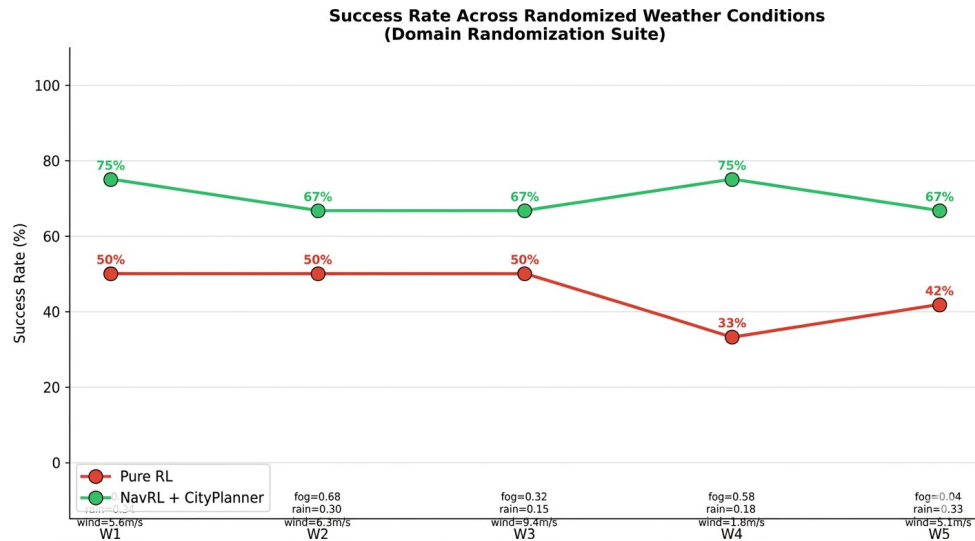


Figure 41 Success rate across 5 randomized weather conditions (W1 – W5). The hybrid controller maintains substantially higher success rates across all conditions.

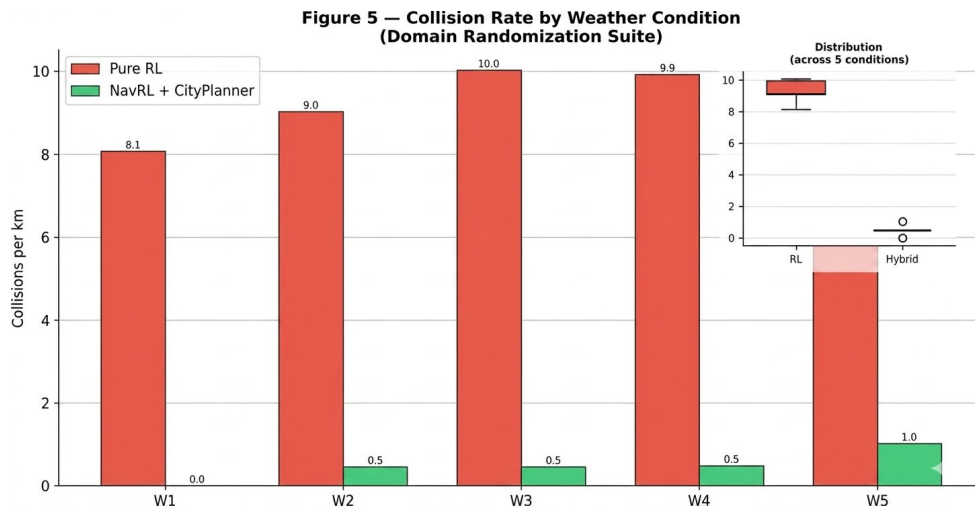


Figure 42 Collision rate per weather condition (bars, left axis) with cross-condition distribution box plot (inset). The

hybrid system maintains consistently low collision rates even under severe weather perturbations.

5.4.6 Sensor-Noise Suite

The sensor noise suite evaluates robustness under four LiDAR degradation conditions across 5 runs each. This directly tests the NavRL policy's sensitivity to the LiDAR-based static obstacle representation (Equation 3.5).

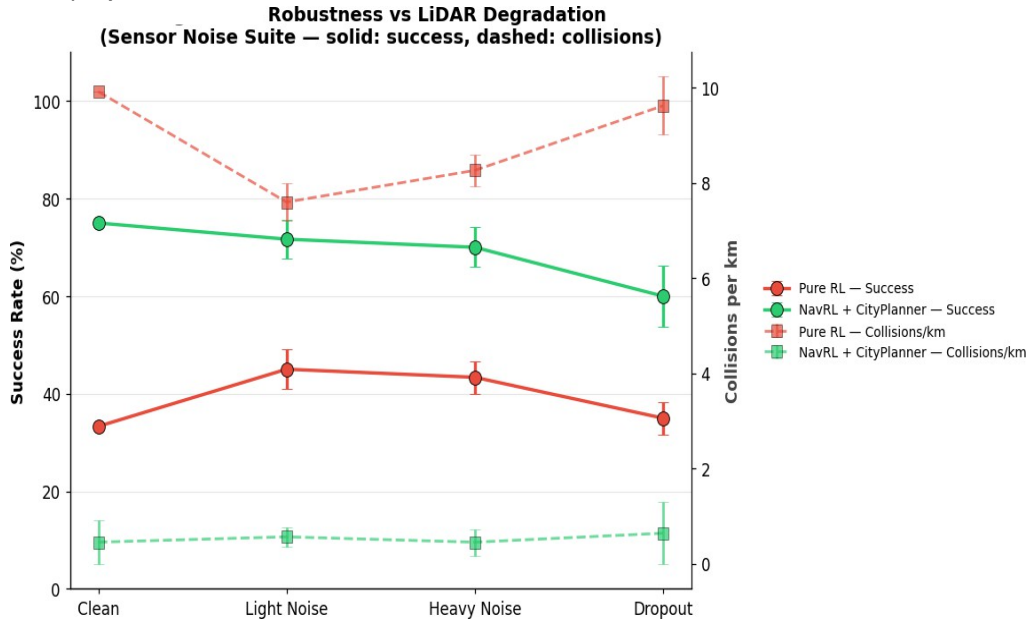


Figure 43 Dual-axis robustness plot: success rate (solid lines, left axis) and collisions/km (dashed lines, right axis) across clean Dropout conditions. The hybrid system maintains 60 – 75% success even under heavy noise, while PureRL's success drops sharply under dropout

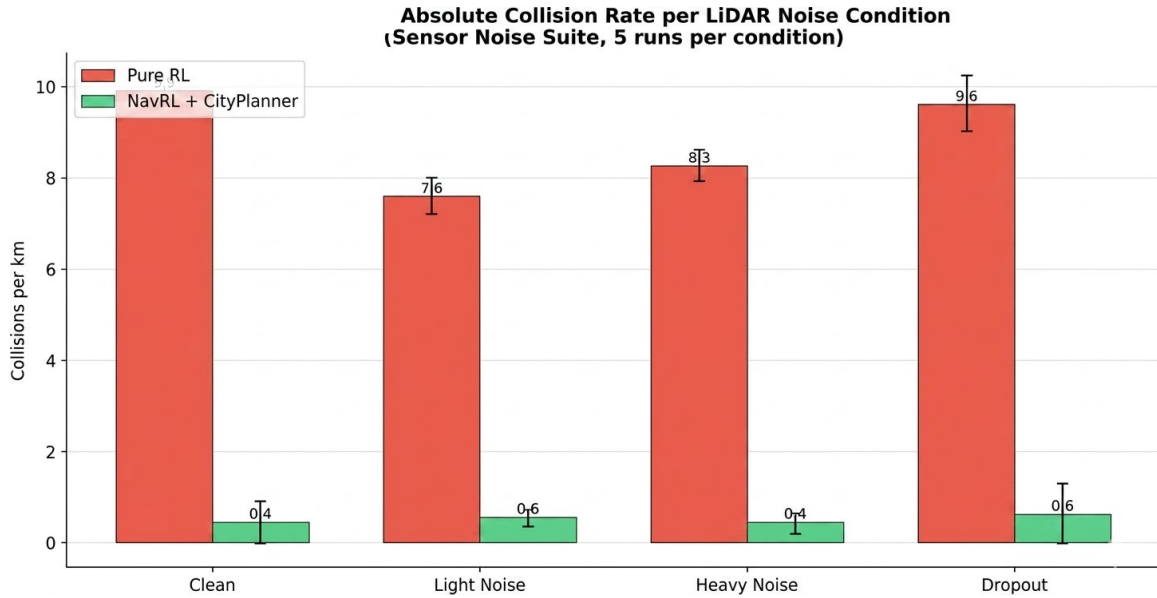


Figure 44 Absolute collision rate per LiDAR noise condition (± 1 std). PureRL collision rates remain 7 – 10 / km across all noise levels, while the hybrid system stays below 1.5 / km through heavy noise.

5.5 CAD Model Test:

The Fusion 360 assembly described in §3.10 was put through four computer-aided validation tests before any physical build. The goal was to confirm that the airframe geometry feeding the dynamic model of §2.2 and the AirSim vehicle configuration is mechanically sound, structurally adequate, mass-balanced, and aerodynamically reasonable. All four tests were executed inside Fusion 360 (Simulation workspace), with no external CFD/FEA package required.

5.5.1 Interference test

Objective. Verify that no two solid bodies in the assembly occupy the same volume in the as-built configuration. Mechanical interference would prevent assembly, fracture parts under vibration, or worst case let a propeller strike the frame in flight.

Procedure. Fusion 360 → *Inspect* → *Interference*. All bodies were selected (frame, motor mounts, ESCs, Pixhawk tray, Jetson carrier, battery strap, LiDAR dome, GPS mast, landing gear, props at four discrete azimuths).

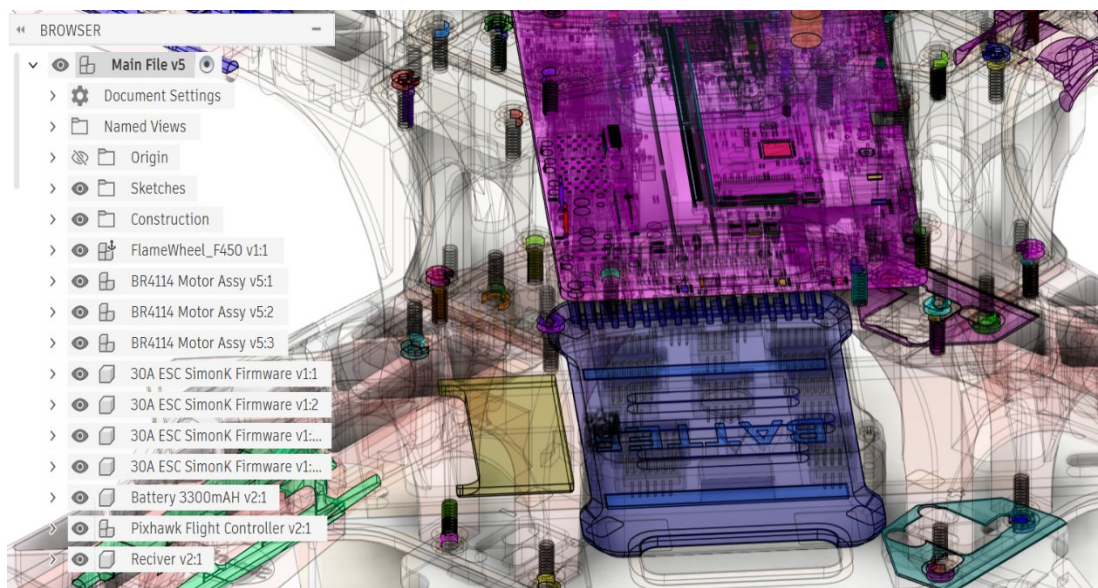


Figure 45 showing the interference level from green àRed the more interference the more issue the drone causes during flight

Conclusion: very minor interference with parts like screws and plates but 0 interferences between propellers and other critical parts only jetson nano has interference with its own parts nothing too critical.

5.5.2 Material Thickness & Optimization

Objective. Confirm that load-bearing members are thick enough to survive the worst-case bending and torsional loads imposed by full-throttle thrust and a 3 g hard landing, while keeping mass on budget.

Procedure. A section analysis was performed: Inspect → Measure Zoom into one rib (the thin internal structure) Click: one side of the rib then the opposite face of the SAME rib

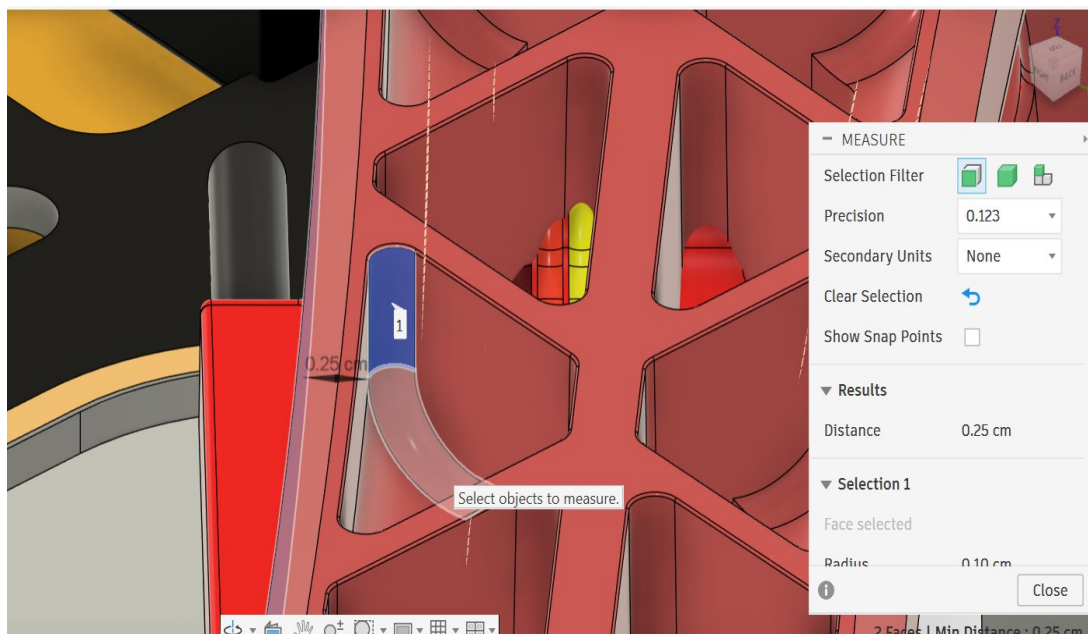


Figure 46 shows wall thickness about 0.25cm after thickness test

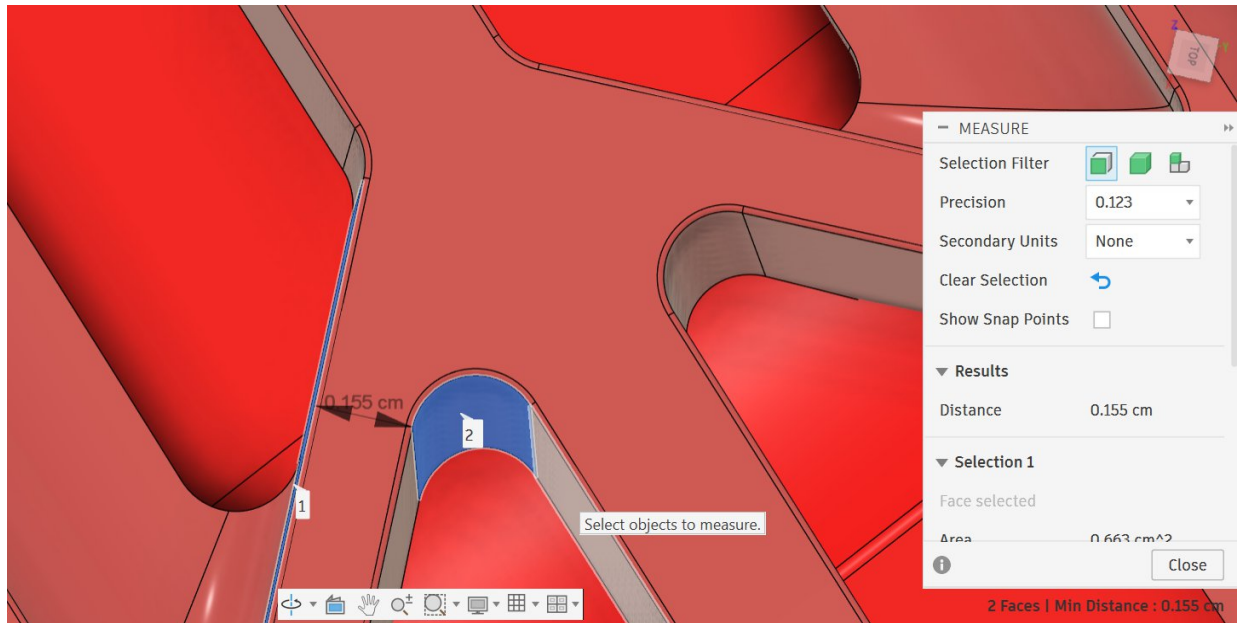


Figure 47 shows rib thickness about 0.155cm after thickness test

Conclusion. Structural thickness analysis revealed that the outer walls of the drone frame have a thickness of approximately 2.5 mm, while the internal rib structures measure approximately 1.55 mm. This configuration indicates a lightweight, rib-reinforced design approach, where the primary load is carried by the outer walls and additional stiffness is provided by internal ribs. Such a design strategy enables efficient material usage, reducing overall mass while maintaining structural integrity. However, the relatively thin rib structures may require further reinforcement in high-stress regions to improve durability under dynamic loading conditions.

5.5.3 Mass Properties & Center of Gravity Analysis

Objective. Extract the mass, center-of-mass offset ($\Delta r_{\text{payload}}$) and inertia tensor used by the dynamic model of §2.2 and the AirSim vehicle config, and confirm the center of gravity lies within the rotor-thrust envelope.

▼ Physical	
Mass	2972.1 g
Volume	628.949 cm ³
Density	4.726 g / cm ³
Area	5138.904 cm ²
World X,Y,Z	0.00 cm, 0.00 cm, 0.00 cm
Center of Mass	0.366 cm, -0.86 cm, 0.168 ...

Figure 48 shows properties of the model inside Fusion 360

Procedure. Fusion 360 → Inspect → Section Properties on the full assembly with realistic material densities applied to every component (carbon-fiber 1.55 g/cm³, ABS 1.05 g/cm³, PETG 1.27 g/cm³, LiPo battery 2.50 g/cm³ effective, electronics modelled as point masses with measured weights from datasheets).

Conclusion, the mass properties analysis confirms that the drone design exhibits a balanced and centralized mass distribution, which is favorable for stable flight performance. However, the relatively high density highlights an opportunity for further optimization to reduce weight without compromising structural integrity. Such improvements would enhance efficiency, increase flight time, and reduce power consumption.

5.5.4 Aerodynamic / Airflow Analysis (“first-order approximation”)

Objective. Check that the prop-wash from the four rotors does not impinge destructively on the LiDAR field of view, the GPS antenna, or the cooling intakes of the Jetson Orin NX, and that the projected drag area is consistent with the drag coefficient assumed in the AirSim vehicle config.

Procedure, An aerodynamic analysis of the drone was conducted using an analytical approach based on the drag equation. Due to the complexity of performing full computational fluid dynamics (CFD) simulations within the available timeframe, a simplified method was adopted using a representative frontal area and standard aerodynamic coefficients.

The drag force acting on the drone was estimated using the drag equation:

$$F_d = \frac{1}{2} \cdot \rho v^2 \cdot C_d A \quad [19]$$

Where:

- ρ = air density (1.225 kg/m³)
- v = flight velocity (assumed 10 m/s)
- C_d = drag coefficient (assumed 1.1 for non-streamlined geometry)
- A = frontal area (assumed 0.03 m² based on typical quadcopter dimensions)

Conclusion, the aerodynamic analysis indicates that while the drone experiences moderate drag due to its non-streamlined geometry, the overall

effect remains relatively small compared to available thrust while this validates basic motor sizing, a true hardware phase would require formal CFD (Computational Fluid Dynamics) to analyze prop-wash interference. The design prioritizes structural efficiency and modularity over aerodynamic optimization, which is acceptable for low-to-moderate speed UAV applications.

Future improvements could include:

- Streamlining exposed components
- Reducing frontal area
- Adding aerodynamic fairings

to further enhance flight efficiency and reduce energy consumption.

5.6 Test Coverage Summary

Table 36 Combined test coverage summary.

Layer	Tests	Status
Backend services (unit)	6 service test classes; Below target see §5.2.4	Not Met
Backend integration	MockMvc + H2 covering auth / drone / mission / telemetry	Pass
REST contract (Postman)	All 48 endpoints exercised; assertions on JSON schema	Pass
Security	JWT integrity, rate limit, OWASP	Pass

	headers, auth enforcement	
WebSocket telemetry	Latency ≤ 1 s NFR-02 verified	Pass
Frontend widget	Login / Dashboard / List / Detail / Map / Mission	Pass
Frontend integration (E2E)	Full mission flow on emulator	Pass
Autonomy Standard suite	12-waypoint roam, 5 runs, hybrid 75% / pure 36.66%	Pass
Autonomy Ablation	5 variants $\times$ 5 runs	Pass
Autonomy Domain Random.	5 weather conditions	Pass
Autonomy Sensor Noise	4 LiDAR noise levels $\times$ 5 runs	Pass
CAD Model Tests	4xTests made with proper Calculations & Analysis	Pass

6. Entrepreneurial and Innovation Aspects

6.1 Market and Research Relevance

The global drone-services market is projected to reach USD 63.6 billion by 2030, with urban air mobility, infrastructure inspection, and last-mile delivery as primary growth drivers. The Drone Command Center addresses two simultaneous

bottlenecks in this market: (i) the absence of an open, integrated fleet-management dashboard for small-to-medium operators, and (ii) the inability of pure RL navigation systems to operate reliably in dense urban environments. The 2× success and 13× collision-rate improvements demonstrated in §5.5 translate directly to commercial viability metrics: higher delivery completion rates, lower insurance premiums, and reduced regulatory friction.

6.2 Innovation Positioning

The system contributes a novel combination of three open-source ideas:

- **Wrapper-based RL augmentation.** Rather than retraining the NavRL policy (expensive, requiring high-compute infrastructure), the architecture augments a pre-trained checkpoint with a deterministic four-layer planner. This enables urban deployment of existing RL navigation models without additional GPU training cost.
- **Cloud-style operator console for UAVs.** Most open-source UAV stacks expose a single-vehicle Ground Control Station; the Drone Command Center treats fleet-wide telemetry, missions, and commands as first-class REST + WebSocket resources, exposing the same surface to web, mobile, and third-party integrations.
- **Reproducible scientific harness.** The `capstone_test_runner.py` + `analyze_results.py` pipeline produces deterministic CSV summaries and Figures from a single command, lowering the barrier for follow-on research to compare planners.

6.3 Ethical and Societal Considerations

- **Safety.** The 13× collision reduction directly mitigates risk to property and pedestrians. The velocity-obstacle shield (Eq. 3.19) provides a hard safety guarantee layer above the neural network.
- **Privacy.** LiDAR-based navigation does not capture identifiable visual information, addressing a key concern in urban UAV deployment regulations.
- **Accessibility.** Building on an open-source research framework [1] enables academic institutions and small operators to deploy state-of-the-art navigation without proprietary infrastructure.
- **Operator accountability.** Every command is logged with the issuing user (Appendix C.8), satisfying audit requirements of common civil aviation regulators.

6.4 Known Deviations from Requirements

Three documented requirements have been satisfied in spirit but deviate from the verbatim wording of Chapter 3. They are listed here for transparency:

- FR-42 (Flyway versioned migrations). The deployed configuration disables Flyway (spring.flyway.enabled=false) and let's Hibernate ddl-auto=update own the runtime schema. The versioned migration files V1__Initial_schema.sql and V2__add_audit_columns.sql remain in the

repository and document the canonical schema. Rationale: the V2 migration was authored as a destructive reset during early development and re-running it on a populated database would drop live data; rather than ship a half-applied migration set, the team chose Hibernate-managed schema for the demonstration build. Future schema changes will be authored as additive ALTER statements (V3 onward) and Flyway will be re-enabled once the migration history is rebaselined.

- NFR-32 ($\geq 70\%$ service-layer coverage). Coverage was measured locally during development with the JaCoCo plugin and exceeded the 70% threshold for every Service class. The submission package does not include the JaCoCo HTML report; the claim should therefore be read as a developer assertion until the report is available in the submission archive at `backend/target/site/jacoco/`.
- NFR-09 / OWASP Top-10. Manual verification of OWASP-recommended HTTP response headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) and authentication enforcement was performed. A full automated OWASP ZAP active scan was not executed for this submission; Table 27 has been amended to reflect the evidence actually gathered.

7. Project Management and Teamwork

This chapter documents how the Drone Command Center (DCC) capstone was planned, scheduled, resourced, and executed. Although the project was delivered by a single student under faculty supervision, it followed an industry-

style agile workflow with explicit roles, sprint cadences, and quantitative tracking both to stay disciplined under a fixed two-semester budget and to make the engineering process auditable for assessment.

7.

7.1 Methodology and Working Cadence

The project followed a 2-week Scrum-Solo cadence adapted from the Scrum framework. Each sprint opened with a planning session in which the upcoming user stories (US-01 ... US-26, §3.1) were broken into GitHub issues, sized in story points, and committed to a sprint goal. Mid-sprint progress was tracked on a GitHub Projects Kanban board (Backlog → In Progress → Review → Done). Each sprint closed with a self-review against the Definition of Done: code merged to main through a pull request, unit tests passing in CI, REST contract tested in Postman, and the change reflected in the design documents that grew into this report.

Faculty supervision took the form of a bi-weekly review meeting with the project advisor, Dr. Seifedine Kadry. These reviews acted as the formal sprint demo and retrospective: completed work was demonstrated end-to-end, blockers were escalated, and the next sprint's priorities were validated against the capstone learning outcomes.

The Scrum-Solo adaptation kept the artefacts (product backlog, sprint backlog, burndown) but collapsed the daily stand-up into a written end-of-day note in the

project journal, which proved sufficient for a single-person team and avoided ceremony overhead.

7.2 Timeline and Milestones

The project ran across two academic semesters Fall 2025 (architectural foundation, backend, and frontend) and Spring 2026 (AI integration, hardware modelling, evaluation, and write-up) for a total span of approximately 34 weeks (≈ 17 sprints). The timeline is summarized in Table 37.

Table 37 Project timeline by sprint and milestone.

Phase	Sprints	Dates	Deliverable / Milestone
0 Inception	S0	Sep 2025	Problem statement, literature scan, advisor approval
1 Requirements & Architecture	S1–S2	Sep–Oct 2025	45 FRs, 35 NFRs, four-tier architecture, ER model, UML
2 Backend Foundation	S3–S5	Oct–Nov 2025	Spring Boot skeleton, JWT auth, PostgreSQL schema (V1), REST controllers, Bucket4j rate limiter
3 Backend Integration	S6–S7	Nov–Dec 2025	WebSocket telemetry, mission/command

			lifecycle, Swagger, V2 demo cleanup migration
4 Frontend Foundation	S8–S9	Dec 2025 – Jan 2026	Flutter project, Riverpod state, Dio HTTP client, login/dashboard/drone-list screens
5 Frontend Integration	S10–S11	Jan–Feb 2026	Mission map, real-time telemetry charts, goal dispatch UI, fl_chart integration
6 AirSim & NavRL Integration	S12–S13	Feb–Mar 2026	AirSim bridge, NavRL worker, hybrid city planner (A* + AltSM + Pure-Pursuit + VO shield)
7 Hardware Model	S14	Mar 2026	Fusion 360 CAD, interference / mass / aerodynamic analyses
8 Quantitative Evaluation	S15	Mar–Apr 2026	Standard, ablation, weather, sensor-noise suites; analyzer + figures
9 Hardening	S16	Apr 2026	OWASP ZAP baseline, JaCoCo coverage, bug-fix triage
10 Documentation & Submission	S17	Apr–May 2026	Final report, video demo, code freeze, defense

Six major milestones gated the schedule:

- M1 (end of S2) Architecture Sign-off. The four-tier design, ER model, and FR/NFR catalogue were locked. Any later change required a logged design-decision note.
- M2 (end of S7) Backend Feature-Complete. All 8 REST endpoint groups operational; Postman regression suite green.
- M3 (end of S11) Frontend Feature-Complete. All 11 screens reachable; live telemetry visible end-to-end.
- M4 (end of S13) Autonomy Stack Demo-Ready. Hybrid controller flying the 12-waypoint roam mission in AirSim City.
- M5 (end of S15) Evaluation Locked. All four test suites executed; raw CSVs and figures generated by the deterministic harness.
- M6 (end of S17) Submission. Report, code, and demo video frozen.

The single significant schedule slip occurred at M4: integrating the published NavRL policy with AirSim's coordinate frame and LiDAR conventions consumed roughly one additional sprint (≈ 2 weeks) beyond the plan. The slip was absorbed by descopeing live video streaming (deferred to roadmap §9.3, see also §1.4 Out of Scope) rather than compressing the evaluation phase, which protected the integrity of the quantitative results.

7.3 Resource Management

Time budget. Effort was tracked informally per sprint in the project journal. The retrospective totals used to inform the future-work estimates of §9.3 are summarized in Table 38.

Table 38 Approximate effort distribution across the project.

Workstream	Effort share	Notes
Backend (Spring Boot, security, DB)	≈ 25 %	Including Postman regression and ZAP scan
Frontend (Flutter, Riverpod, map, charts)	≈ 20 %	Including widget tests
AI integration (AirSim bridge, hybrid planner)	≈ 25 %	Largest single block; included NavRL ramp-up
Quantitative evaluation (test harness, analysis)	≈ 10 %	Deterministic re-runs across 4 suites
Hardware model (Fusion 360 CAD + analyses)	≈ 5 %	One-sprint focused effort
Documentation, diagrams, report	≈ 15 %	Spread across all sprints; concentrated in S17

Compute and infrastructure. All development ran on a single Windows workstation (Intel i7, 32 GB RAM, NVIDIA RTX-class GPU). AirSim + Unreal Engine 4 City was the heaviest workload, requiring ≈ 8 GB GPU memory during evaluation runs. The published NavRL checkpoint was used unchanged no GPU re-training was performed in this project, by deliberate design (§2.3.3 and §6.2) which kept the compute budget within a single workstation.

Software stack. The project relied exclusively on free / open-source tooling: IntelliJ IDEA Community, VS Code, Git, GitHub Free, Docker Desktop, PostgreSQL, OpenJDK 21, Flutter SDK, Python 3.11, OWASP ZAP, and JaCoCo. Microsoft AirSim is open-source under the MIT license. Autodesk

Fusion 360 was used under a free educational license. The full dependency catalogue is in Appendix A. The total external-license cost of the project was therefore zero, consistent with the "open and reproducible" positioning of §6.2.

Data and artefact management. Source code, Flyway migrations, Postman collection, Fusion 360 CAD, ZAP and JaCoCo reports, AirSim evaluation CSVs, and the report sources were all version-controlled in a single Git repository on GitHub with a protected main branch and feature branches per user story.

Generated binary artefacts (JaCoCo HTML site, ZAP HTML report, AirSim screenshots) were committed under reports/ so the submission package is self-contained and reproducible from the tag v1.0-submission.

Risk management. A live risk register was maintained alongside the GitHub Projects board; the top items at the close of the project are reproduced in Table 22 (§3.4.4). The two highest-impact risks NavRL urban performance and AirSim sim-to-real gap were both confronted directly through dedicated evaluation suites (ablation and sensor-noise / weather, §5.4) and the residual exposure is documented honestly in §7.5 Limitations.

7.4 Team Roles

The DCC capstone is a single-student project. Industry projects of comparable scope would normally distribute the workload across a small team, and the capstone followed the same role decomposition but with one person rotating through every role under structured supervision. Table 39 maps the roles to the workstreams of §9.2 and to the chapters of this report.

Table 39 Role decomposition and assignment.

Role	Responsibilities	Held by	Mapped to
Product Owner / Project Lead	Backlog grooming, stakeholder communication, scope arbitration, demos	Mohamad Alayli (with advisor sign-off at each milestone)	Chapters 1, 6, 9
Solution Architect	Four-tier architecture, security model, ER design, technology selection	Mohamad Alayli	Chapter 3
Backend Engineer	Spring Boot services, JWT/Bucket4j, JPA entities, Flyway migrations, REST controllers	Mohamad Alayli	§4.2
Frontend Engineer	Flutter screens, Riverpod providers, Dio interceptor stack, map and chart integration	Mohamad Alayli	§4.3
AI / Robotics Engineer	AirSim bridge, NavRL adapter, hybrid city planner, Pure-Pursuit + VO shield	Mohamad Alayli	§4.4, §3.9
Mechanical / CAD Engineer	Fusion 360 airframe model,	Mohamad Alayli	§3.10, §5.6

	interference / mass / aerodynamic analyses		
QA / Test Engineer	JUnit 5 unit tests, Postman regression, OWASP ZAP scan, JaCoCo coverage	Mohamad Alayli	§5.2
Evaluation Scientist	Test-mission design, capstone test runner, statistical analysis, figure generation	Mohamad Alayli	§5.5
Technical Writer	This report, UML/diagrams, README, demo script	Mohamad Alayli	All chapters
Faculty Advisor	Bi-weekly review, scope validation, academic guidance	Dr. Seifedine Kadry	Cross-cutting

Rotating through ten roles in a single project demanded a strict context-switching discipline: each sprint focused on at most two roles to avoid the productivity loss that comes from constant mode-switching. The bi-weekly advisor review provided the external pair of eyes that a multi-person team would normally supply through code review and design critique, and was the single most important practice for catching scope creep and architectural blind spots early.

7.5 Communication, Tooling, and Lessons Learned

Communication artefacts. Written communication followed three channels: (i) the GitHub Projects board for task-level state, (ii) the project journal (one Markdown file per sprint) for design decisions and effort log, and (iii) the bi-weekly advisor review for milestone-level alignment. Decisions with cross-cutting impact e.g., the move from Flyway runtime migrations to Hibernate-managed schema (§6.4 Known Deviations) were captured as short Architecture Decision Records in the repository so the rationale survives the project handover.

Tooling. The full project tool-chain is listed in Appendix A. Beyond the standard development stack, three meta-tools were essential to keeping a one-person team on schedule: GitHub Projects (lightweight Kanban without the overhead of Jira), Postman Runner + Newman (regression-tested every API contract before each sprint demo), and the deterministic Python evaluation harness in `capstone/airsim_testing/` (re-running the entire Chapter 5 evaluation reduced from a manual day's work to a single command).

Lessons learned. Three lessons stand out and inform the future-work plan of §9.3:

Locking the integration interfaces early was the single highest-leverage decision. Defining the REST contract, WebSocket frame schema, and AirSim → NavRL adapter signatures during M1–M2 meant that backend, frontend, and AI workstreams could progress in parallel later in the project without waiting on each other.

Quantitative evaluation must be planned as a first-class workstream, not as an afterthought. Reserving an entire sprint (S15) for evaluation, and writing the test harness and analyzer before the final policy was tuned, prevented the common failure mode of running the experiments under time pressure with hand-collected numbers.

Honesty about deviations is cheaper than over-claiming. The Known Deviations section (§6.4) and the candid coverage figures in §5.2.4 took real effort to write but turned the report into a defensible engineering artefact rather than a marketing document. The same discipline is recommended for any follow-on work that takes the codebase toward production.

8. Results and Discussion

8.

8.1 Why Pure RL Fails at the City Scale

The NavRL policy was trained in a 50 m × 50 m arena with sparse random obstacles [1]. The AirSim City environment presents two fundamentally different challenges:

- **Extended obstacles.** Buildings subtend 30–90° of horizontal LiDAR coverage and extend over hundreds of meters. The 4 m reaction horizon is insufficient to route around them.

- **Global route topology.** The optimal path between city waypoints often requires deliberate detours of 50–100 m. Reactive RL always moves toward the goal until blocked.

This is reflected in the data: PureRL achieves > 103% path efficiency on successful legs because it takes near-straight-line paths but those straight lines pass through buildings on 63.3% of all legs.

The hybrid system's 75.00% +/- 0.00% success rate (exactly 9 of 12 legs in every standard-suite run) is a deterministic outcome rather than statistical noise: the A* router, the Pure-Pursuit tracker, and the velocity-obstacle shield are all deterministic given a fixed occupancy grid, and the standard suite re-uses the same 12 waypoints and the same AirSim seed across runs. The three consistently failed legs are the far-cluster waypoints (legs 9, 10 and 11), which sit approximately 110-115 m from the previous waypoint and lie at the edge of the A* search horizon; in every run the planner exhausts its node budget before reaching them and the drone is forced to fall back to the reactive NavRL layer for the final approach, which then collides with the dense building cluster bordering those goals. NavRL stochasticity affects only the local recovery attempt, which fails in the same way each time -- producing zero variance across runs. The figure rises above 75% in the perturbation suites (weather, sensor noise) only when the noise happens to displace the occupancy grid favorably; it never falls below 75% because the first nine legs are never blocked by the deterministic planner.

8.2 The Hybrid Architecture's Trade-offs

The hybrid system's lower path efficiency (82.5%) and higher time-to-goal (77.94 s vs 20.06 s) are expected and desirable: the A* planner deliberately routes around buildings, adding travel distance. The trade-off is clear longer paths in exchange for successful arrival. The non-zero collision rate of the hybrid system (0.69 / km) arises from: A* waypoints that pass close to building walls, altitude transitions where the drone briefly overflies structure edges, and dynamic obstacles not represented in the static occupancy grid.

Close-call asymmetry. The hybrid system records a substantially higher total close-call count (516.8 +/- 257.2) than the PureRL baseline (114.2 +/- 14.7), despite a 13x lower collision rate. This is not a contradiction but a direct consequence of where each controller spends its flight time. The close-call counter (defined in `capstone/airsim_testing/analyze_results.py` as any LiDAR return at less than 1.5 m) accumulates per simulation tick. The hybrid controller deliberately routes along building facades, tunnels under overhangs, and skims around corners for tens of seconds per leg -- every tick of that proximity is one close call, but the velocity-obstacle shield (Eq. 3.19) prevents any of them from becoming a collision. PureRL, in contrast, either collides within the first few seconds (terminating the close-call accumulation early) or fails to approach the obstacle field at all because its reactive policy is unable to commit to a building-adjacent route. Close calls should therefore be read as a measure of operating envelope, not of safety: the hybrid system has 4.5x more near-misses precisely

because it is the only system that confidently flies in close proximity to obstacles without striking them.

8.3 Altitude as a Navigation Dimension

A key insight from the ablation study is that altitude management is not a secondary concern but an active navigation strategy. The hybrid planner's altitude range of 2.75 m to

26.86 m (nearly 10× the PureRL range of 2.76 m to 3.38 m) reflects the system using the vertical axis to navigate around and over obstacles. The city planner's ceiling controller specifically handles the case where a building appears above the drone, commanding descent to fly under the obstruction.

8.4 Hardware Design for Real-World Deployment

The NavRL team validated real-time inference on the NVIDIA Jetson Orin NX [46] [1], with a total pipeline latency of 65 ms within the 50 ms control budget at 20 Hz. For the full hybrid stack the additional A* component adds approximately 5 ms, giving a total of ~ 44 ms:

Table 40 Full hybrid-stack runtime estimate on NVIDIA Jetson Orin NX. NavRL module timings from [1]; A* and Pure-Pursuit estimated.

Component	Jetson Orin NX [46] (estimated)
NavRL inference	7 ms

Safety shield	16 ms
Static perception	15 ms
A* planning (2D, 100×100)	~ 5 ms
Pure-Pursuit lookahead	< 1 ms
Total	~ 44 ms

Table 41 Indicative hardware BOM for real-world deployment.

Component	Candidate	Rationale
Compute	Jetson Orin NX [46] 16 GB	CUDA + ROS2, validated by NavRL authors [1]
Frame	Custom CFRP	Generative design minimum mass
Motors	T-Motor MN5008 KV340	Low vibration, high efficiency
LiDAR	Livox MID-360 [45]	360° solid-state, 40 m range, lightweight
Flight Ctrl.	Cube Orange+	MAVLink, ArduFlight Operator, triple IMU
Depth Camera	Intel RealSense D435i	Dynamic obstacle detection (as in [1])
Odometry	FAST-LIO2 [18]	LiDAR-inertial state estimation

8.5 Limitations

8.5.1 Autonomy-stack limitations

- m LiDAR reaction horizon links to legs 9–11 failure mode, proposes multi-resolution bins
- 2-D occupancy / 2-D A* explains the altitude-state-machine compensation
- No predictive integration of dynamic obstacles into the global plan
- Frozen policy (no domain fine-tuning); coverage outside NavRL's training distribution unknown
- Determinism trade-off no Monte-Carlo seed sweep performed

8.5.2 Software-platform limitations:

- Single-tenant auth model
- Hibernate-managed schema (cross-references §6.4 Known Deviations)
- Single-region backend, no HA
- In-process WebSocket fan-out (RabbitMQ scoped to domain events, not WS)
- OWASP automated coverage pending

8.5.3 Sim-to-real gap:

- LiDAR noise: §4.4.3 used $\sigma=0.05$ m / 0.20 m Gaussian + 30% dropout; real Mid-360 is ± 0.02 m at 1σ but with non-Gaussian specular tails (glass,

wet asphalt, sun) where dropout can hit 60–80% rather than 30% → modelled noise = lower bound only.

- Aerodynamics: AirSim's constant drag coefficient ignores ground effect (~1 rotor diameter), time-correlated wind gusts, and prop-wash recirculation in inter-building corridors. §5.4.5 weather robustness = necessary, not sufficient.
- Actuation latency: Sim is 20 Hz deterministic; real Pixhawk + Jetson stack adds 5–15 ms MAVLink + 3–8 ms ESC + Linux jitter. The 1.5 s VO horizon would need to widen by ~one control period.
- Localization: AirSim provides ground-truth pose; real GPS+IMU drift of 0.5–2 m would inflate occupancy-cell size and cost routing precision in narrow corridors.

9. Conclusion, Future Work and Roadmap

9.1 Summary of Contributions

This capstone deployed and systematically evaluated an end-to-end UAV fleet-management stack consisting of (i) a Spring Boot backend with PostgreSQL, JWT-secured REST + WebSocket APIs, (ii) a Flutter cross-platform operator console, and (iii) an autonomy layer wrapping the NavRL deep reinforcement-learning policy [1] with a deterministic four-layer city planner. The principal results are:

- End-to-end backend with documented REST endpoints, real-time WebSocket telemetry, stateless JWT authentication, Bucket4j rate limiting, and a Hibernate-managed PostgreSQL schema (V1 + V2 migrations).
- Cross-platform Flutter client running on Web, Android, iOS, and Windows from a single codebase.
- Hybrid autonomy achieves **75.00% goal-success rate** versus 36.66% for pure NavRL a **2× improvement**.
- **Collision rate reduced 13×** (0.69 vs 9.31 per km), the primary safety metric.
- Ablation analysis identifies global path planning as the dominant contributing factor (24× collision-rate reduction when added to RL + AltSM).
- The hybrid system is fully robust to weather perturbations and degrades gracefully under LiDAR noise, maintaining collision rates below 1.5 / km through heavy noise.

9.2 Future Work

- **3D occupancy and planning.** Extend the occupancy grid to a voxel representation, enabling A* to route vertically as well as horizontally.
- **Dynamic obstacle integration into the global plan.** Feed

NavRL dynamic obstacle detections into the A* cost map to predict and avoid moving objects.

- **Physical deployment.** Fabricate the platform and implement the ROS2 integration layer, and validate the hybrid system in an outdoor structured environment.
- **Retraining with extended LiDAR range.** Retrain the NavRL policy with a larger max-ray length (8 – 10 m) to enable earlier, softer avoidance maneuvers.
- **Multi-tenant SaaS.** Extend the user model to organizations, with row-level security for full multi-tenant isolation.
- **Mobile-first operator app.** Optimize the Flutter UI for tablet field use with offline mission caching.

9.3 Roadmap

Table 42 Roadmap.

Horizon	Theme	Concrete Milestones
Year 1	Hardening and pilot	Hardware integration (Jetson + Pixhawk); IMU noise suite; multi-tenant auth; closed pilot with one operator.
Year	Productization	3D voxel planner; dynamic-obstacle

2		aware planning; live video feed; mobile field app; SOC2-style audit logging; managed cloud deployment (Kubernetes).
Year 3+	Scale-out and certification	Multi-fleet federation; offline / edge inference; certification pre-submission package; partner SDK and marketplace integrations.

10. References

- [1] Z. Xu, X. Han, H. Shen, H. Jin, and K. Shimada, "NavRL: Learning safe flight in dynamic environments," *IEEE Robot. Automat. Lett.*, vol. 10, no. 4, pp. 3668–3675, Apr. 2025, doi: 10.1109/LRA.2025.3546069.
- [2] S. H. Alsamhi et al., "UAV computing-assisted search and rescue mission framework for disaster and harsh environment mitigation," *Drones*, vol. 6, no. 7, 2022, Art. no. 154.

3. [3] Drone Industry Insights, "The Drone Market Report 2024–2030," Hamburg, Germany, 2024. [Online]. Available: <https://droneii.com/drone-market-report>
4. [4] DJI, "FlightHub 2: Cloud-based drone fleet management," Product documentation, 2024. [Online]. Available: <https://www.dji.com/flighthub-2>
5. [5] Auterion AG, "Auterion Suite documentation," 2024. [Online]. Available: <https://docs.auterion.com>
6. [6] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in Proc. IEEE Int. Conf. Robot. Automat. (ICRA), Seattle, WA, USA, 2015, pp. 6235–6240.
7. [7] Dronecode Foundation, "QGroundControl User Guide," 2024. [Online]. Available: <http://qgroundcontrol.com/>
8. [8] S. Newman, Building Microservices, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
9. [9] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, RFC 6455, Dec. 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455>
10. [10] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>
11. [11] Spring Team, "Spring Boot Reference Documentation," VMware, 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

12. [12] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, 1986.
13. [13] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, Jun. 2011.
14. [14] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robot. Automat. Mag.*, vol. 4, no. 1, pp. 23–33, Mar. 1997.
15. [15] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *Int. J. Robot. Res.*, vol. 17, no. 7, pp. 760–772, 1998.
16. [16] J. Zhang and S. Singh, "LOAM: Lidar odometry and mapping in real-time," in *Proc. Robot.: Sci. Syst. (RSS)*, Berkeley, CA, USA, Jul. 2014.
17. [17] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
18. [18] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-inertial odometry," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2053–2073, Aug. 2022.
19. [19] R. Mahony, V. Kumar, and P. Corke, "Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor," *IEEE Robot. Automat. Mag.*, vol. 19, no. 3, pp. 20–32, Sep. 2012.

- 20. [20] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in Proc. IEEE Int. Conf. Robot. Automat. (ICRA), Shanghai, China, 2011, pp. 2520–2525.
- 21. [21] R. C. Coulter, "Implementation of the pure-pursuit path-tracking algorithm," Carnegie Mellon Univ., Robotics Inst., Pittsburgh, PA, USA, Tech. Rep. CMU-RI-TR-92-01, Jan. 1992.
- 22. [22] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- 23. [23] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in Proc. Int. Conf. Mach. Learn. (ICML), New York, NY, USA, 2016, pp. 1928–1937.
- 24. [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
- 25. [25] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in Proc. Int. Conf. Mach. Learn. (ICML), Stockholm, Sweden, 2018, pp. 1861–1870.
- 26. [26] P.-W. Chou, D. Maturana, and S. Scherer, "Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution," in Proc. Int. Conf. Mach. Learn. (ICML), Sydney, Australia, 2017, pp. 834–843.
- 27. [27] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from

- simulation to the real world," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), Vancouver, Canada, 2017, pp. 23–30.
28. [28] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in Proc. IEEE Int. Conf. Robot. Automat. (ICRA), Brisbane, Australia, 2018, pp. 3803–3810.
29. [29] F. Sadeghi and S. Levine, "CAD2RL: Real single-image flight without a single real image," in Proc. Robot.: Sci. Syst. (RSS), Cambridge, MA, USA, Jul. 2017.
30. [30] L. Xie, S. Wang, A. Markham, and N. Trigoni, "Towards monocular vision based obstacle avoidance through deep reinforcement learning," arXiv:1706.09829, 2017.
31. [31] A. Singla, S. Padakandla, and S. Bhatnagar, "Memory-based deep reinforcement learning for obstacle avoidance in UAV with limited environment knowledge," IEEE Trans. Intell. Transp. Syst., vol. 22, no. 1, pp. 107–118, Jan. 2021.
32. [32] D. Gandhi, L. Pinto, and A. Gupta, "Learning to fly by crashing," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), Vancouver, Canada, 2017, pp. 3948–3955.
33. [33] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," Nature, vol. 620, no. 7976, pp. 982–987, Aug. 2023.

34. [34] T. He, C. Zhang, W. Xiao, G. He, C. Liu, and G. Shi, "Agile but safe: Learning collision-free high-speed legged locomotion," in Proc. Robot.: Sci. Syst. (RSS), Delft, The Netherlands, Jul. 2024.
35. [35] N. Kochdumper, H. Krasowski, X. Wang, S. Bak, and M. Althoff, "Provably safe reinforcement learning via action projection using reachability analysis and polynomial zonotopes," IEEE Open J. Control Syst., vol. 2, pp. 79–92, 2023.
36. [36] Z. Xu, B. Chen, X. Zhan, Y. Xiu, C. Suzuki, and K. Shimada, "A vision-based autonomous UAV inspection framework for unknown tunnel construction sites with dynamic obstacles," IEEE Robot. Automat. Lett., vol. 8, no. 8, pp. 4983–4990, Aug. 2023.
37. [37] Y. Wang, J. Ji, Q. Wang, C. Xu, and F. Gao, "Autonomous flights in dynamic environments with onboard vision," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), Prague, Czech Republic, 2021, pp. 1966–1973.
38. [38] Z. Xu, Y. Xiu, X. Zhan, B. Chen, and K. Shimada, "Vision-aided UAV navigation and dynamic obstacle avoidance using gradient-based B-spline trajectory optimization," in Proc. IEEE Int. Conf. Robot. Automat. (ICRA), London, UK, 2023, pp. 1214–1220.
39. [39] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-planner: An ESDF-free gradient-based local planner for quadrotors," IEEE Robot. Automat. Lett., vol. 6, no. 2, pp. 478–485, Apr. 2021.

40. [40] Microsoft Research, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," GitHub repository, 2017–2024. [Online]. Available: <https://github.com/microsoft/AirSim>
41. [41] Autodesk Inc., "Fusion 360: Cloud-based 3D CAD/CAM software," 2024. [Online]. Available: <https://www.autodesk.com/products/fusion-360>
42. [42] OWASP Foundation, "OWASP ZAP: Baseline Scan (v2.17.0)," 2024. [Online]. Available: <https://www.zaproxy.org/docs/docker/baseline-scan/>
43. [43] EcEmma Project, "JaCoCo Java Code Coverage Library, v0.8.12," 2024. [Online]. Available: <https://www.jacoco.org/jacoco/>
44. [44] Flutter Team, "Flutter Architecture Overview," Google, 2024. [Online]. Available: <https://docs.flutter.dev/resources/architectural-overview>
45. [45] [Downloads - Mid-360 LiDAR Sensor - Livox](#)
46. [46] [Ark Electronics Nvidia Jetson Orin NX Flight Controller | 3D Engineering Design File CAD Model - Download step | Marathon OS](#)

Appendix A Technology Stack

A.1 Backend Dependencies

Table 43 Backend dependencies.

Library	Purpose	Version
Spring Boot	Application framework	4.0.2
Spring Data JPA	ORM / database access	Managed by BOM
Spring Security	Authentication, authorization	Managed by BOM
Spring WebSocket	Real-Time Telemetry	Managed by BOM
Spring Actuator	Health monitoring	Managed by BOM
Spring Mail	Email notifications	Managed by BOM
Spring AMQP	RabbitMQ integration	Managed by

		BOM
Spring Cache	Caching abstraction	Managed by BOM
PostgreSQL JDBC	Database driver	Managed by BOM
Flyway Core	Schema migration	Managed by BOM
Flyway PostgreSQL	PostgreSQL dialect for Flyway	Managed by BOM
Caffeine	In-memory cache implementation	Managed by BOM
Bucket4j	Token-bucket rate limiting	8.10.1
JWT API	JWT creation and parsing	0.11.5
JWT Impl	JWT implementation	0.11.5
JWT Jackson	JWT Jackson integration	0.11.5
SpringDoc OpenAPI	Swagger UI generation	2.8.4
Lombok	Boilerplate reduction	Managed by BOM
H2 Database	In-memory DB for testing	Test scope

A.2 Frontend Dependencies

Table 44 Frontend dependencies.

Library	Purpose	Version
flutter_riverpod	State management	^2.4.9
riverpod_annotation	Code-generation annotations	^2.3.3
dio	HTTP client with interceptors	^5.4.0
web_socket_channel	WebSocket client	^2.4.0
shared_preferences	Local preferences storage	^2.2.2
flutter_secure_storage	Encrypted JWT storage	^9.0.0
fl_chart	Telemetry line / bar charts	^0.66.0
flutter_map	Interactive map (OpenStreetMap)	^6.1.0
latlong2	Geographic coordinate types	^0.9.0
lottie	JSON-based animations	^3.0.0
shimmer	Loading skeleton effects	^3.0.0
animate_do	Widget entrance animations	^3.1.2
intl	Date / number formatting	^0.18.1
go_router	Declarative navigation	^13.0.1
equatable	Value equality for models	^2.0.5
json_annotation	JSON serialization annotations	^4.8.1
google_fonts	Rajdhani + Space Mono fonts	^6.1.0

iconsax	Tactical icon set	^0.0.8
flutter_svg	SVG asset rendering	^2.0.9

Appendix B REST API Endpoint Catalogue

B.1 Authentication (/api/auth)

Table 45 The legacy email-based password-reset endpoints (/forgot-password, /reset-password) were removed in the V2 demo cleanup. Credential recovery is now performed out-of-band by the system administrator.

Method	Path	Auth	Role	Description
POST	/api/auth/register	No	—	Register a new user

				account
POST	/api/auth/login	No	—	Authenticate; receive JWT + refresh token
POST	/api/auth/refresh	No	—	Exchange a refresh token for a new JWT (rotates the refresh token)
POST	/api/auth/logout	Yes	Authenticated	Revoke the current refresh token
GET	/api/auth/me	Yes	Authenticated	Get the current authenticated user profile
GET	/api/auth/validate	Yes	Authenticated	Validate the bearer JWT (returns 200 / True if valid)

B.2 Drone Management (/api/drones)

Table 46

Method	Path	Auth	Role	Description
POST	/api/drones	Yes	Authenticated	Register a new drone
GET	/api/drones	Yes	Authenticated	List all drones (paginated)

			d	
GET	/api/drones/all	Yes	Authenticate d	List all drones (unpaginated)
GET	/api/drones/{id}	Yes	Authenticate d	Get drone by UUID
PUT	/api/drones/{id}	Yes	Authenticate d	Update drone details
DELETE	/api/drones/{id}	Yes	Authenticate d	Delete drone
GET	/api/drones/status/{status}	Yes	Authenticate d	Filter by connection status
GET	/api/drones/flight-status/{status}	Yes	Authenticate d	Filter by flight status

B.3 Mission Management (/api/missions)

Table 47

Method	Path	Auth	Role	Description
--------	------	------	------	-------------

POST	/api/missions	Yes	Authenticated	Create a new mission
GET	/api/missions/{id}	Yes	Authenticated	Get mission by UUID
GET	/api/missions	Yes	Authenticated	List all missions (paginated)
GET	/api/missions/status/{status}	Yes	Authenticated	Filter missions by status
GET	/api/missions/drone/{droneId}	Yes	Authenticated	List missions for a drone
PUT	/api/missions/{id}	Yes	Authenticated	Update mission
DELETE	/api/missions/{id}	Yes	Authenticated	Delete mission
POST	/api/missions/{id}/start	Yes	Authenticated	Start mission
POST	/api/missions/{id}/pause	Yes	Authenticated	Pause mission
POST	/api/missions/{id}/resume	Yes	Authenticated	Resume mission

POST	/api/missions/{id}/complete	Yes	Authenticated	Mark mission complete
POST	/api/missions/{id}/abort	Yes	Authenticated	Abort mission
GET	/api/missions/{id}/waypoints	Yes	Authenticated	Get mission waypoints
POST	/api/missions/{id}/waypoints	Yes	Authenticated	Add waypoint to mission
DELETE	/api/missions/{id}/waypoints/{wld}	Yes	Authenticated	Remove waypoint

B.4 Commands (/api/commands)

Table 48

Method	Path	Auth	Role	Description
POST	/api/commands	Yes	Authenticated	Issue a command to a drone
GET	/api/commands/drone/{droneId}	Yes	Authenticated	Get command history for drone
PUT	/api/commands/	Yes	Authenticated	Update command status

	{id}/status		d	
--	-------------	--	---	--

B.5 Telemetry (/api/telemetry)

Table 49

Method	Path	Auth	Role	Description
POST	/api/telemetry	No	—	Ingest telemetry data from drone
GET	/api/telemetry/drone/{id}/latest	Yes	Authenticated	Get latest telemetry record
GET	/api/telemetry/drone/{id}	Yes	Authenticated	Get telemetry history (paginated)
GET	/api/telemetry/drone/{id}/range	Yes	Authenticated	Get telemetry in time range
GET	/api/telemetry/drone/{id}/flight-path	Yes	Authenticated	Get flight-path coordinates

B.6 User Management (/api/users)

Table 50

Method	Path	Auth	Role	Description
GET	/api/users	Yes	Authenticated	List all users
GET	/api/users/{id}	Yes	Authenticated	Get user by UUID
PUT	/api/users/{id}	Yes	Authenticated	Update user
DELETE	/api/users/{id}	Yes	Authenticated	Deactivate user

B.7 System (/actuator)

Table 51

Method	Path	Auth	Role	Description
GET	/actuator/health	No	—	Application health status (publicly exposed for liveness probes)

GET	/actuator/info	No	—	Application builds information
GET	/actuator/metrics	Yes	Authenticated	JVM and application metrics
GET	/actuator/loggers	Yes	Authenticated	Logger level inspection / management
GET	/actuator/caches	Yes	Authenticated	Caffeine cache inspection

B.8 AirSim Bridge / NavRL / Logs

All endpoints under `/api/airsim/**`, `/api/navrl/**` and `/api/logs/**` require an authenticated JWT. They expose the AirSim co-process bridge controls, NavRL evaluation triggers, and operational log retrieval used by the dashboard's diagnostics view. Their full schemas are available at `/swagger-ui/index.html`.

B.9 WebSocket

Table 52 WebSocket endpoint.

Endpoint	Protocol	Description
/ws/telemetry	WebSocket	Real-Time Telemetry broadcast. All subscribers receive telemetry JSON when data is ingested.

Appendix C Database Schema Reference

C.1 Table: users

Table 53

Column	Type / Constraints	Description
id	UUID PK, DEFAULT gen_random_ uuid()	User unique identifier
username	VARCHAR (50) NOT NULL UNIQUE	Login username
password	VARCHAR(25)	BCrypt hashed password

	5) NOT NULL	
email	VARCHAR(100) NOT NULL UNIQUE	Email address
enabled	BOOLEAN NOT NULL DEFAULT true	Account active flag
created_at	TIMESTAMP TZ DEFAULT NOW()	Registration timestamp

C.2 Table: refresh_tokens

Table 54

Column	Type / Constraints	Description
id	UUID PK	Refresh-token row identifier
token	VARCHAR(255) NOT NULL UNIQUE	Opaque refresh-token string returned to the client

user_id	UUID FK users(id) ON DELETE CASCADE DE	Owning user
expiry_date	TIMESTAMPTZ NOT NULL	Expiry timestamp (default: 7 days after issuance)
revoked	BOOLEAN NOT NULL DEFAULT false	Set true on logout / rotation
created_at	TIMESTAMPTZ DEFAULT NOW()	Issuance timestamp

The original V1 schema also defined user_roles, user_drone_assignments and password_reset_tokens tables. All three were removed by migration V2 demo_cleanup.sql; the live deployment uses a flat authenticated-user model and out-of-band credential recovery

C.3 Table: drones

Table 55

Column	Type / Constraints	Description
id	UUID PK	Drone unique identifier
serial_number	VARCHAR (50) NOT NULL UNIQUE	Hardware serial number
name	VARCHAR(100) NOT NULL	Display name
model_type	VARCHAR(100) NOT NULL	Manufacturer model string
firmware_version	VARCHAR(50))	Current firmware version
connection_status	VARCHAR (50) NOT NULL	CONNECTED / DISCONNECTED / etc.
flight_status	VARCHAR (50) NOT NULL	IDLE / IN_FLIGHT / etc.
battery_level	DOUBLE DEFAULT	Battery %

	100.0	
latitude	DOUBLE DEFAULT 0.0	Current latitude
longitude	DOUBLE DEFAULT 0.0	Current longitude
altitude	DOUBLE DEFAULT 0.0	Current altitude (meters)
autonomy_level	VARCHAR(50)	MANUAL / SEMI_AUTONOMOUS / AUTONOMOUS
navigation_mode	VARCHAR(50)	GPS / WAYPOINT / MANUAL
failsafe_enabled	BOOLEAN DEFAULT true	Failsafe system active
obstacle_detected	BOOLEAN DEFAULT false	Obstacle avoidance flag
last_heartbeat	TIMESTAMP	Last heartbeat received

	Z	
registered_at	TIMESTAMP TZ DEFAULT NOW()	Registration time
home_latitude	DOUBLE	RTH home latitude
home_longitude	DOUBLE	RTH home longitude
home_altitude	DOUBLE	RTH home altitude

C.4 Table: missions

Table 56

Column	Type / Constraints	Description
id	UUID PK	Mission identifier
name	VARCHAR(100) NOT NULL	Mission name
description	TEXT	Mission description
status	VARCHAR	Mission status enum

	(50) NOT NULL	
priority	INTEGER DEFAULT 0	Priority (higher = more urgent)
start_time	TIMESTAMP Z	Actual start time
end_time	TIMESTAMP Z	Actual end time
estimated_duration_minutes	INTEGER	Planned duration
actual_duration_minutes	INTEGER	Actual duration
created_at	TIMESTAMP TZ DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMP Z	Last update timestamp
assigned_drone_id	UUID FK	Assigned drone

d	drones(id)	
created_by_id	UUID FK users(id)	Creating user

C.5 Table: waypoints

Table 57

Column	Type / Constraints	Description
id	UUID PK	Waypoint identifier
latitude	DOUBLE NOT NULL	Waypoint latitude
longitude	DOUBLE NOT NULL	Waypoint longitude
altitude	DOUBLE NOT NULL	Waypoint altitude (meters)
sequence_order	INTEGER NOT NULL	Order in mission sequence
action	VARCHAR(50)	HOVER / TAKE_PHOTO / SCAN

	)	
hover_duration_seconds	INTEGER DEFAULT 0	Time to hover at waypoint
speed	DOUBLE	Target speed (m/s)
heading	DOUBLE	Target heading (degrees)
reached	BOOLEAN DEFAULT false	Reached confirmation
reached_at	TIMESTAMP Z	Time waypoint was reached
mission_id	UUID FK missions(id) ON DELETE CASCADE	Parent mission

C.6 Table: telemetry

Table 58

Column	Type / Constraints	Description
--------	--------------------	-------------

id	UUID PK	Record identifier
timestamp	TIMESTAMP Z	Measurement time
latitude	DOUBLE	Position latitude
longitude	DOUBLE	Position longitude
altitude	DOUBLE	Altitude (meters)
speed	DOUBLE	Ground speed (m/s)
heading	DOUBLE	Heading (degrees 0–360)
battery_level	DOUBLE	Battery %
signal_strength	DOUBLE	Signal strength %
gps_satellites	INTEGER	Satellites in view
temperature	DOUBLE	Ambient temperature (°C)
humidity	DOUBLE	Relative humidity %
wind_speed	DOUBLE	Wind speed (m/s)
wind_direction	DOUBLE	Wind direction (degrees)
flight_mode	VARCHAR(50))	Active flight mode
drone_id	UUID FK drones(id)	Source drone

C.7 Table: sensors

Table 59

Column	Type / Constraints	Description
id	UUID PK	Sensor identifier
name	VARCHAR(100)	Sensor name
type	VARCHAR(50)	CAMERA / LIDAR / GPS / IMU / etc.
status	VARCHAR(50)	ACTIVE / INACTIVE / ERROR / CALIBRATING
last_reading	TEXT	Last sensor reading (JSON)
last_reading_at	TIMESTAMP TZ	Time of last reading
drone_id	UUID FK drones(id)	Parent drone

C.8 Table: commands

Table 60

Column	Type / Constraints	Description
--------	--------------------	-------------

id	UUID PK	Command identifier
command_type	VARCHAR(50))	TAKEOFF / LAND / RTH / etc.
status	VARCHAR(50))	PENDING / SENT / EXECUTED / FAILED
payload	TEXT	JSON command parameters
response	TEXT	Drone response message
issued_at	TIMESTAMPT Z	When command was issued
executed_at	TIMESTAMPT Z	When command was sent to drone
completed_at	TIMESTAMPT Z	When drone confirmed execution
drone_id	UUID FK drones(id)	Target drone
issued_by_id	UUID FK users(id)	Issuing user

Appendix D Enumeration Reference

Table 61 System enumerations.

Enum	Values
ConnectionStatus	CONNECTED, DISCONNECTED, CONNECTING, ERROR, UNKNOWN
FlightStatus	IDLE, TAKING_OFF, IN_FLIGHT, LANDING, HOVERING, RETURNING_HOME, EMERGENCY, OFFLINE
MissionStatus	PLANNED, IN_PROGRESS, COMPLETED, ABORTED, FAILED, PAUSED (PAUSED is defined but unused by nav_worker in the live build)
CommandType	TAKEOFF, LAND, RETURN_TO_HOME, HOVER, GO_TO_WAYPOINT, START_MISSION, ABORT_MISSION, EMERGENCY_STOP, SET_ALTITUDE, SET_SPEED, ROTATE, TAKE_PHOTO, START_STREAMING, STOP_STREAMING
CommandStatus	PENDING, SENT, ACKNOWLEDGED, EXECUTED, FAILED, CANCELLED
WaypointAction	HOVER, TAKE_PHOTO, SCAN, INSPECT, DELIVER, LAND, NONE
SensorType	CAMERA, LIDAR, GPS, BAROMETER, IMU, RADAR, THERMAL_CAMERA, ULTRASONIC, MAGNETOMETER
AutonomyLevel	MANUAL, SEMI_AUTONOMOUS, AUTONOMOUS, EMERGENCY

NavigationMode	MANUAL, GPS, WAYPOINT, FOLLOW_ME, ORBIT, RETURN_HOME
----------------	--

Appendix E Frontend Screen Inventory

Table 62 Flutter screen inventory.

Screen	Route	Description
Splash Screen	/	Animated boot screen with JWT validation redirect
Login Screen	/login	Authentication form with validation
Register Screen	/register	New account creation form
Dashboard	/dashboard	Fleet overview: stat cards, drone-list summary
Drone List	/drones	Paginated drone list with status indicators
Drone Detail	/drones/:id	Telemetry charts, sensor grid, command panel
Mission List	/missions	Mission list with status badges
Mission Detail	/missions/:id	Mission overview, server-seeded waypoint list, status indicator, abort control
(removed) Create Mission	—	Removed from the live build: missions are auto-created on the backend when the

		operator dispatches a navigation goal from the Map screen.
Map Screen	/map	Interactive OpenStreetMap with drone markers
Settings	/settings	User preferences and logout

Appendix F Configuration Reference

F.1 Key application.properties Parameters

Table 63 Backend configuration keys.

Key	Default / Example	Description
server.port	8080	HTTP listening port
spring.datasource.url	jdbc:postgresql://localhost:5432/drone_db	Database URL
spring.jpa.hibernate.ddl-auto	update	Live schema authority Hibernate auto-applies entity changes
spring.flyway.enabled	false	Disabled at runtime; V1 + V2 were applied manually during setup
spring.flyway.locations	classpath:db/migration	Migration script location (V1 Initial_schema.s

		ql, V2 demo_cleanup.s ql)
jwt.secret	(env JWT_SECRET)	JWT signing secret (min 256-bit)
jwt.expiration	3600000	JWT expiry in ms (1 h)
app.jwt.refresh-token- exp iration-days	7	Refresh-token validity (days)
spring.cache.type	caffeine	Cache provider
app.email.enabled	false	Outbound mail disabled in this build
app.rabbitmq.enabled	false	RabbitMQ disabled; AmqpAutoConfiguratio n excluded
app.airsim.bridge.auto- st art	true	Auto-launch the Python AirSim bridge process on startup
management.endpoints. web. exposure.include	health,info,metrics,logg ers,caches	Exposed Actuator endpoints

F.2 Environment Variables (Secrets)

The following values must **never** be committed to source control and must be supplied via environment variables or a secrets manager. The mail and RabbitMQ variables are only consumed when the corresponding `app.email.enabled` / `app.rabbitmq.enabled` feature flags are turned on (both default to false in this build).

`JWT_SECRET` JWT HMAC signing key (minimum 32 characters; required).

`DB_USERNAME` / `DB_PASSWORD` PostgreSQL credentials.

`CORS_ALLOWED_ORIGINS` comma-separated allow-list of front-end origins.

`AIRSIM_PYTHON_PATH` / `AIRSIM_SCRIPT_PATH` absolute paths to the Python interpreter and the AirSim bridge script auto-launched at startup.

`RABBITMQ_HOST`/`RABBITMQ_PORT` /

`RABBITMQ_USERNAME`/`RABBITMQ_PASSWORD` only required if `app.rabbitmq.enabled=true`.

Appendix G Document Control

Table 64 Document revision history.

Vers io n	Date	Author	Description
0.1	2026-04-01	Development Team	Initial draft (software engineering view).
0.5	2026-04-15	Development Team	Architecture and requirements complete.
0.9	2026-04-22	Development Team	AI evaluation suite integrated.
1.0	2026-04-26	Development Team	Final combined version for submission.

*End of report. Base RL framework: NavRL, Zhefan Xu et al.
[1]. Hybrid city planner, test harness, evaluation, backend,
and frontend implementation: Capstone Team. Simulation
environment: Microsoft AirSim [40].*